

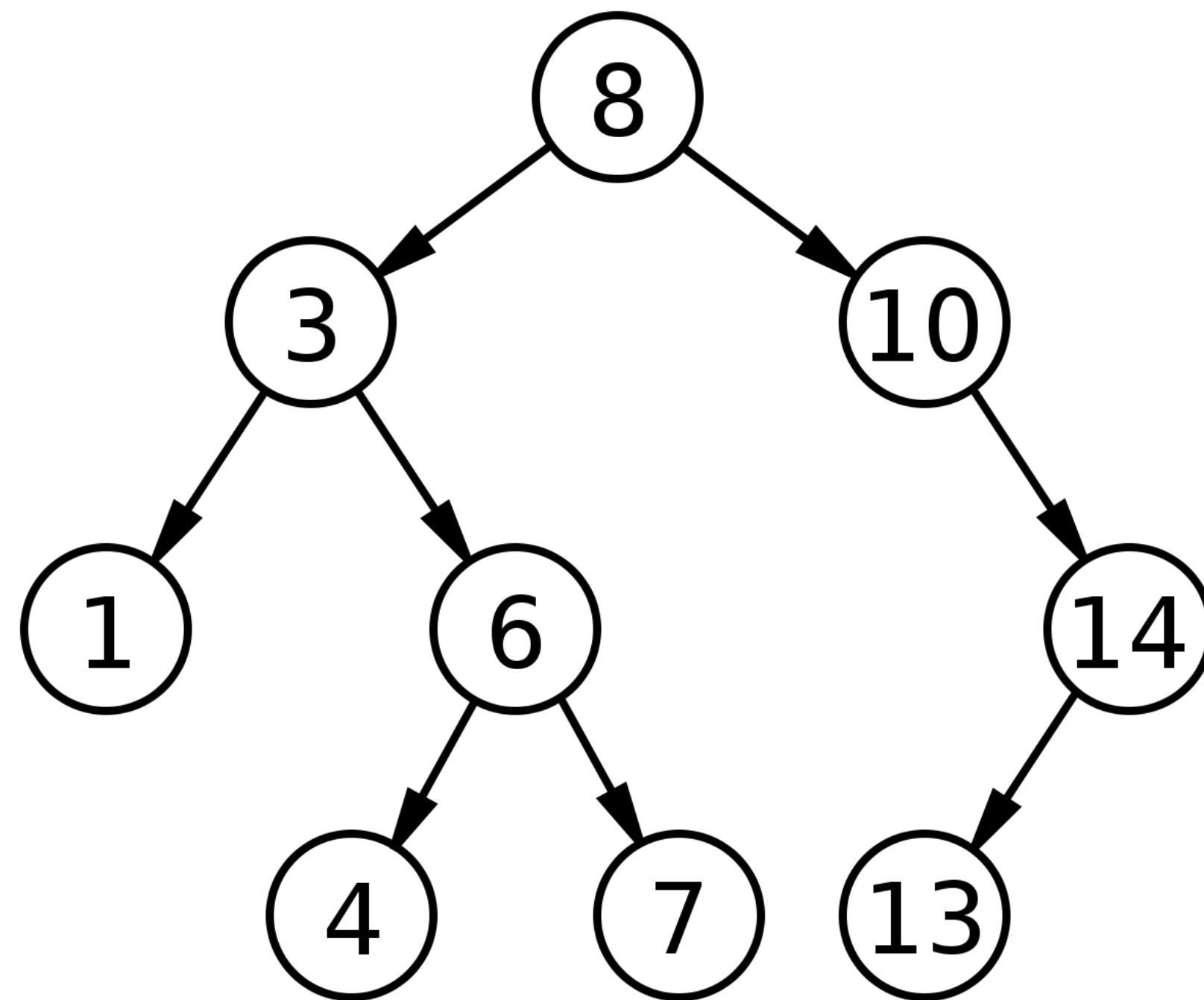
Wavelet Trees: Extending Rank, Select and Access to non-binary alphabets



Content adopted from www.langmead-lab.org/teaching-materials.

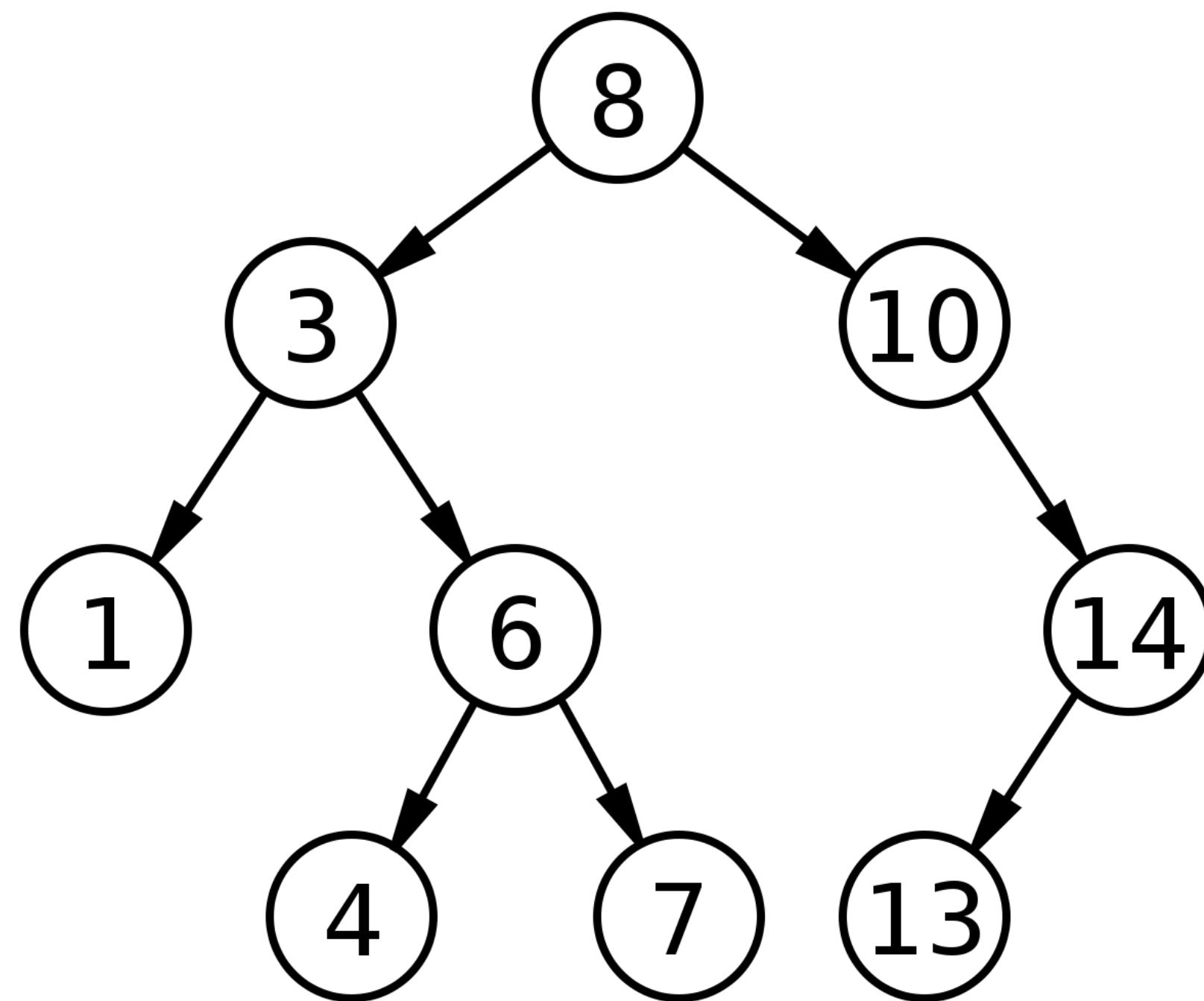
Trees

We're used to binary trees that repeatedly partition "value space"



Trees

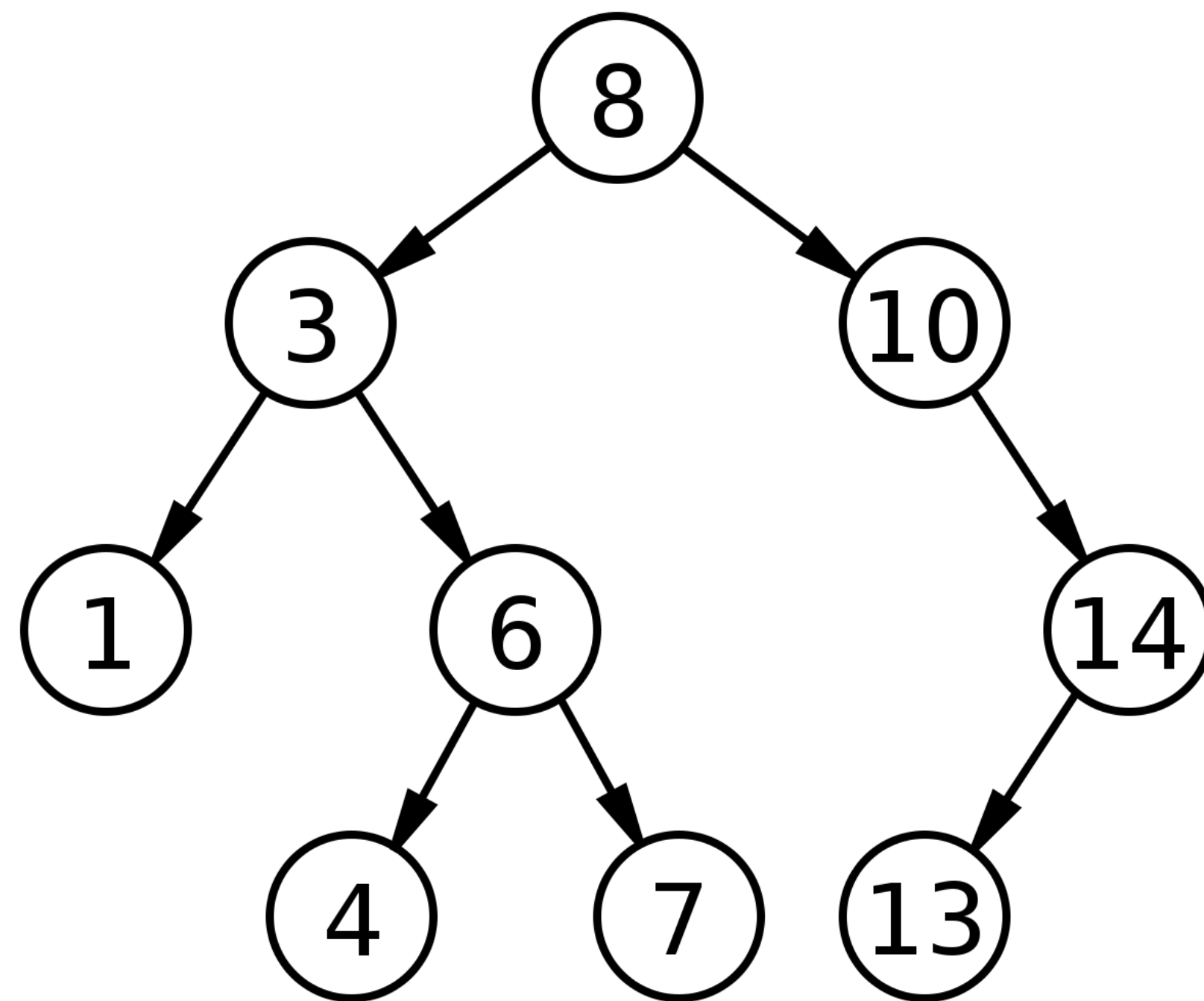
We're used to binary trees that repeatedly partition "value space"



Rank and select are about alphabet space; where are the 0s and 1s?
Where are the a's, c's, t's and g's?

Trees

We're used to binary trees that repeatedly partition "value space"



Rank and select are about alphabet space; where are the 0s and 1s?
Where are the a's, c's, t's and g's?

Idea: partition alphabet space

Wavelet trees

mississippi

Wavelet trees

mississippi

Partition alphabet into $\{i, m\}$, $\{p, s\}$

Wavelet trees

mississippi

Partition alphabet into $\{i, m\}$, $\{p, s\}$

mississippi

Wavelet trees

mississippi

Partition alphabet into $\{i, m\}$, $\{p, s\}$

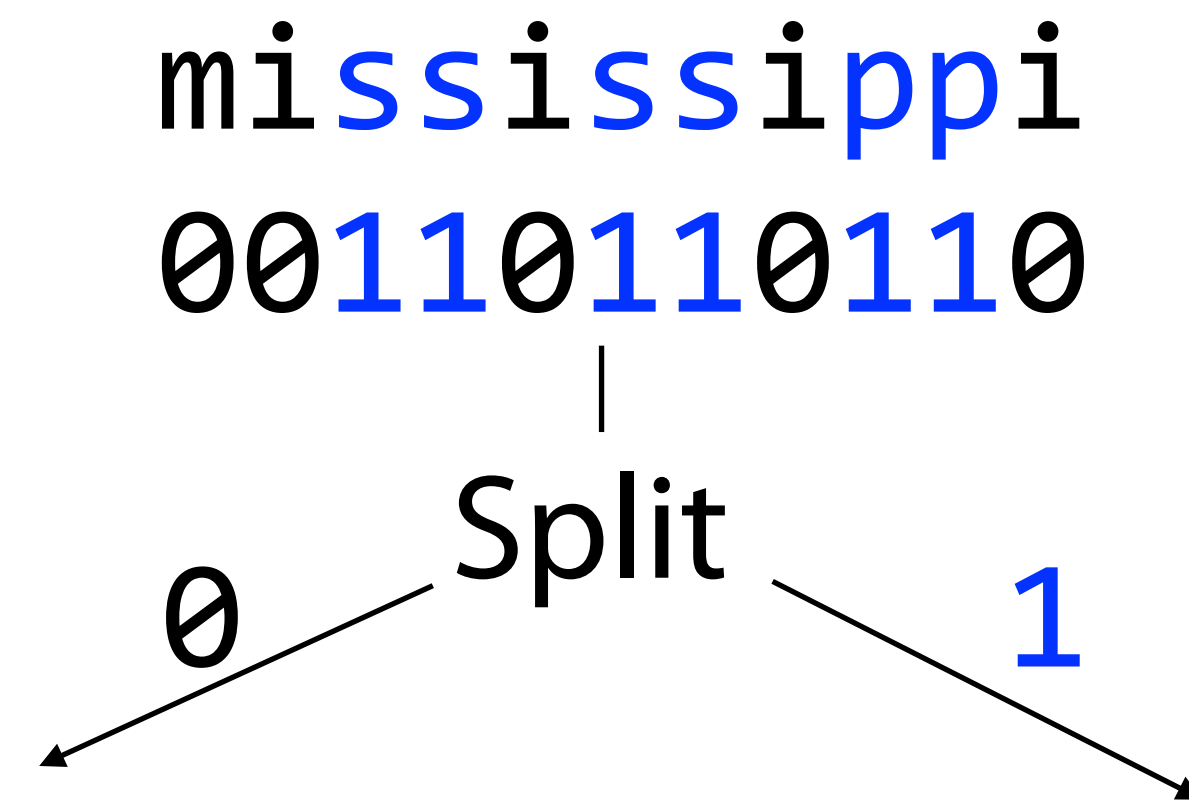
mississippi

00110110110

Wavelet trees

mississippi

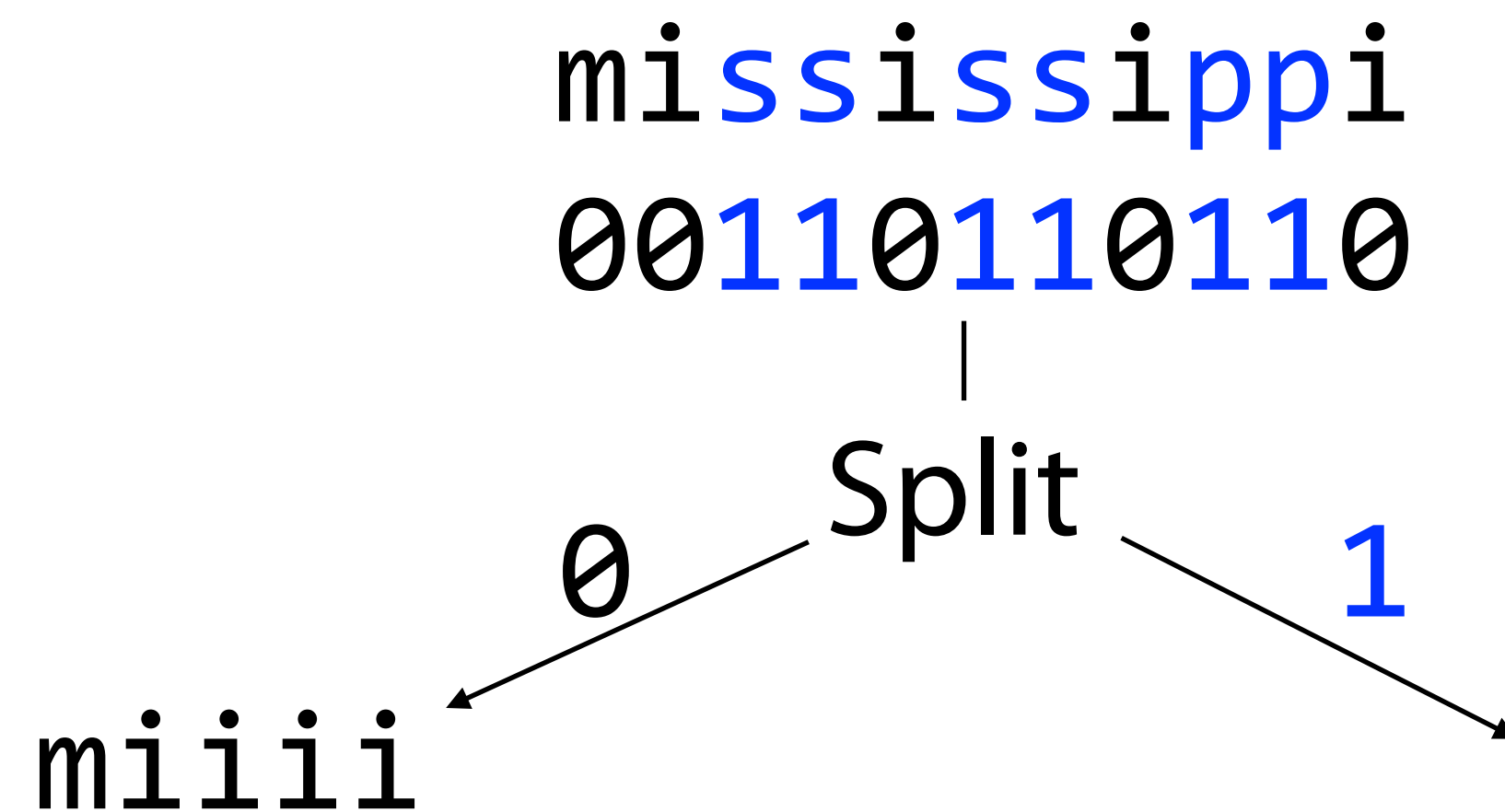
Partition alphabet into $\{i, m\}$, $\{p, s\}$



Wavelet trees

mississippi

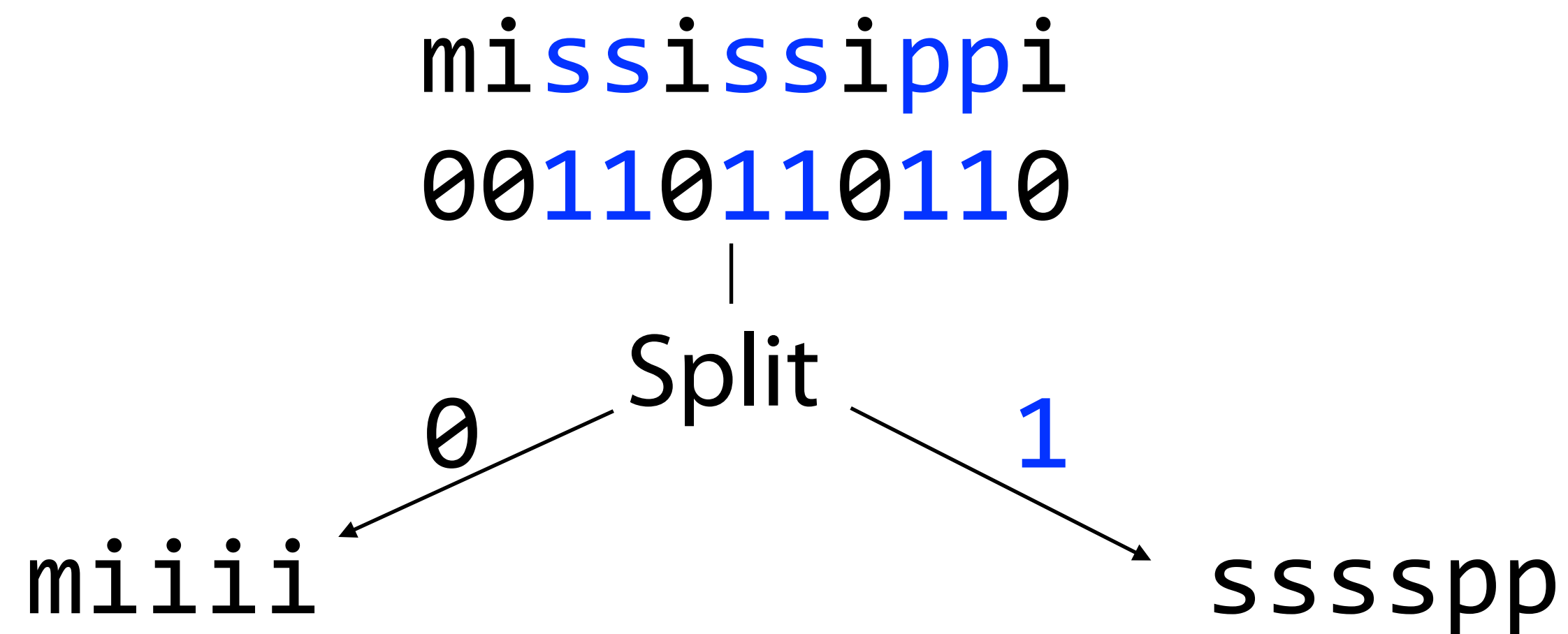
Partition alphabet into {i, m}, {p, s}



Wavelet trees

mississippi

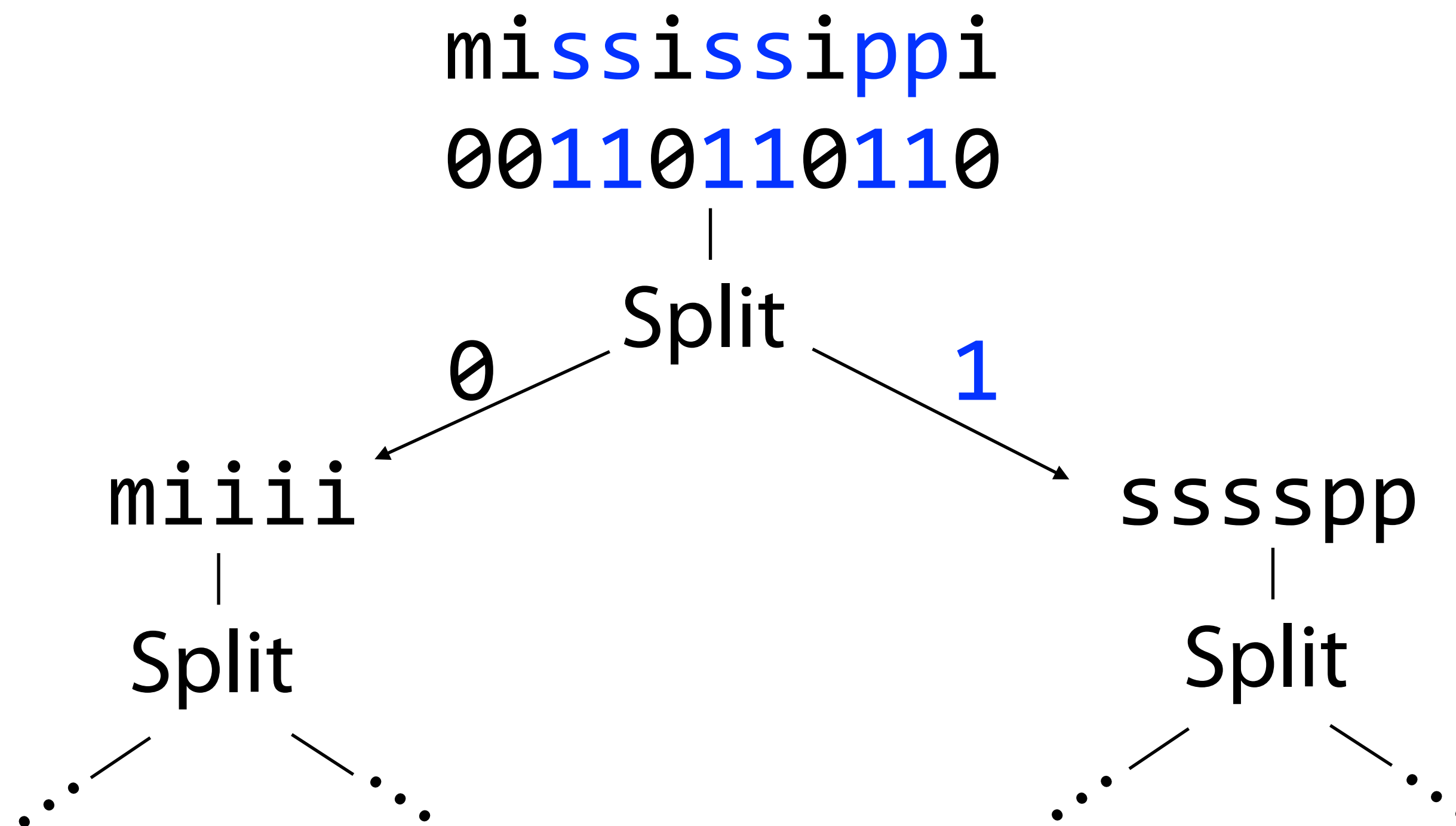
Partition alphabet into {i, m}, {p, s}



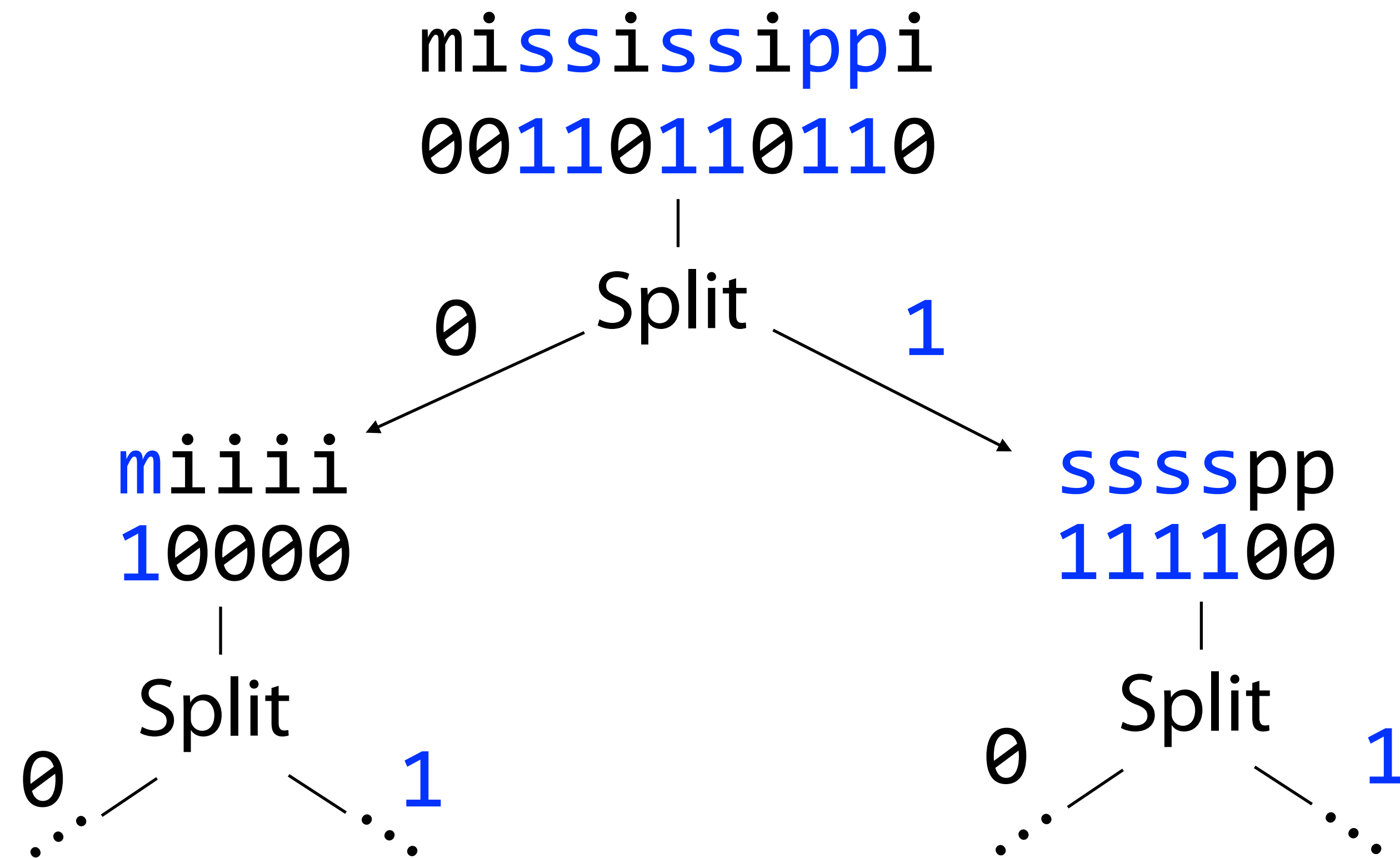
Wavelet trees

mississippi

Partition alphabet into $\{i, m\}$, $\{p, s\}$

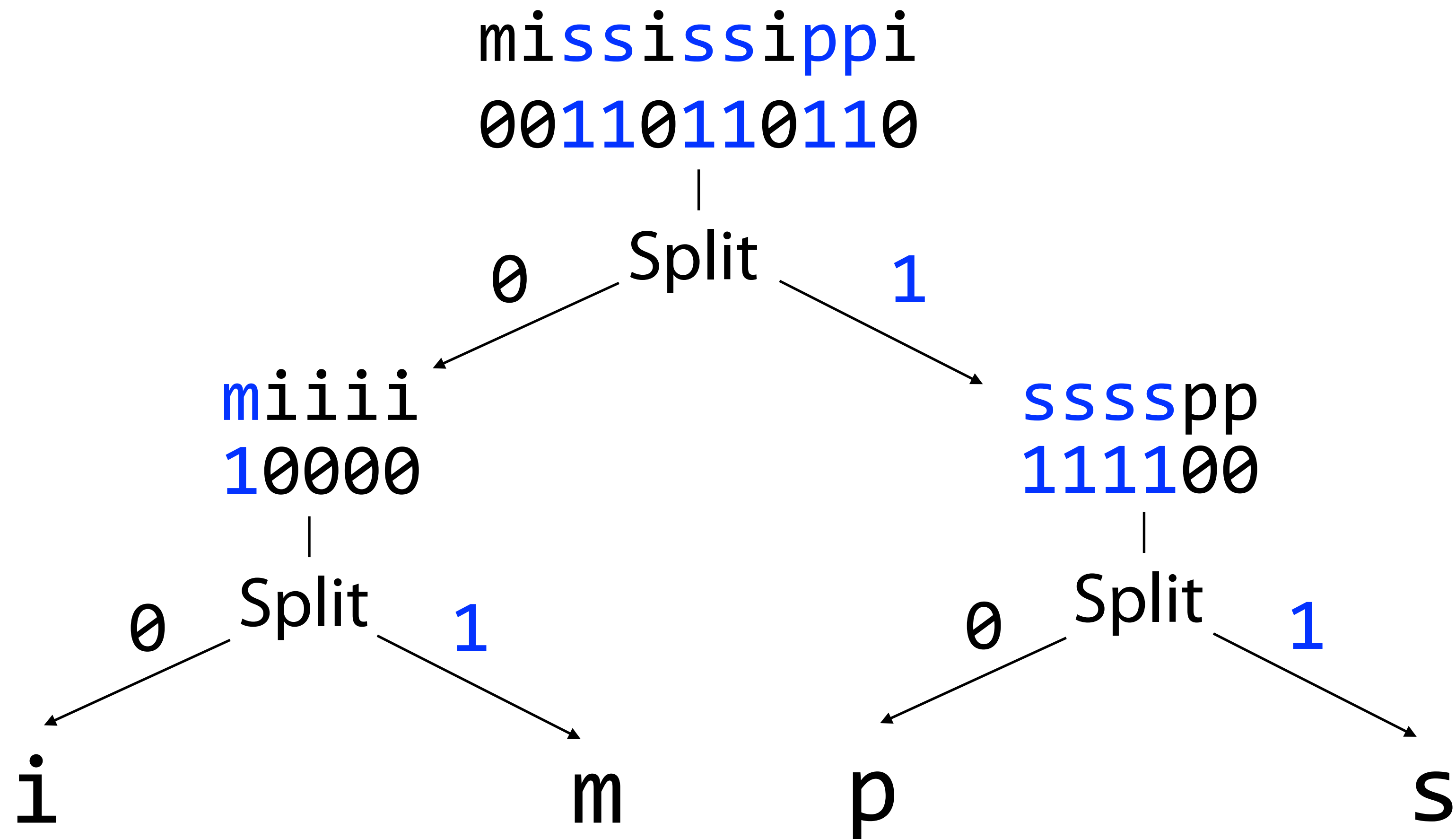


Wavelet trees

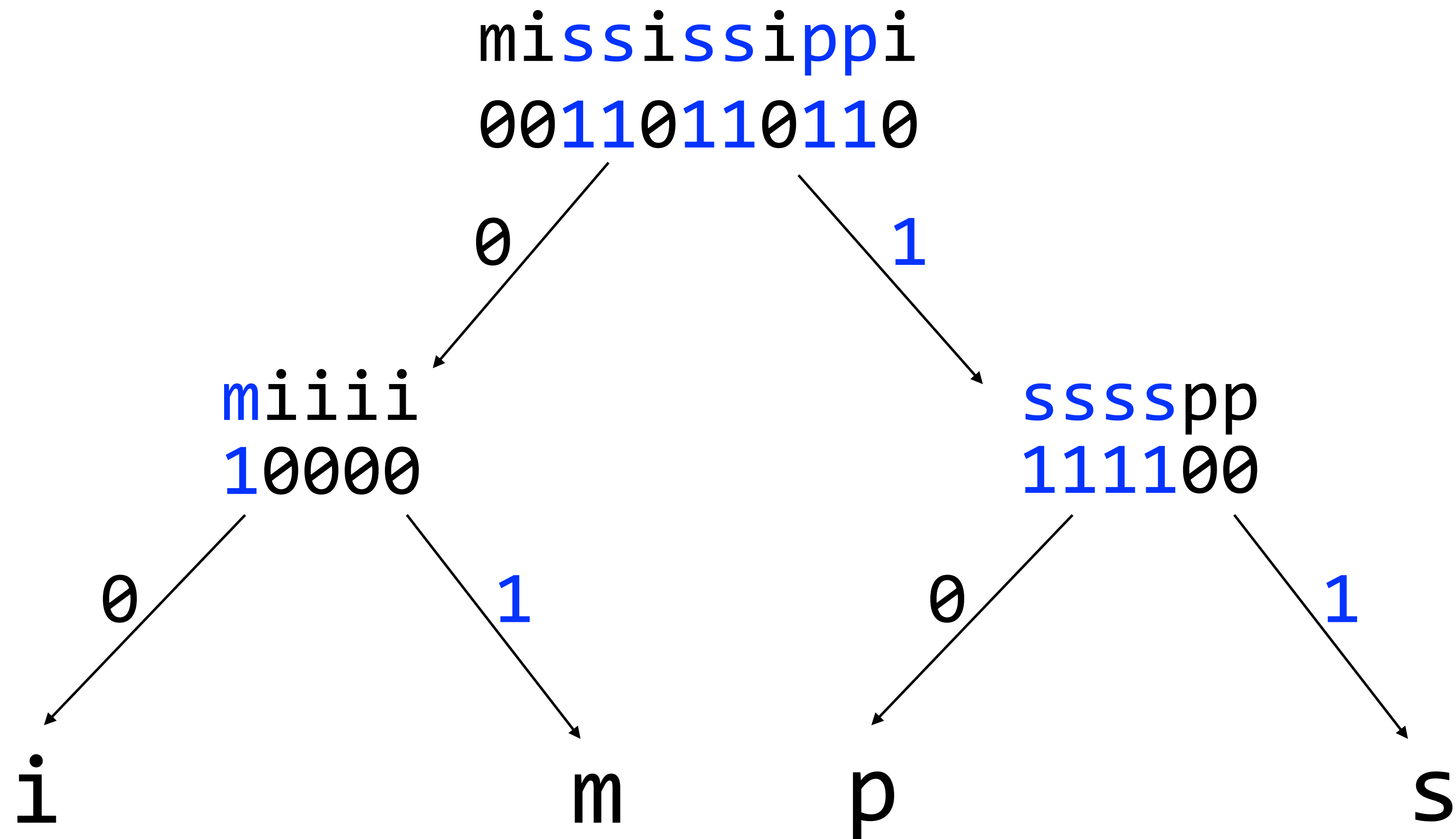


What goes in this next layer?

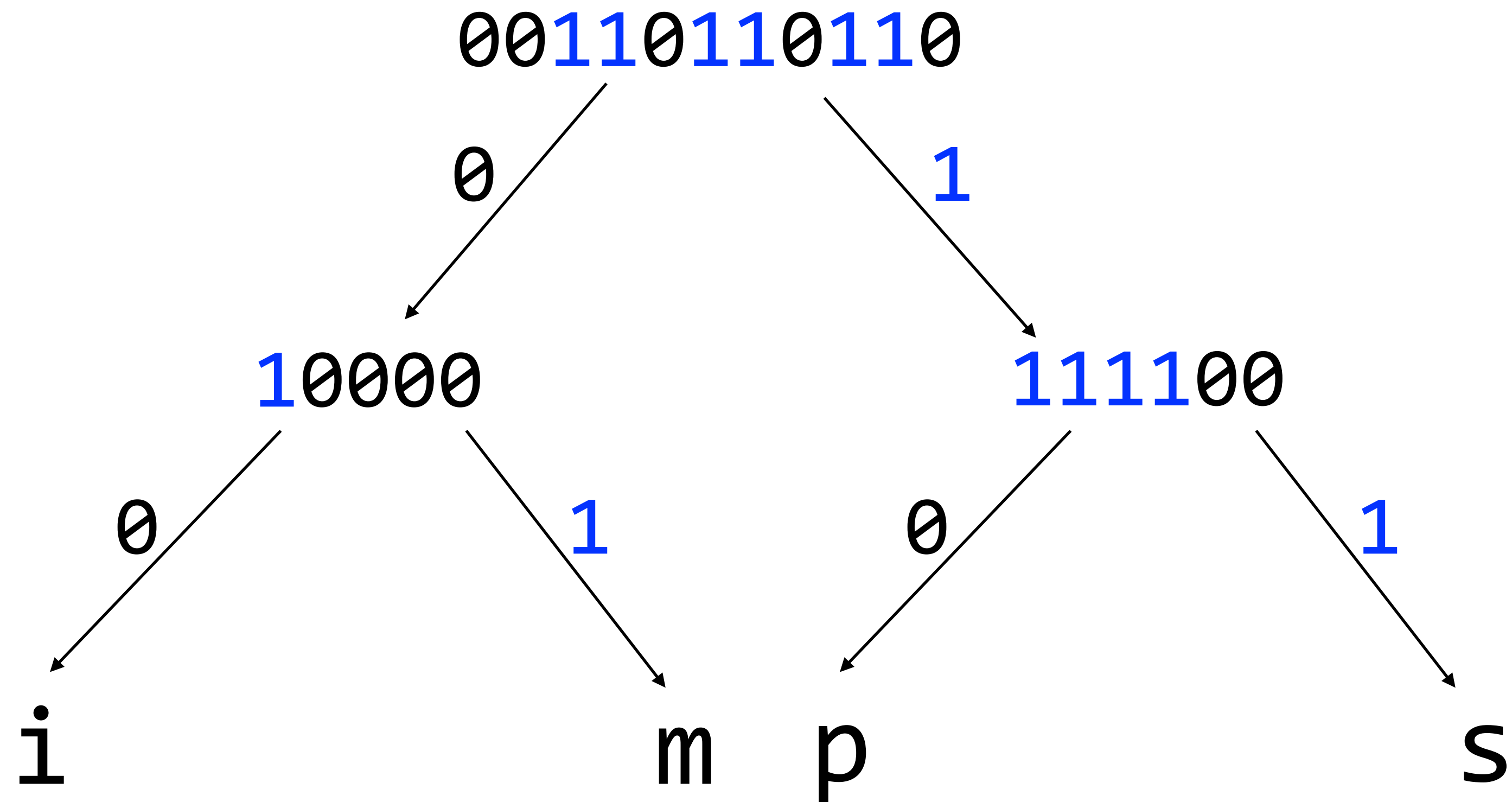
Wavelet trees



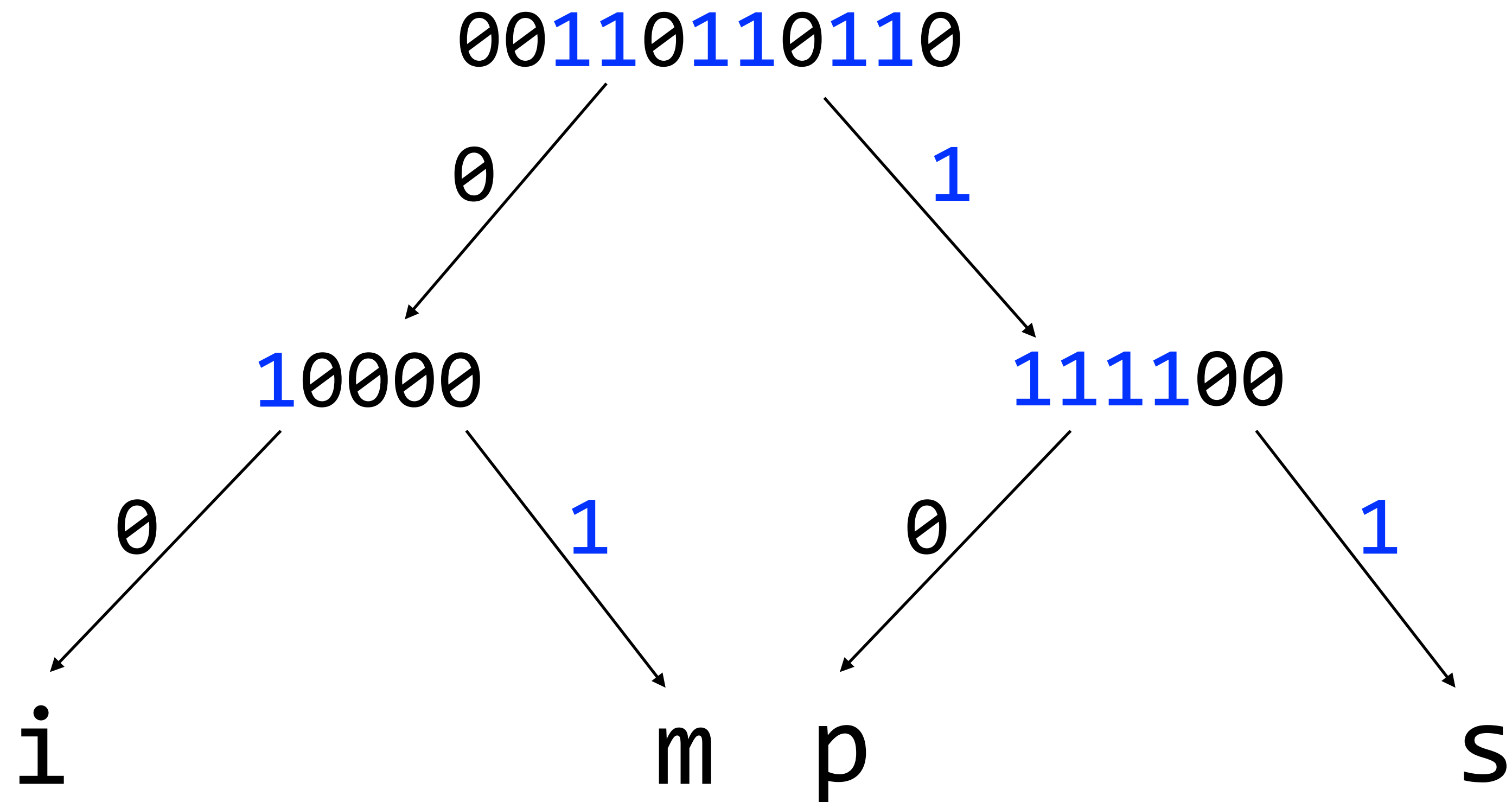
Wavelet trees



Wavelet trees



Wavelet trees



Can we do full-alphabet versions
of access, rank and select?

How big is this?

Wavelet trees

RSA queries extend naturally to strings:

Wavelet trees

RSA queries extend naturally to strings:

$$S . \text{access}(i) = S[i]$$

Wavelet trees

RSA queries extend naturally to strings:

$$S . \text{access}(i) = S[i]$$

$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

Wavelet trees

RSA queries extend naturally to strings:

$$S . \text{access}(i) = S[i]$$

$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

$$S . \text{select}_c(i) = \max \{ j \mid S . \text{rank}_c(j) = i \}$$

Wavelet trees

RSA queries extend naturally to strings:

$$S . \text{access}(i) = S[i]$$

$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

$$S . \text{select}_c(i) = \max \{ j \mid S . \text{rank}_c(j) = i \}$$

Where S is a string with alphabet Σ and $c \in \Sigma$

Wavelet trees

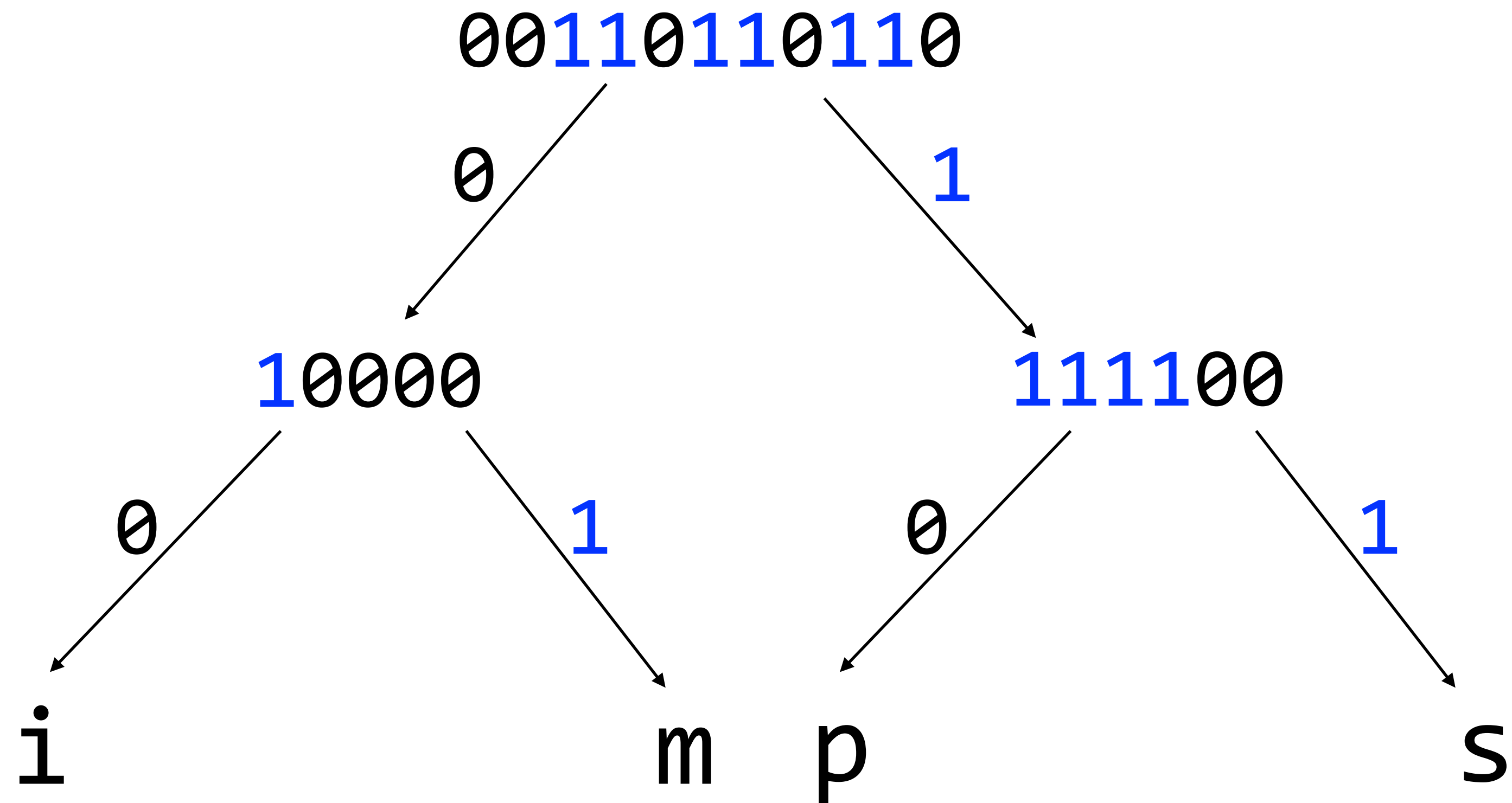
$$S . \text{access}(i) = S[i]$$

$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

$$S . \text{select}_c(i) = \max \{ j \mid S . \text{rank}_c(j) = i \}$$

Wavelet trees

$S . \text{access}(4)$



Wavelet trees

$S . \text{access}(4)$



00**11**0**11**0**11**0 . $\text{access}(4) = 0$

0

1

10000

111100

0

1

0

1

i

m

p

s

Wavelet trees

$S . \text{access}(4)$



00**11**0**11**0**11**0 . $\text{access}(4) = 0$

{i, m}

0

1

{p, s}

10000

111100

0

1

0

1

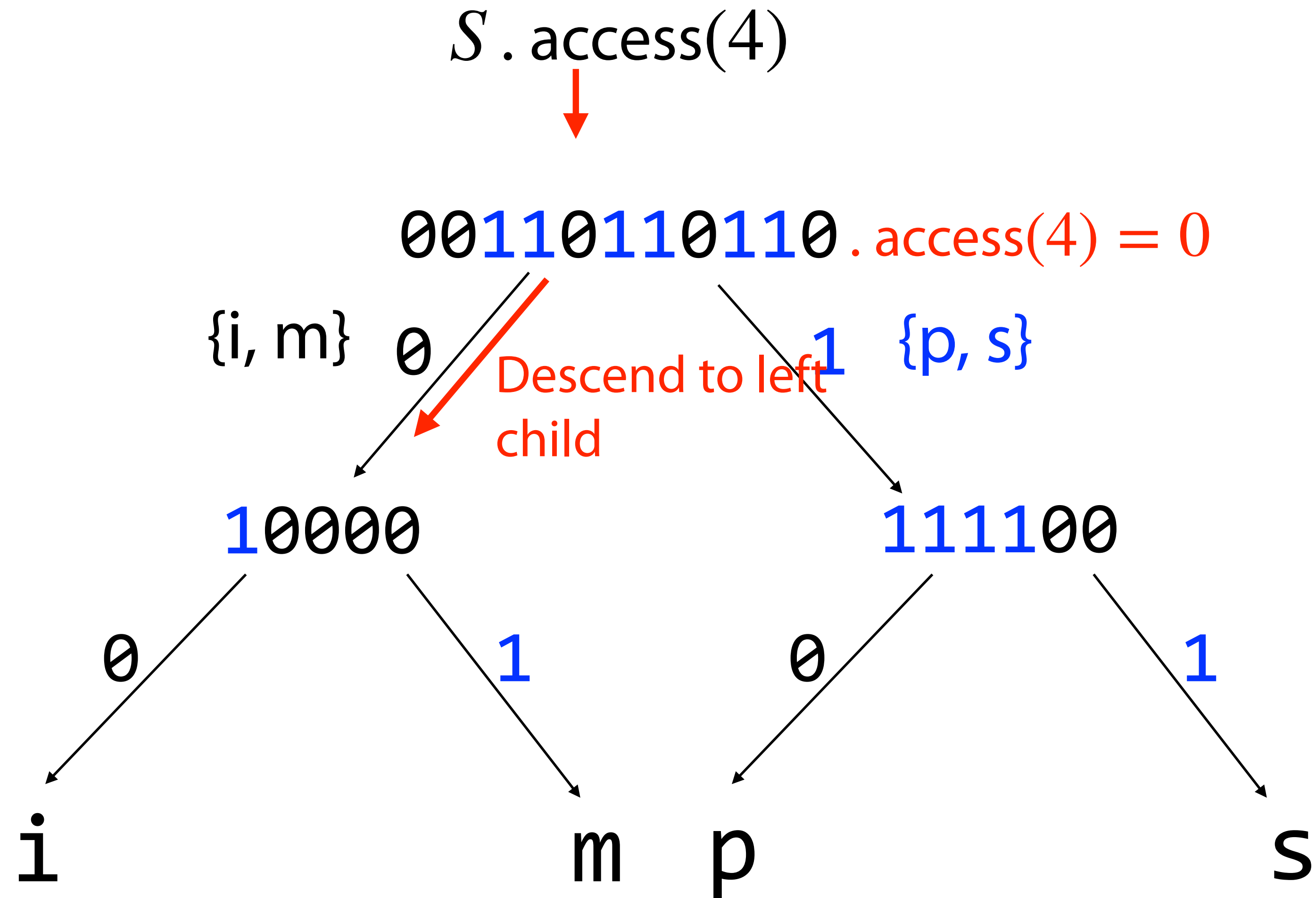
i

m

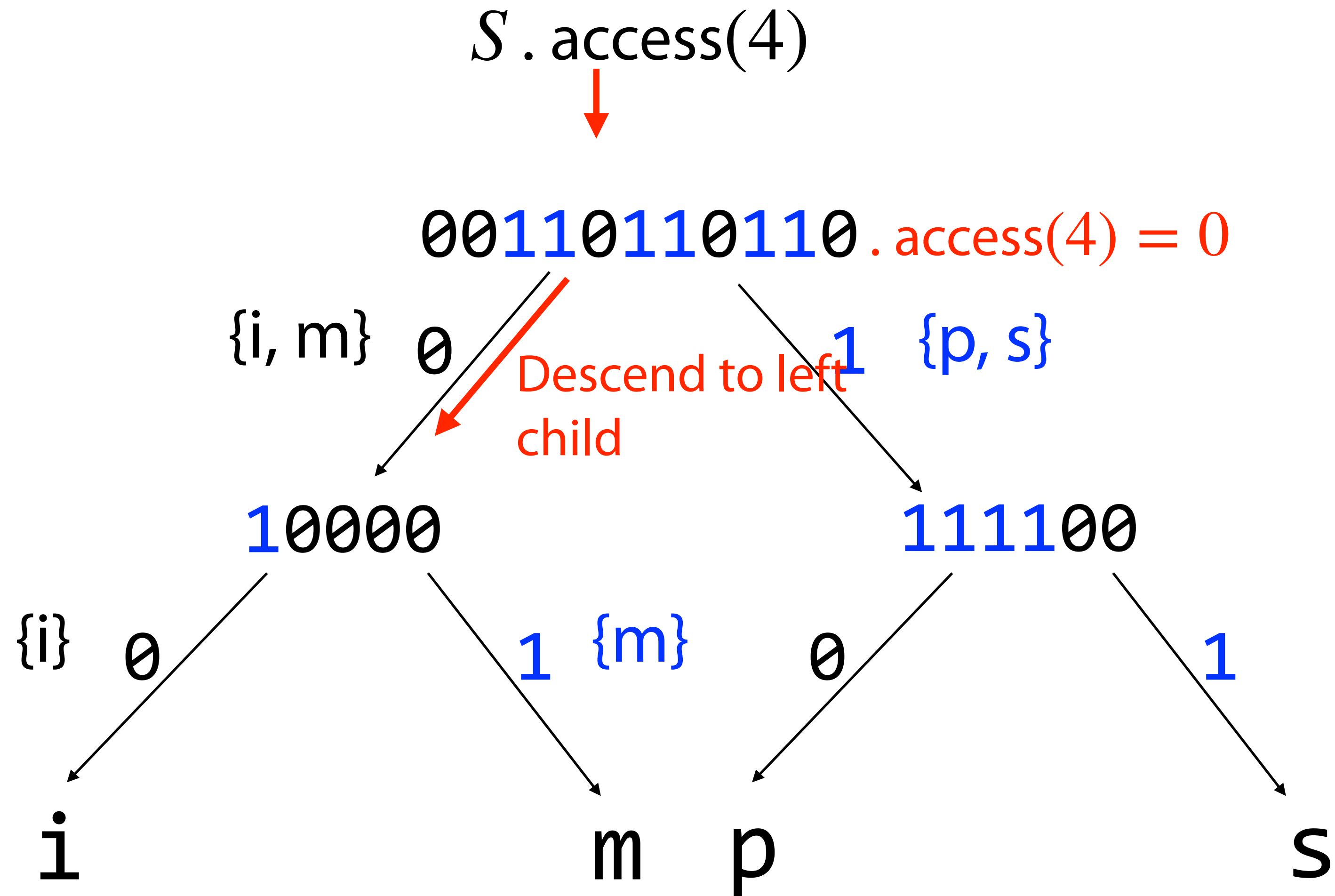
p

s

Wavelet trees



Wavelet trees



Wavelet trees

$S . \text{access}(4)$



00**11**0**11**0**11**0 . $\text{access}(4) = 0$

0



Descend to left
child

10000

{i}

0

i

1

{m}

m

Wavelet trees

$S . \text{access}(4)$



$00110110110 . \text{access}(4) = 0$

$\emptyset$

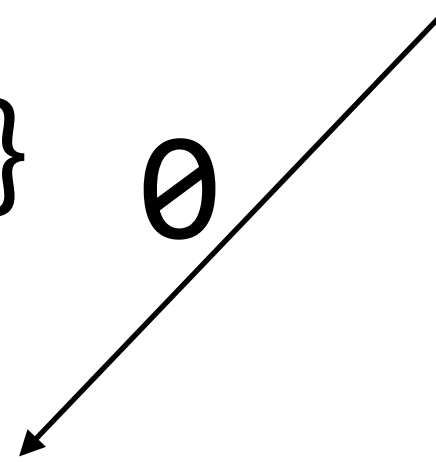


Descend to left
child

$10000 . \text{access}(\text{parent.rank}_0(4)) =$

$\{i\}$

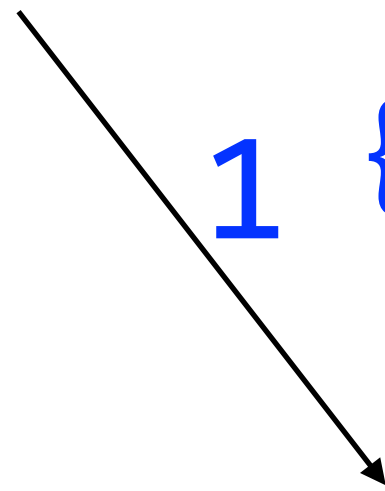
$\emptyset$



i

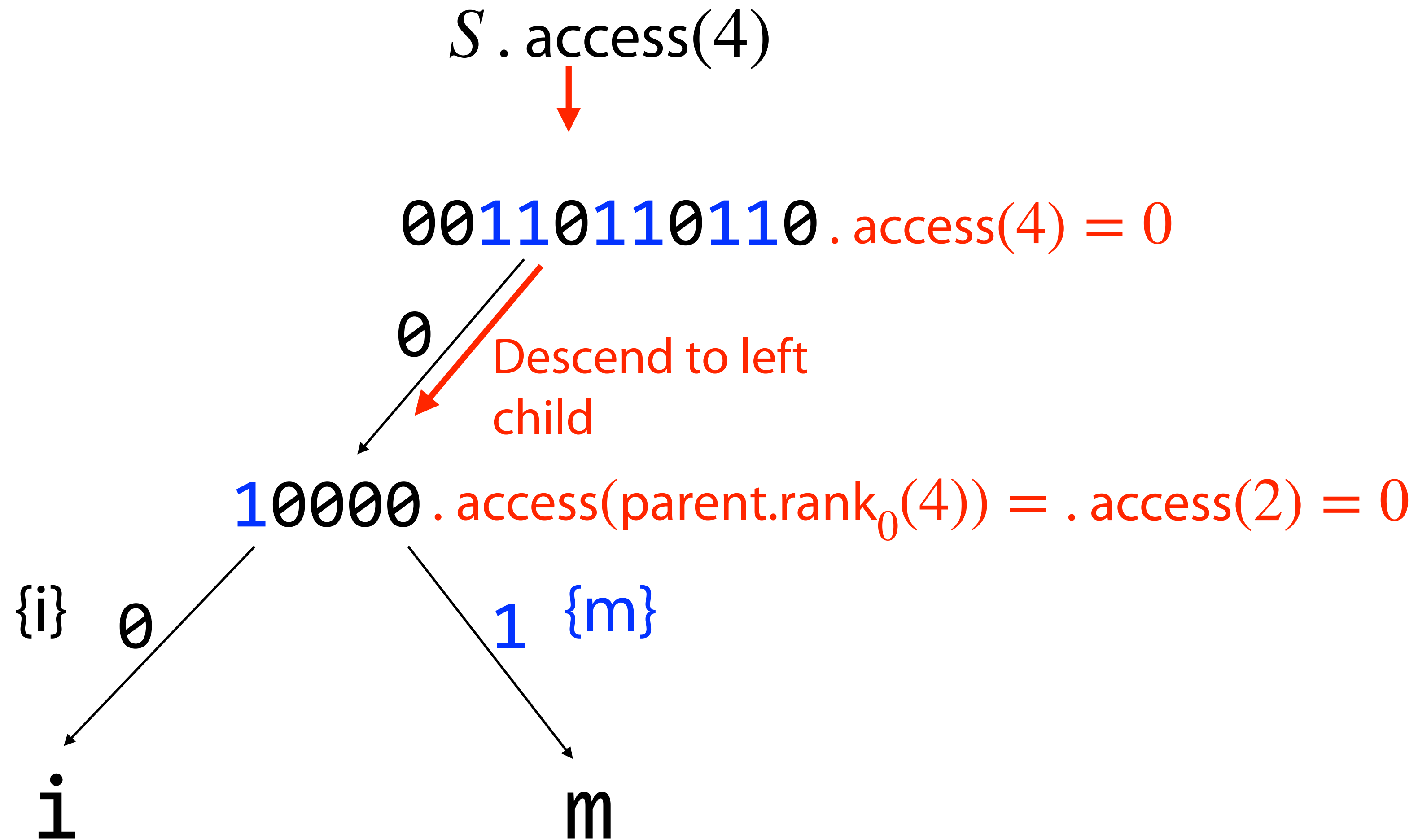
1

$\{m\}$

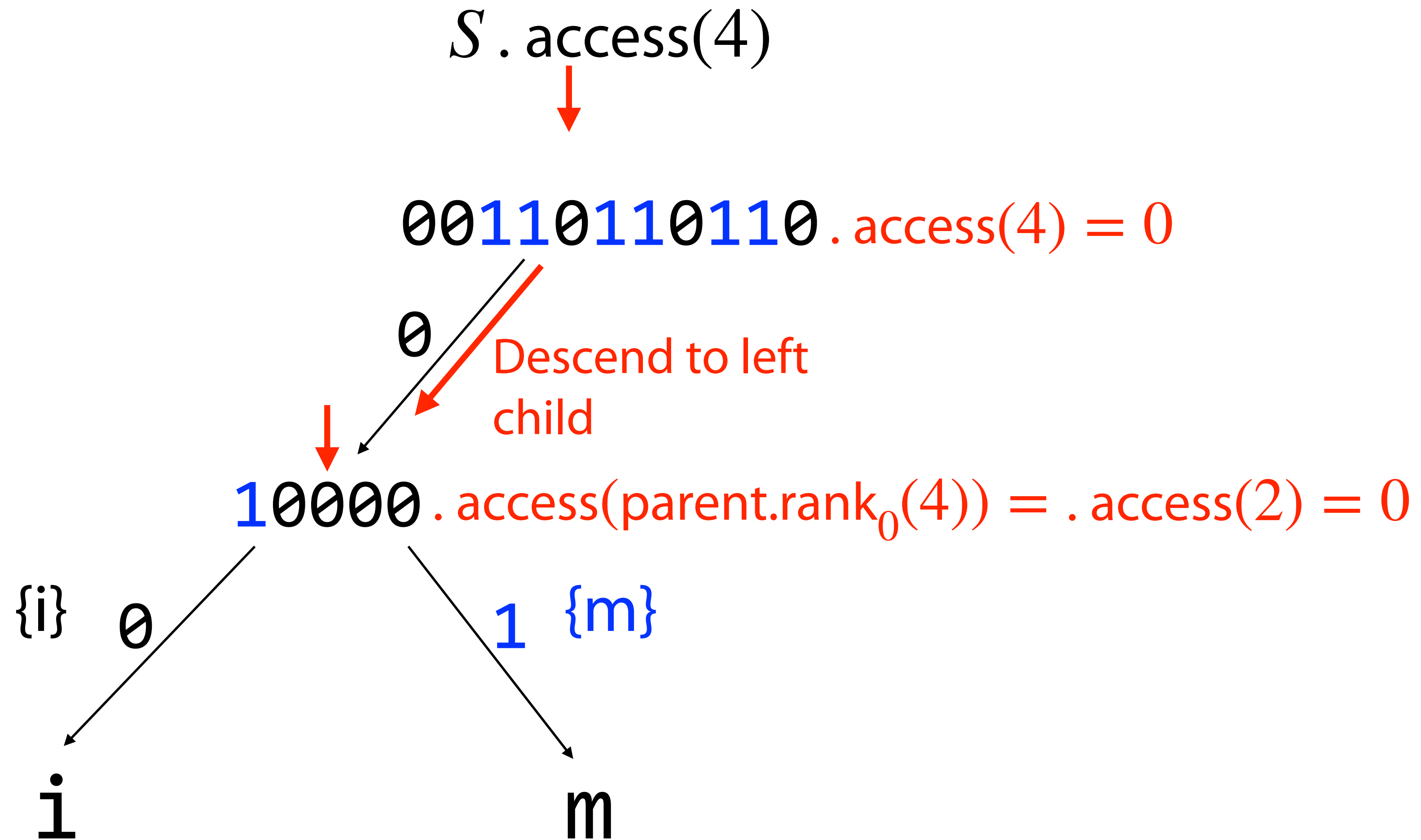


m

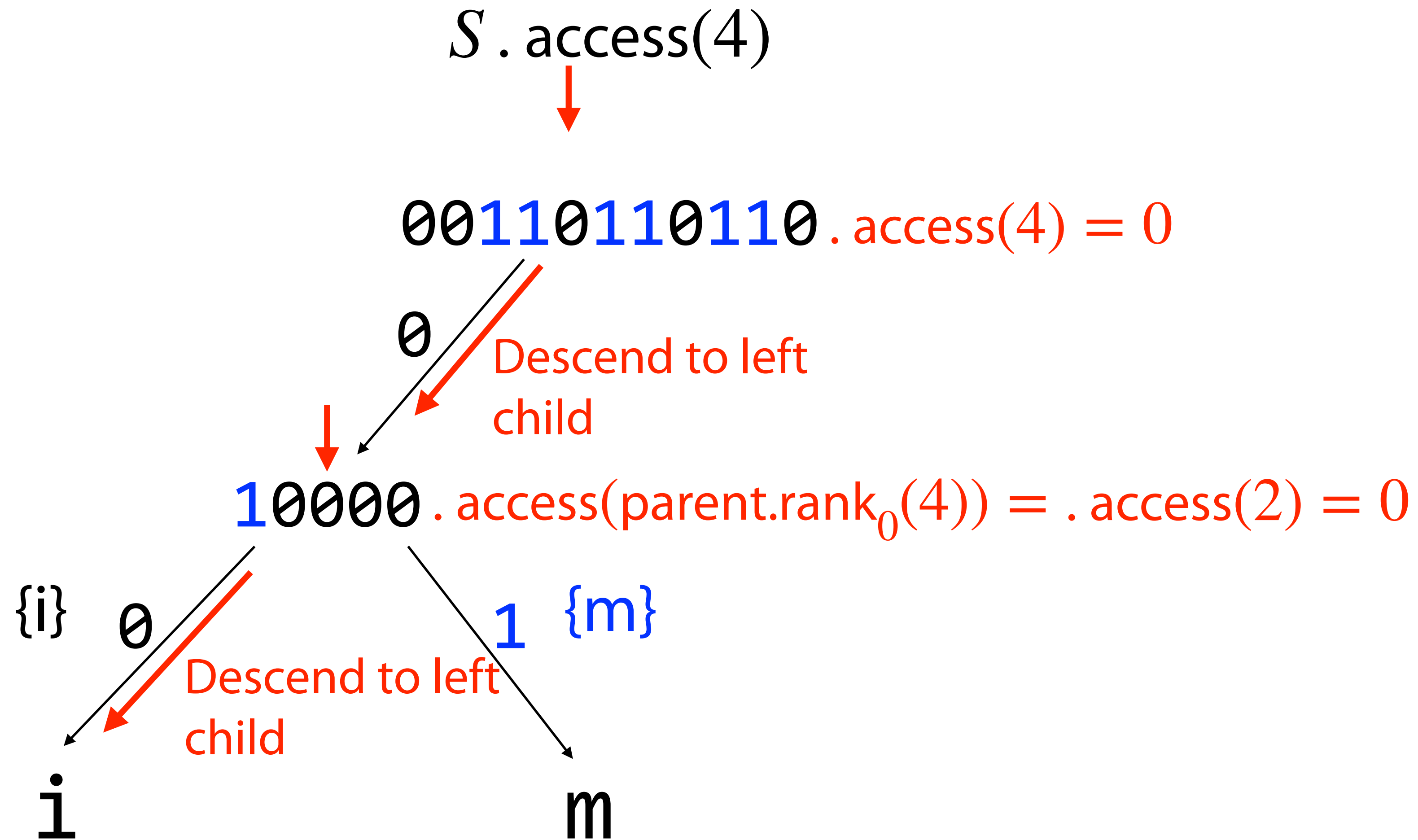
Wavelet trees



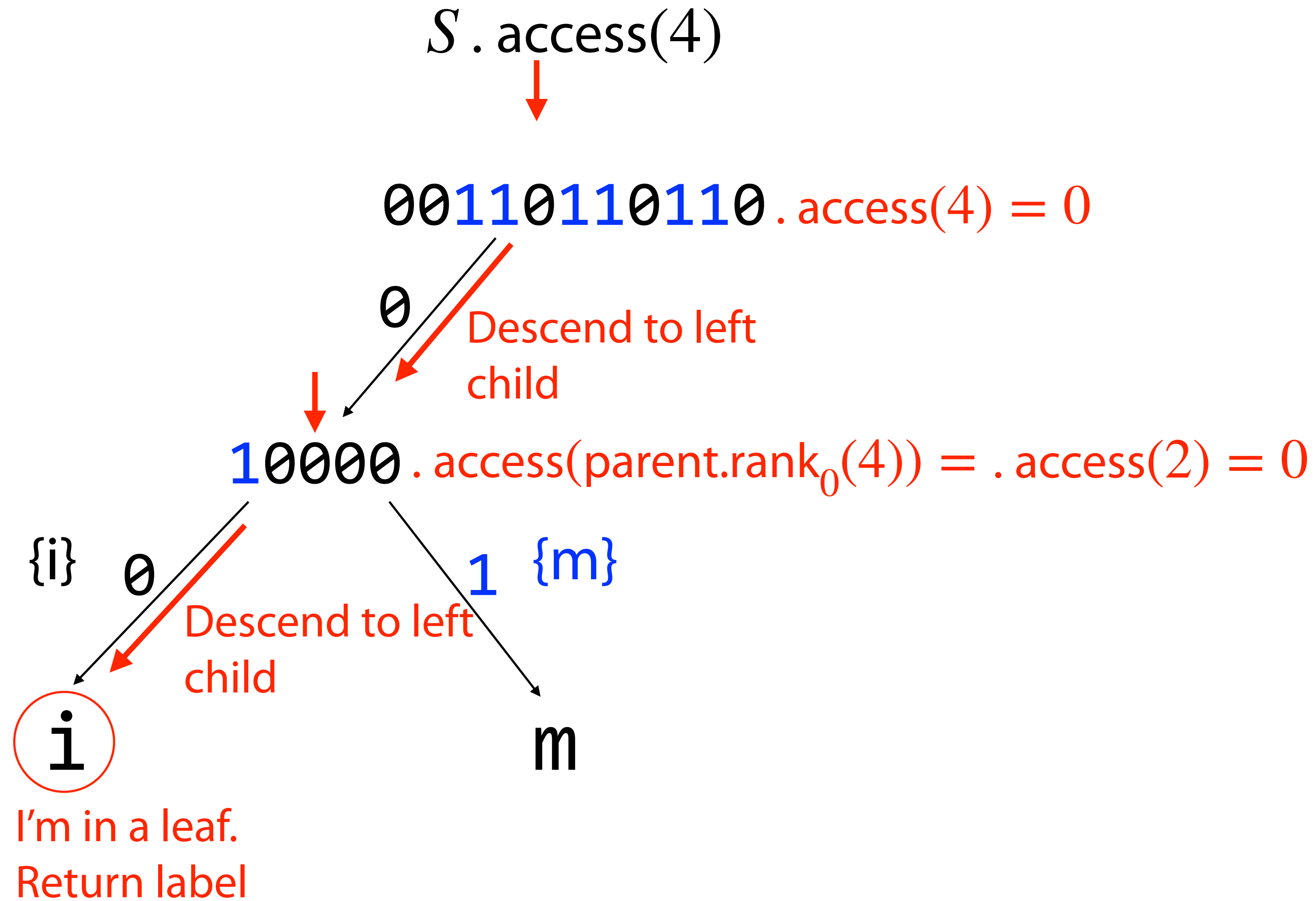
Wavelet trees



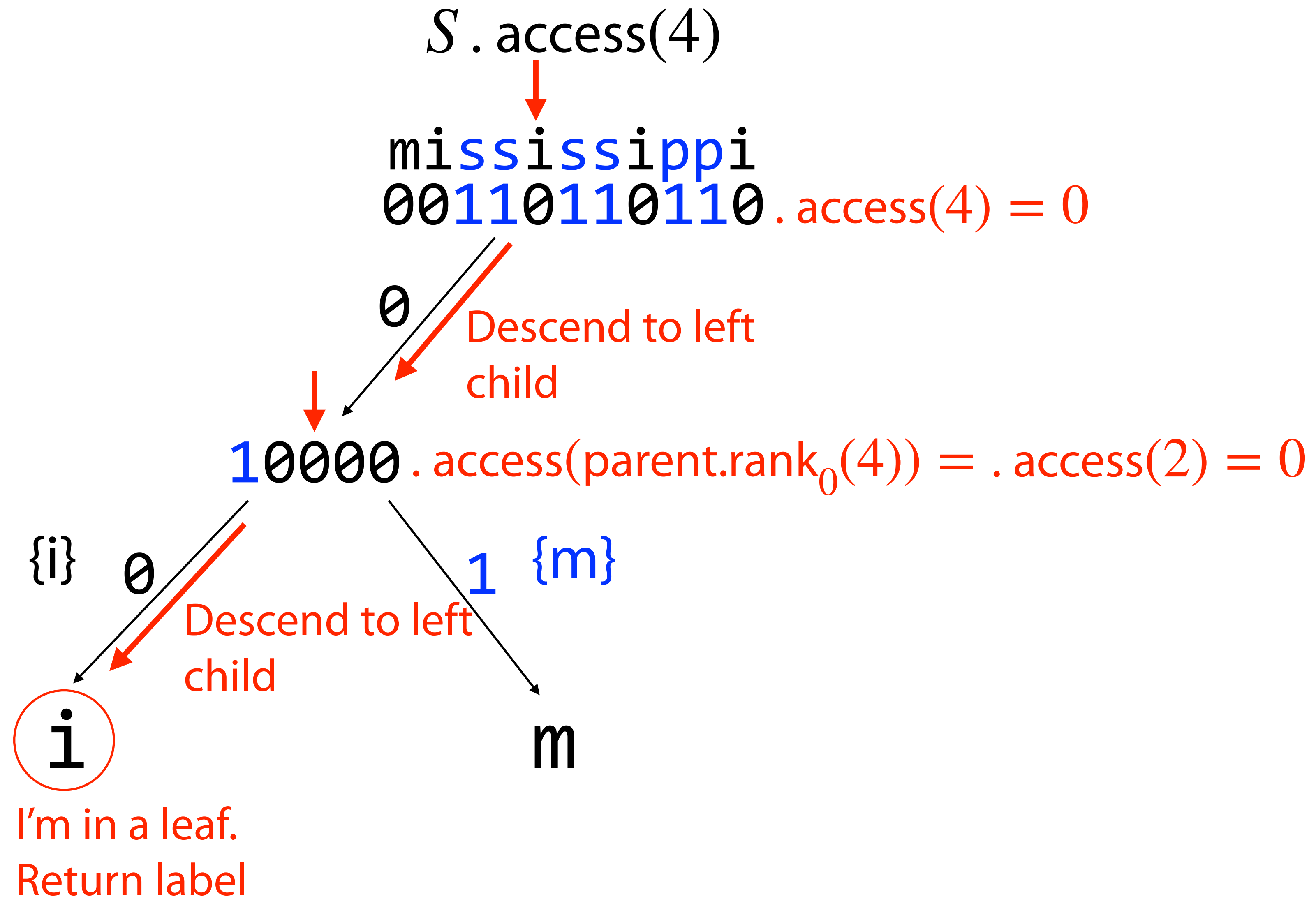
Wavelet trees



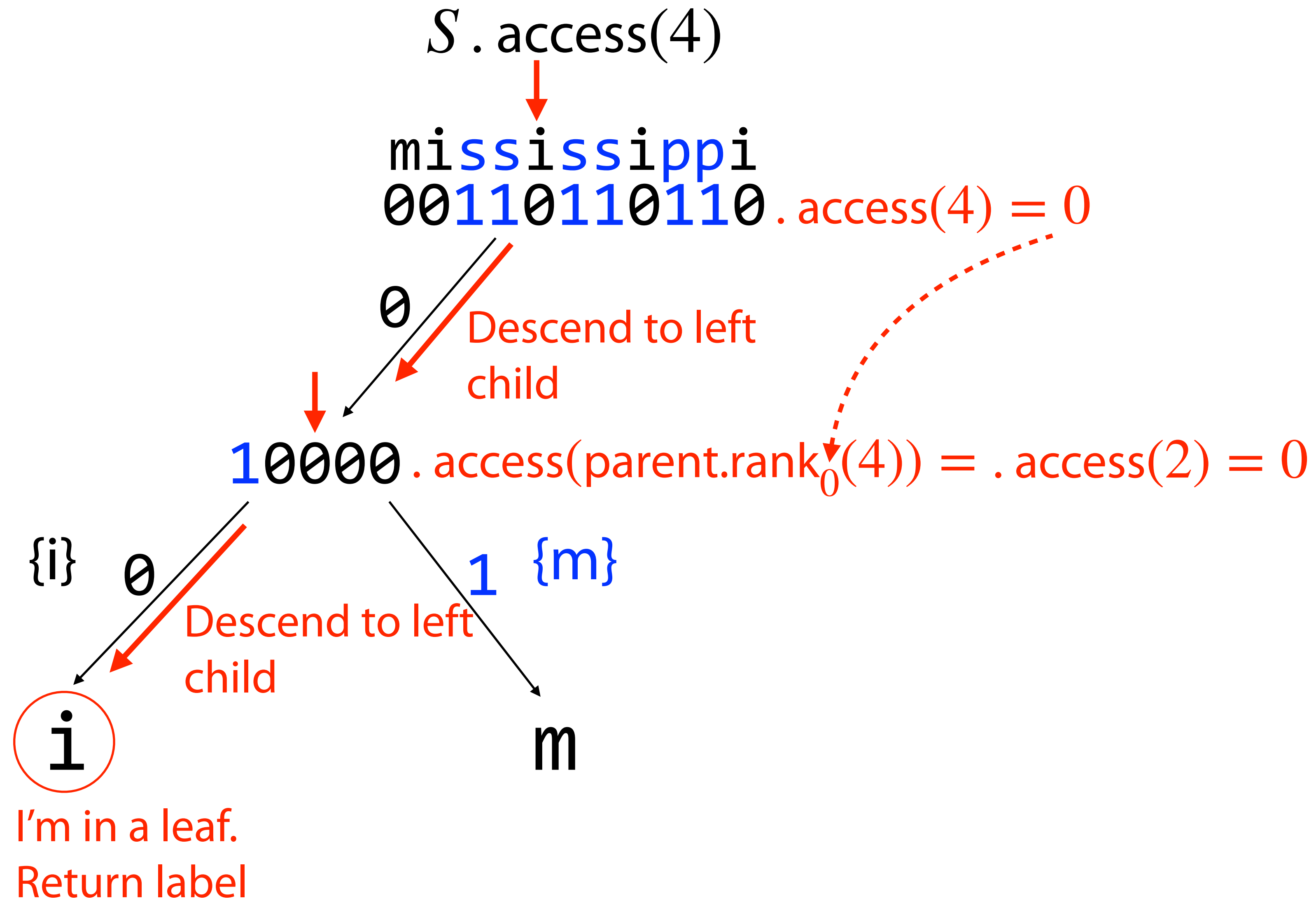
Wavelet trees



Wavelet trees



Wavelet trees



Wavelet trees

Wavelet tree access(i):

```
 $N \leftarrow \text{root}$   
while  $N$  is not leaf  
     $B \leftarrow N.\text{bitvector}$   
     $b \leftarrow B[i]$   
     $N \leftarrow N.\text{child}(b)$   
     $i \leftarrow B.\text{rank}_b(i)$   
return  $N.\text{label}$ 
```

Wavelet trees

Wavelet tree access(i):

Given offset i :

$N \leftarrow root$

while N is not leaf

$B \leftarrow N.bitvector$

$b \leftarrow B[i]$

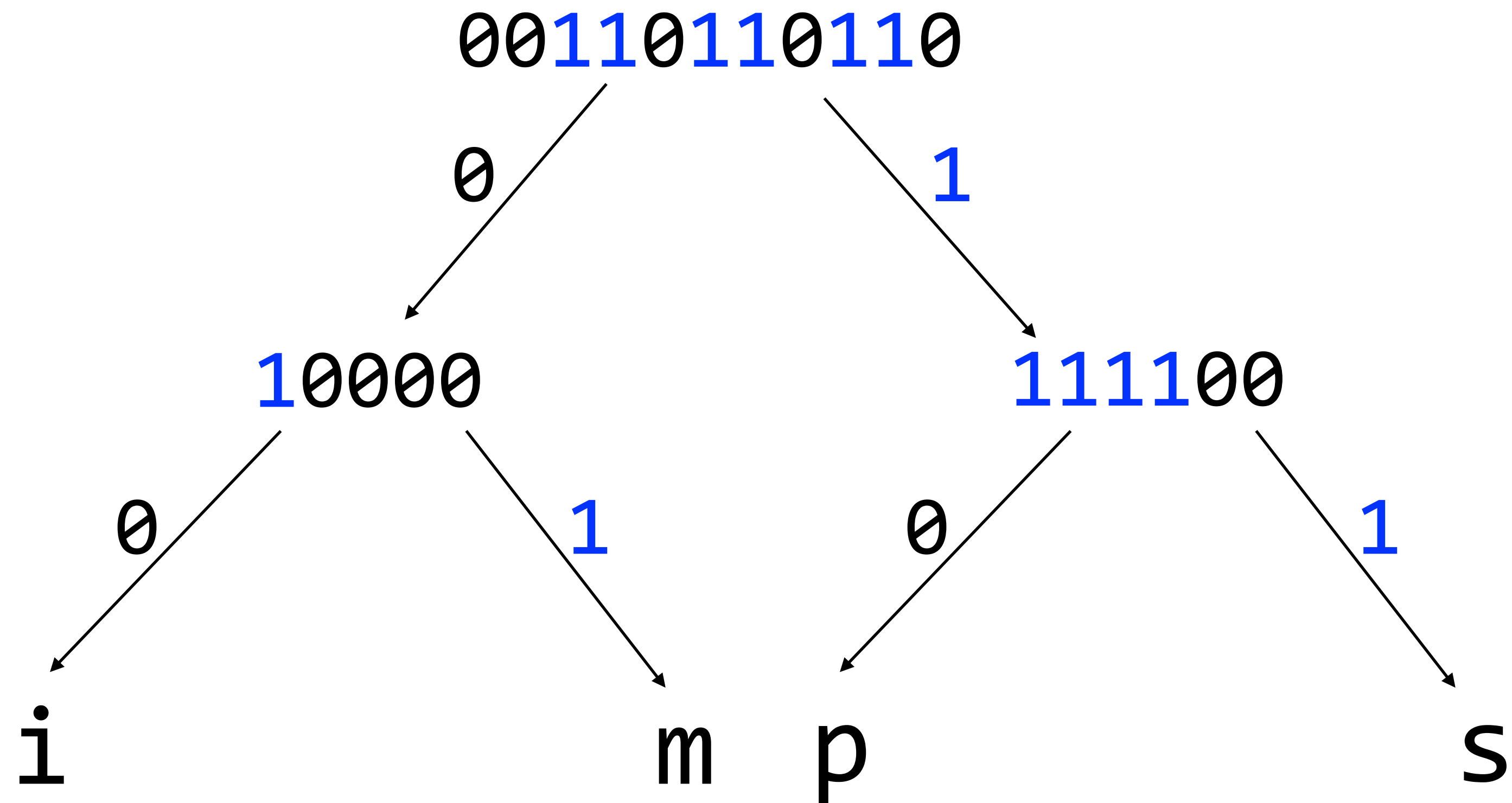
$N \leftarrow N.child(b)$

$i \leftarrow B.rank_b(i)$

return $N.label$

Wavelet trees

$S . \text{access}(6)$



Wavelet trees

$S . \text{access}(6)$



00**11**0**11**0**11**0 . $\text{access}(6) = 1$

0

1

10000

111100

0

1

0

1

i

m

p

s

Wavelet trees

$S . \text{access}(6)$



00**11**0**11**0**11**0 . $\text{access}(6) = 1$

0

Descend to
right child

1

10000

111100

0

1

0

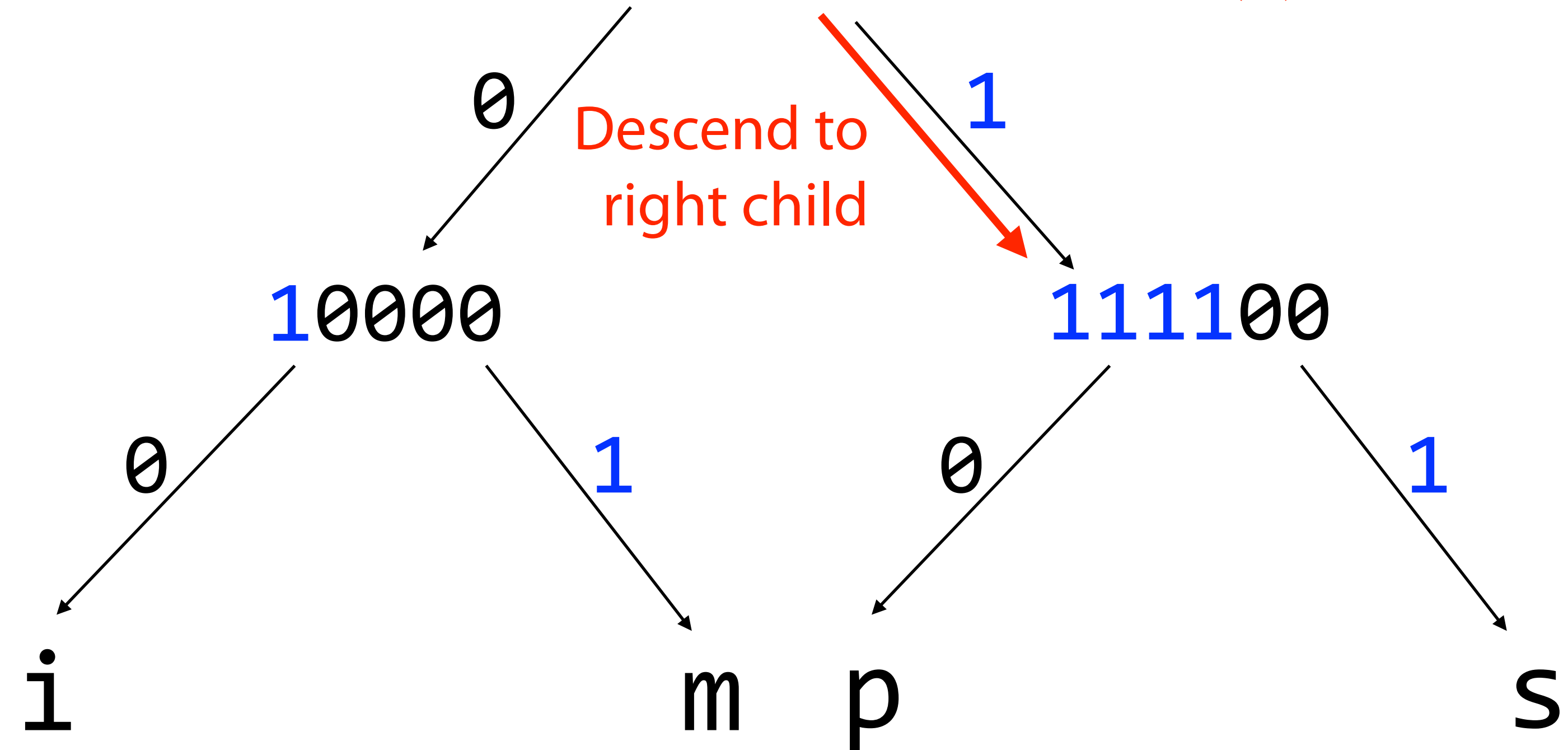
1

i

m

p

s



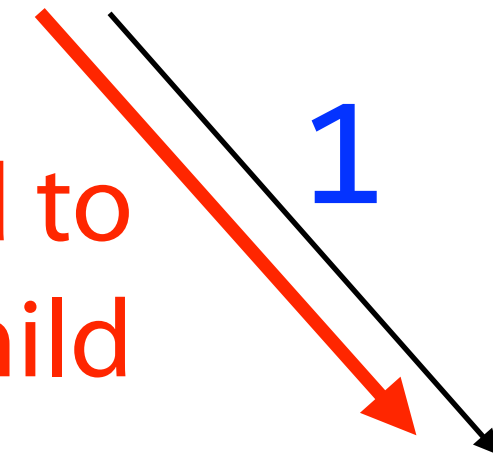
Wavelet trees

$S . \text{access}(6)$



00110110110 . access(6) = 1

Descend to
right child



1

111100

0

p

1

s

Wavelet trees

$S . \text{access}(6)$



$00110110 . \text{access}(6) = 1$

Descend to
right child

1

$111100 . \text{access}(\text{parent} . \text{rank}_1(6)) =$

0

p

1

s

Wavelet trees

$S . \text{access}(6)$



$00110110 . \text{access}(6) = 1$

Descend to
right child

1

$111100 . \text{access}(\text{parent} . \text{rank}_1(6)) = 3$

0

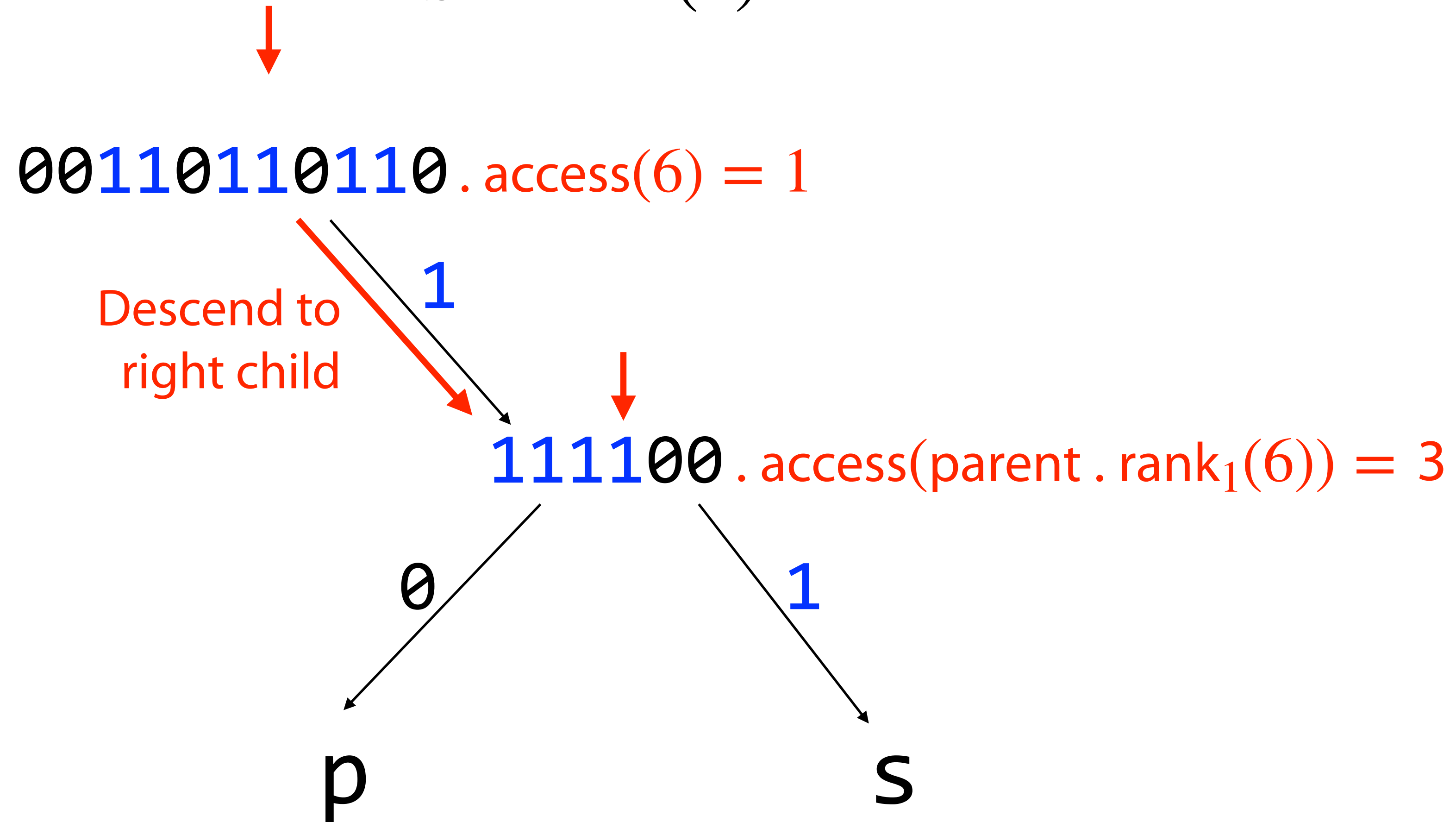
1

p

s

Wavelet trees

$S . \text{access}(6)$



Wavelet trees

$S . \text{access}(6)$

$00110110 . \text{access}(6) = 1$

Descend to
right child

1

$111100 . \text{access}(\text{parent} . \text{rank}_1(6)) = 3$

Descend to
right child

1

p

s

Wavelet trees

$S . \text{access}(6)$

$00110110 . \text{access}(6) = 1$

Descend to
right child

1

$111100 . \text{access}(\text{parent} . \text{rank}_1(6)) = 3$

Descend to
right child

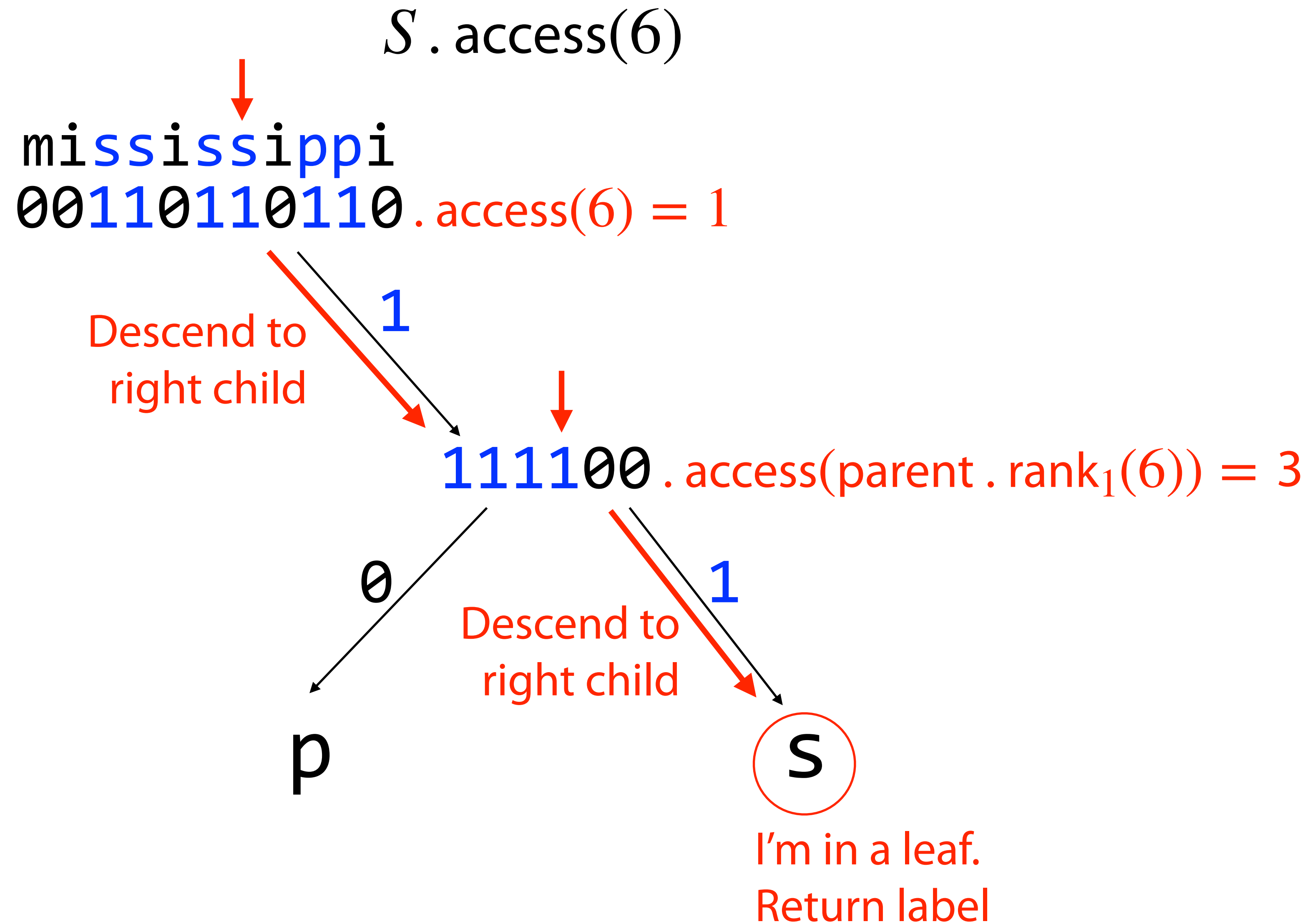
1

p

S

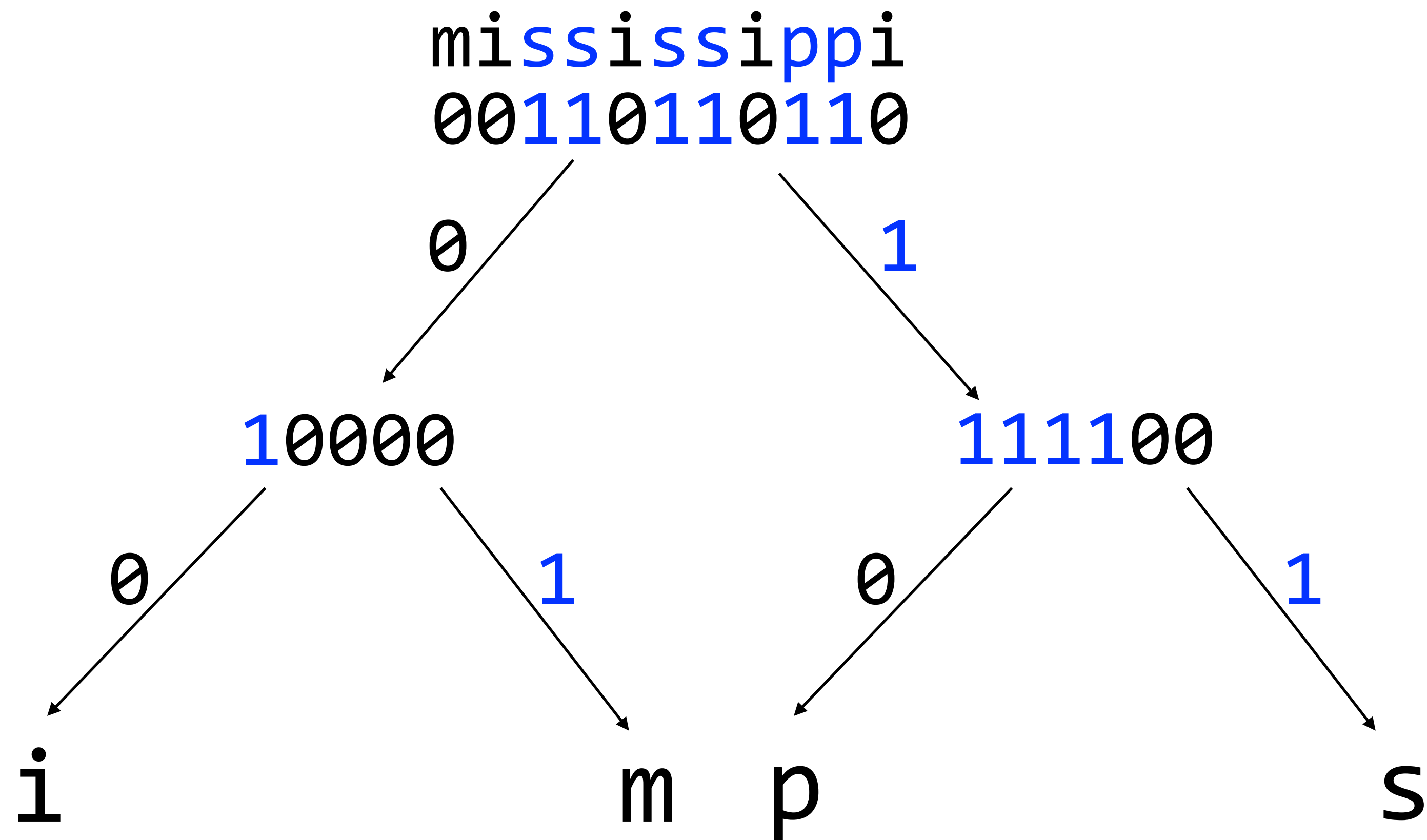
I'm in a leaf.
Return label

Wavelet trees



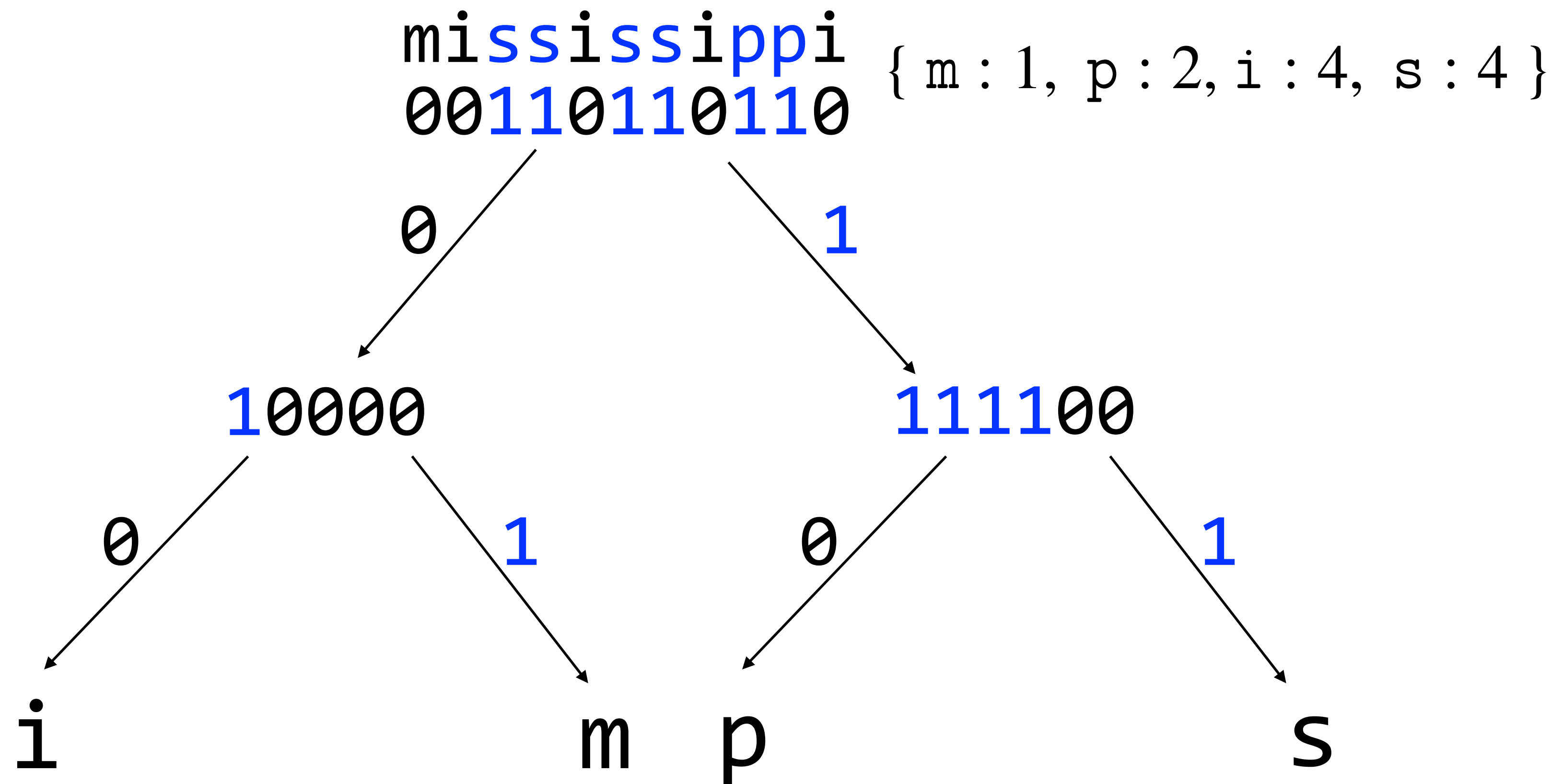
Wavelet trees

Could have picked a different shape for the tree



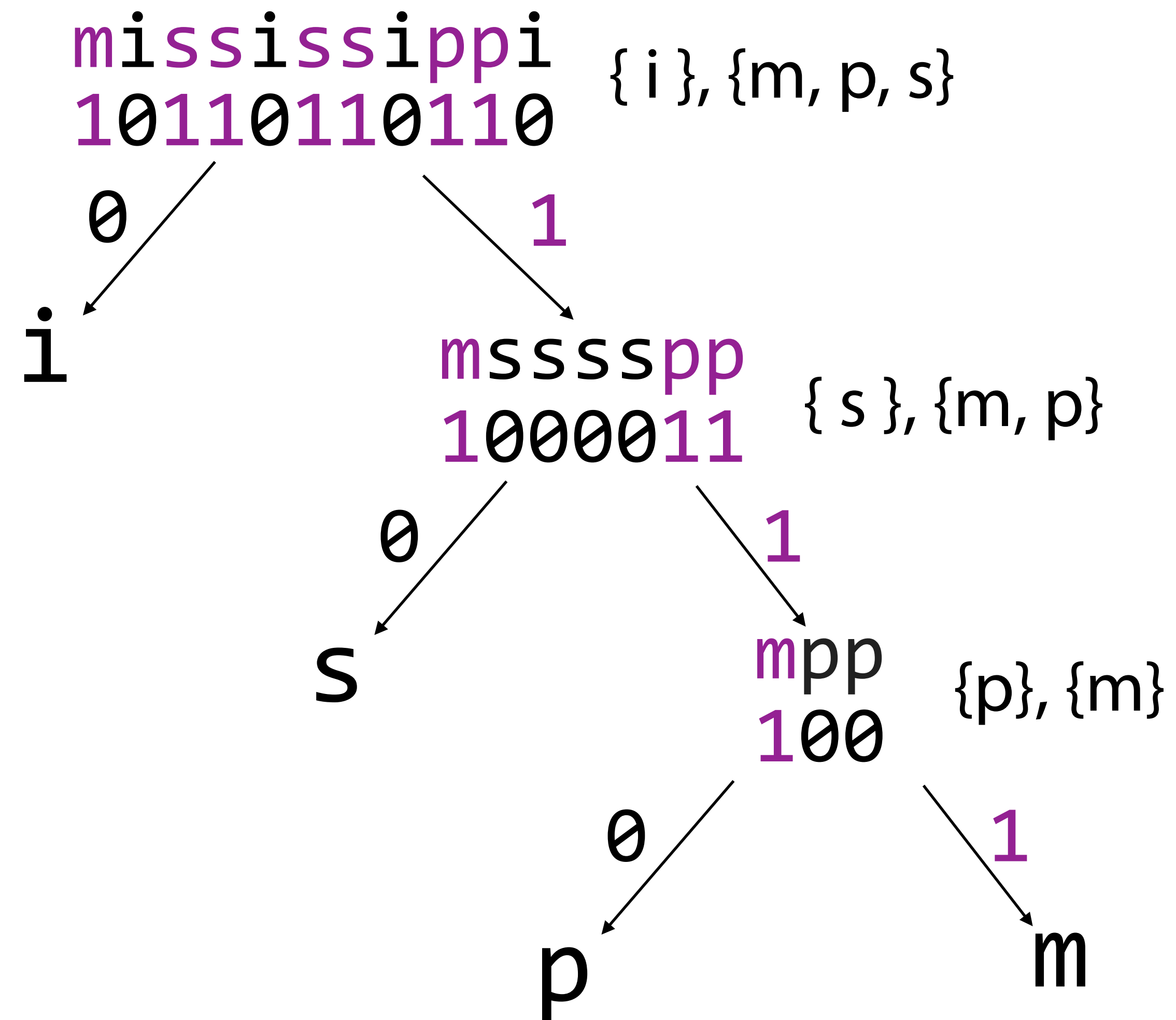
Wavelet trees

Could have picked a different shape for the tree



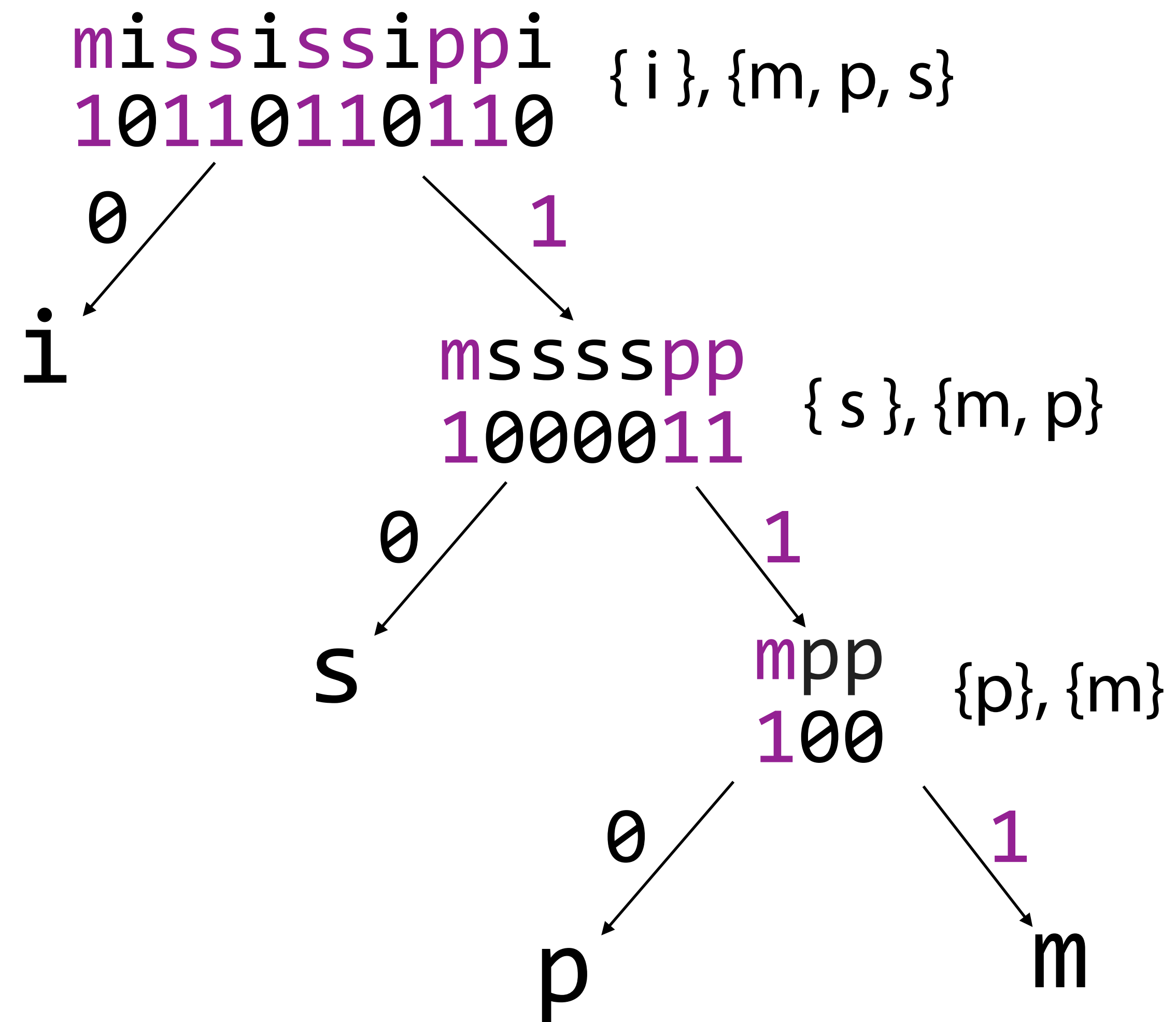
Wavelet trees

Could have picked a different shape for the tree



Wavelet trees

Tree shape
defines a (prefix)
code



Wavelet trees

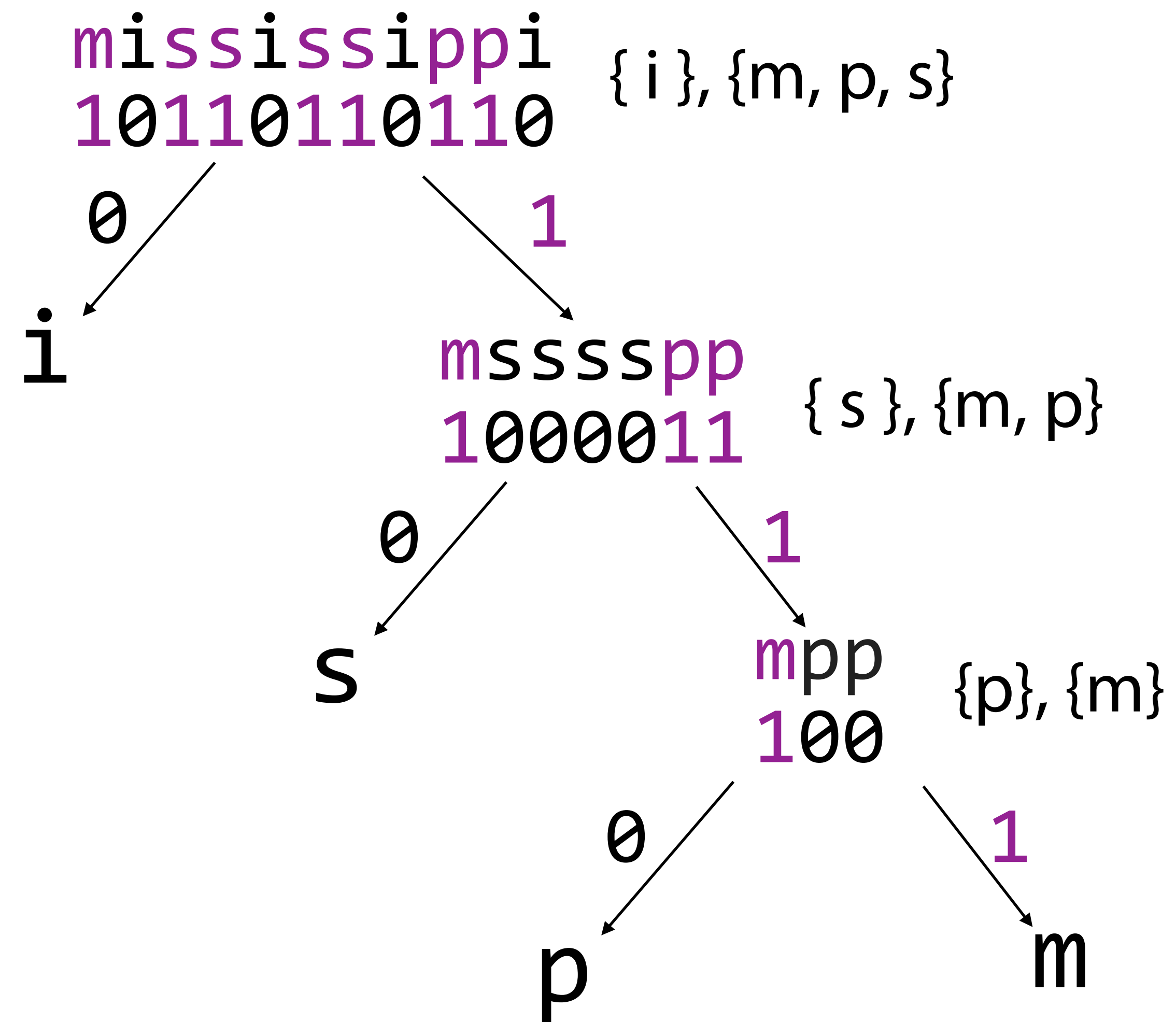
Tree shape
defines a (prefix)
code

$$C(i) = 0$$

$$C(s) = 10$$

$$C(p) = 110$$

$$C(m) = 111$$



Wavelet trees

Tree shape
defines a (prefix)
code

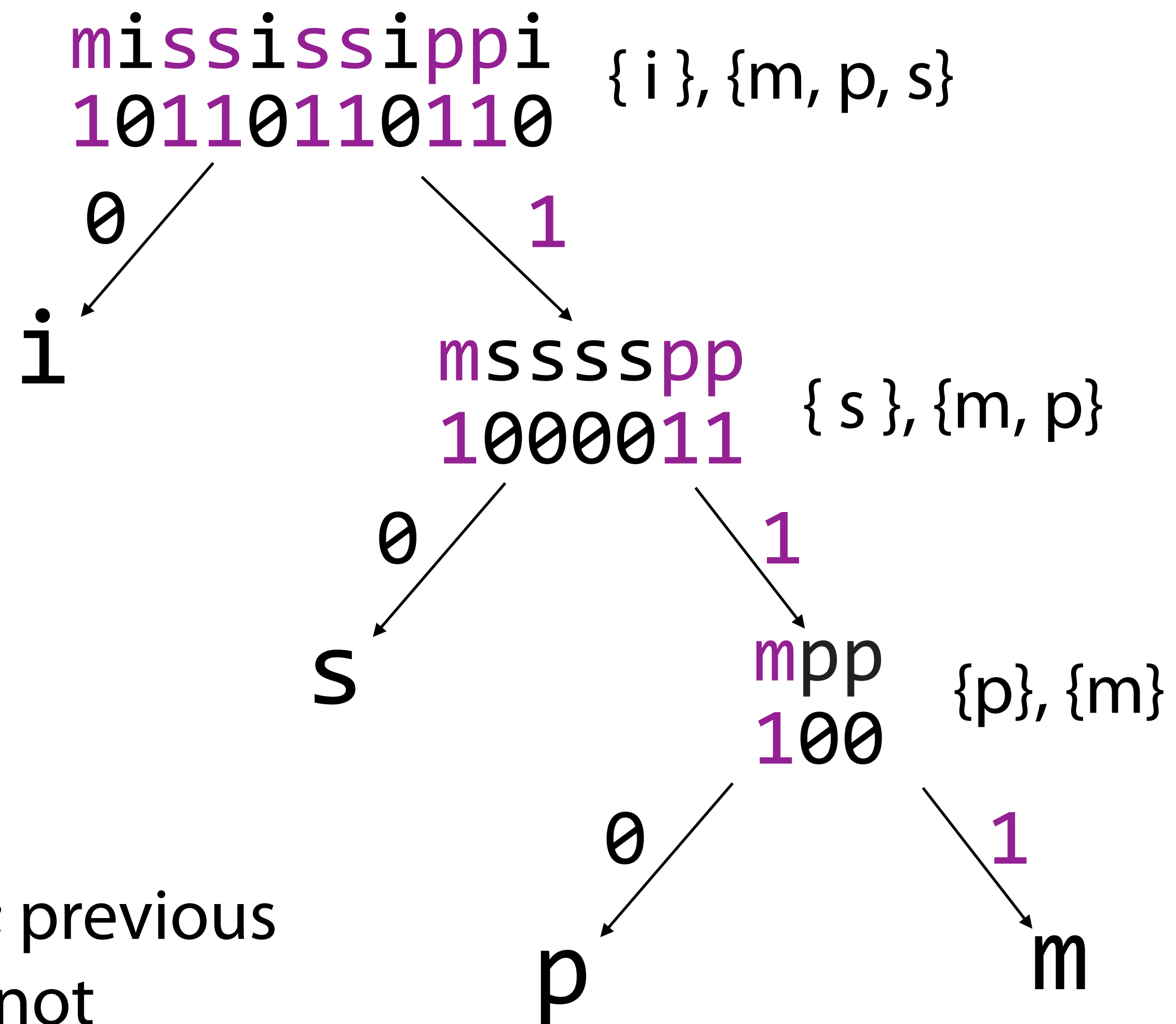
$$C(i) = 0$$

$$C(s) = 10$$

$$C(p) = 110$$

$$C(m) = 111$$

This tree is Huffman; previous
(balanced) tree was not



Wavelet trees

$$S . \text{access}(i) = S[i]$$

$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

$$S . \text{select}_c(i) = \max \{ j \mid S . \text{rank}_c(j) = i \}$$

Note that rank can ask about any character c at any position i

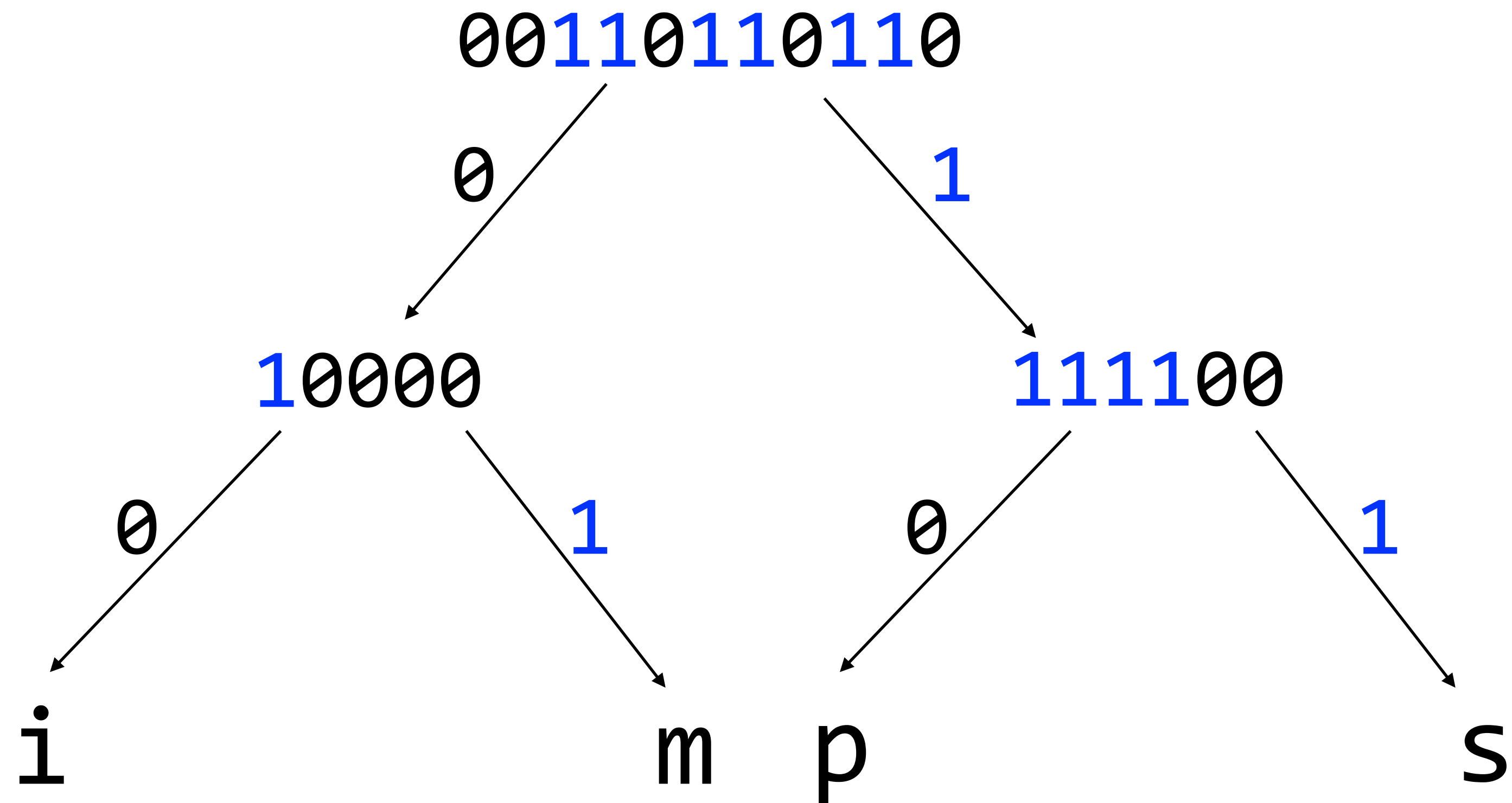
Wavelet trees

$$S.\text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

Algorithm will be similar to access...

Wavelet trees

$S.\text{rank}_i(7)$



Wavelet trees

$S.\text{rank}_i(7)$



00**11**0**11**0**11**0. $\text{access}(7) = 0$

0

1

10000

111100

0

1

0

1

i

m

p

s

Wavelet trees

$S.\text{rank}_i(7)$



00**11**0**11**0**11**0. $\text{access}(7) = 0$

0

1

10000

111100

0

1

0

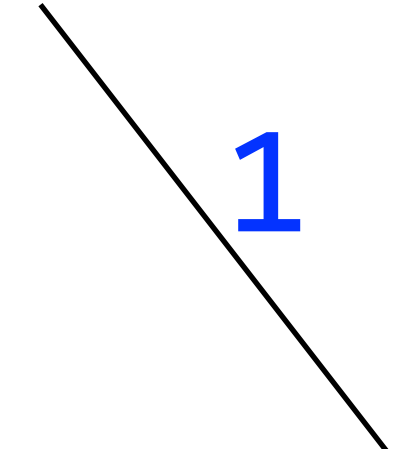
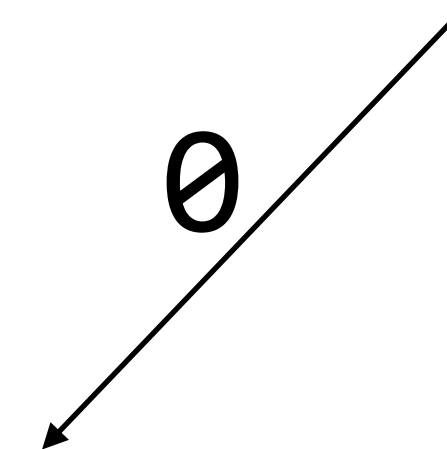
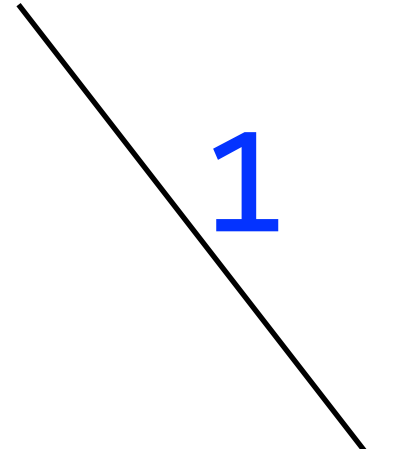
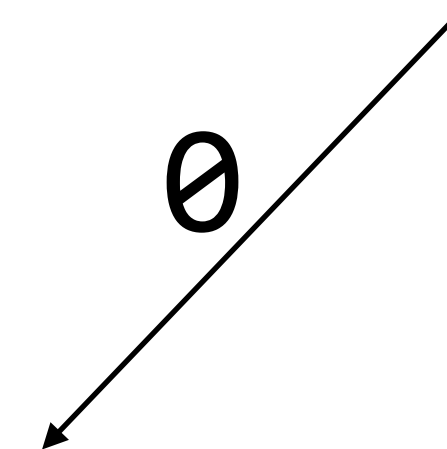
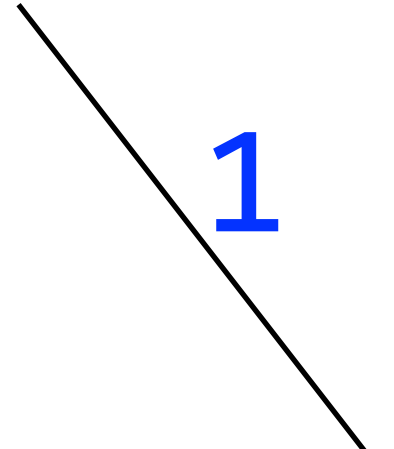
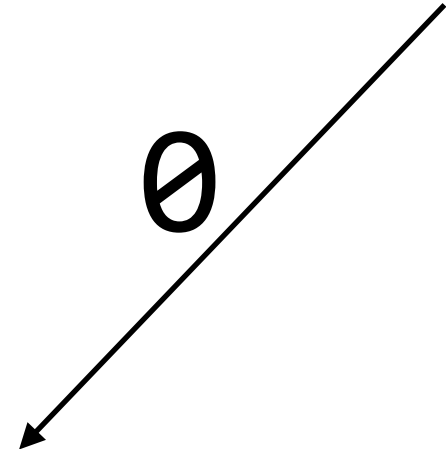
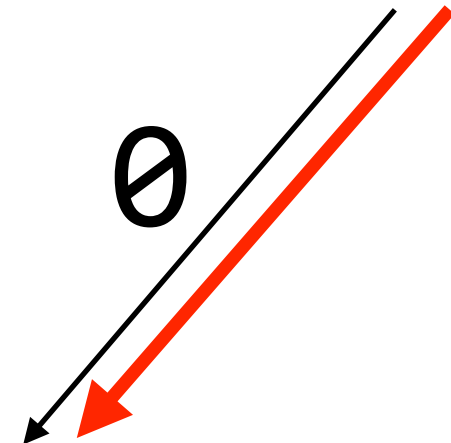
1

i

m

p

s

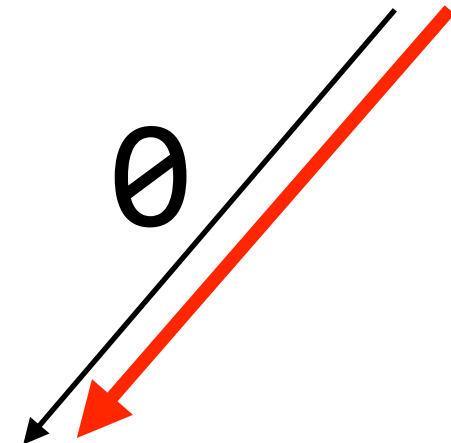


Wavelet trees

$S.\text{rank}_i(7)$



00**1**10**1**10**1**10. $\text{access}(7) = 0$



10000

θ

1

i

m

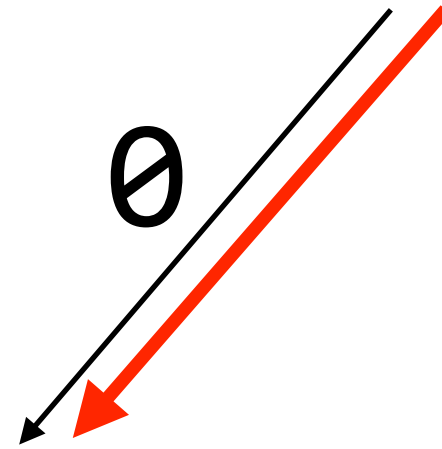
Wavelet trees

$S.\text{rank}_i(7)$



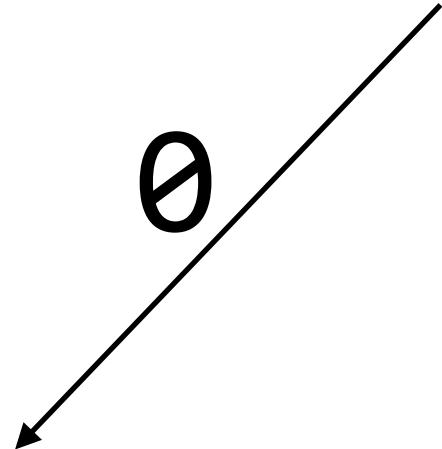
00**1**10**1**10**1**10. $\text{access}(7) = 0$

$\emptyset$



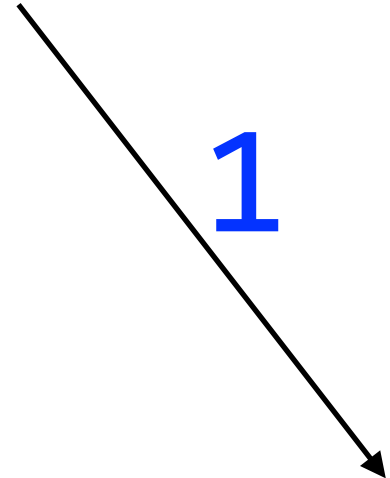
10000. $\text{access}(\text{parent}.\text{rank}_0(7)) = \text{access}(3) = 0$

$\emptyset$



i

1



m

Wavelet trees

$S.\text{rank}_i(7)$



00**1**10**1**10**1**10. $\text{access}(7) = 0$

$\emptyset$



10000. $\text{access}(\text{parent}.\text{rank}_0(7)) = \text{access}(3) = 0$

$\emptyset$

i

1

m

Wavelet trees

$S.\text{rank}_i(7)$



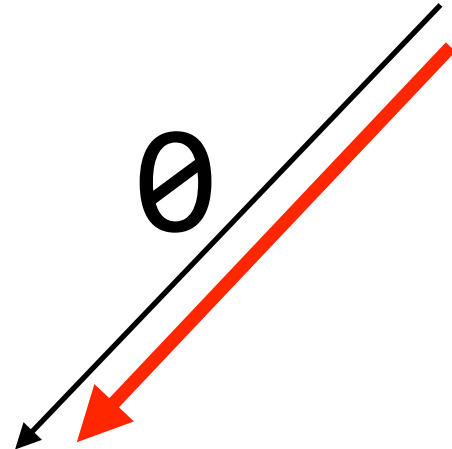
00**1**10**1**10**1**10. $\text{access}(7) = 0$

$\emptyset$



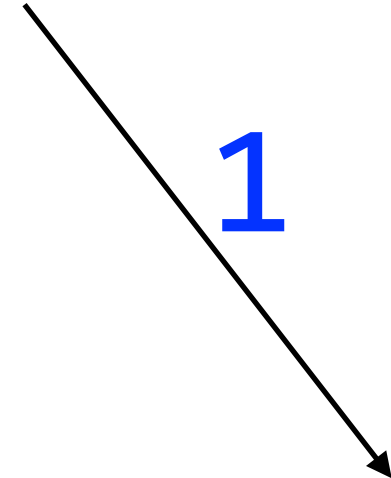
10000. $\text{access}(\text{parent}.\text{rank}_0(7)) = \text{access}(3) = 0$

$\emptyset$



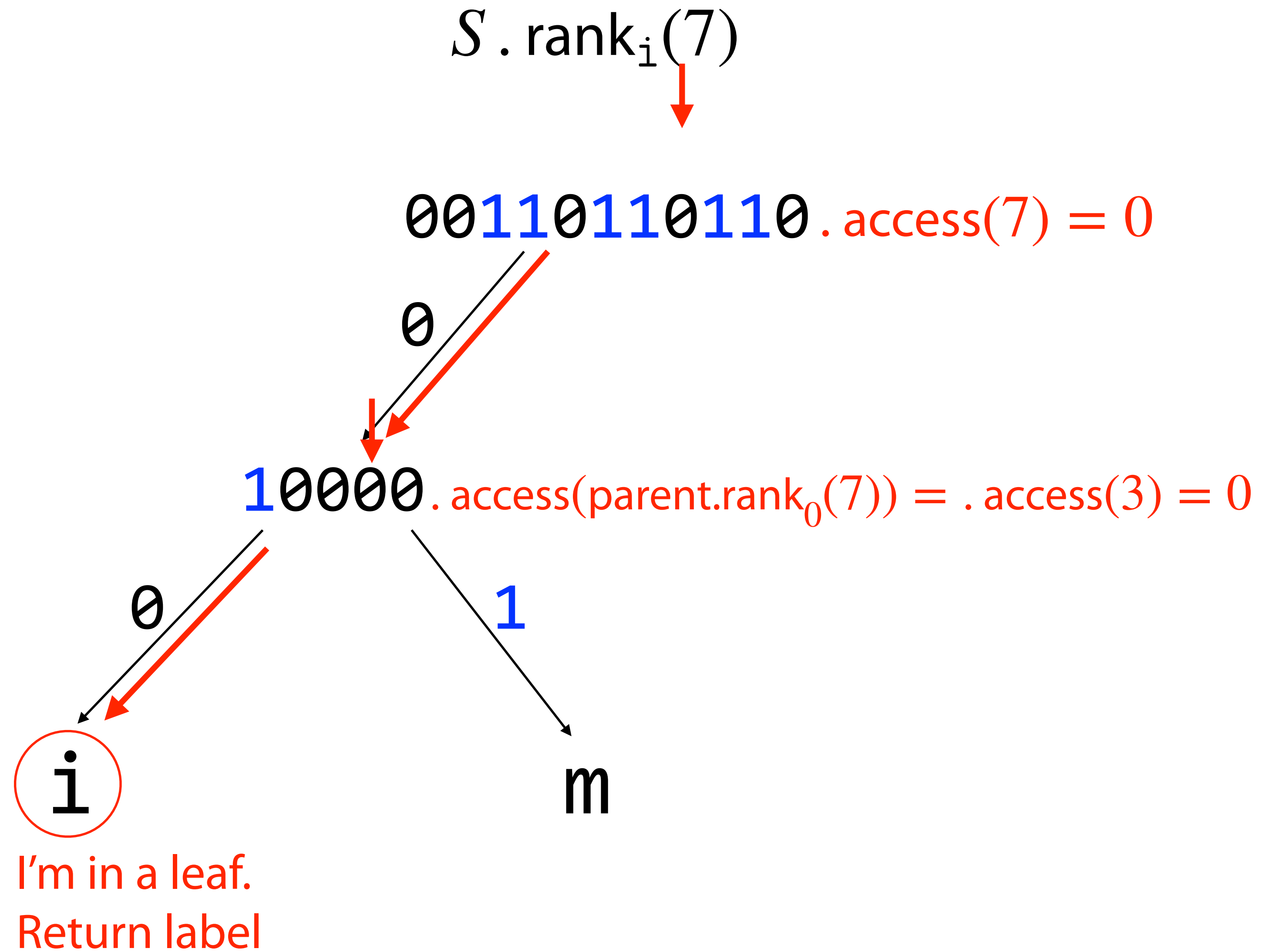
i

1

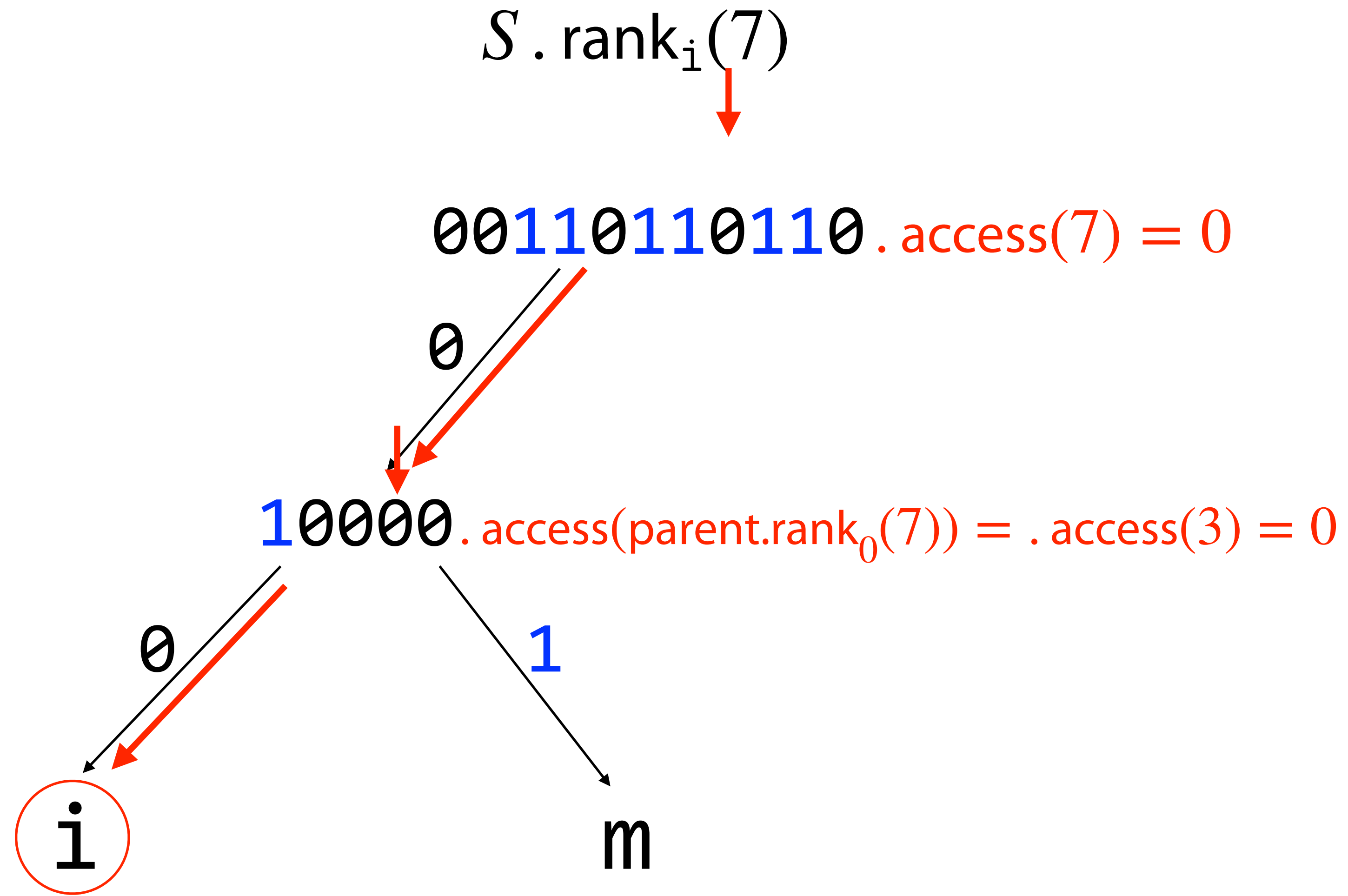


m

Wavelet trees



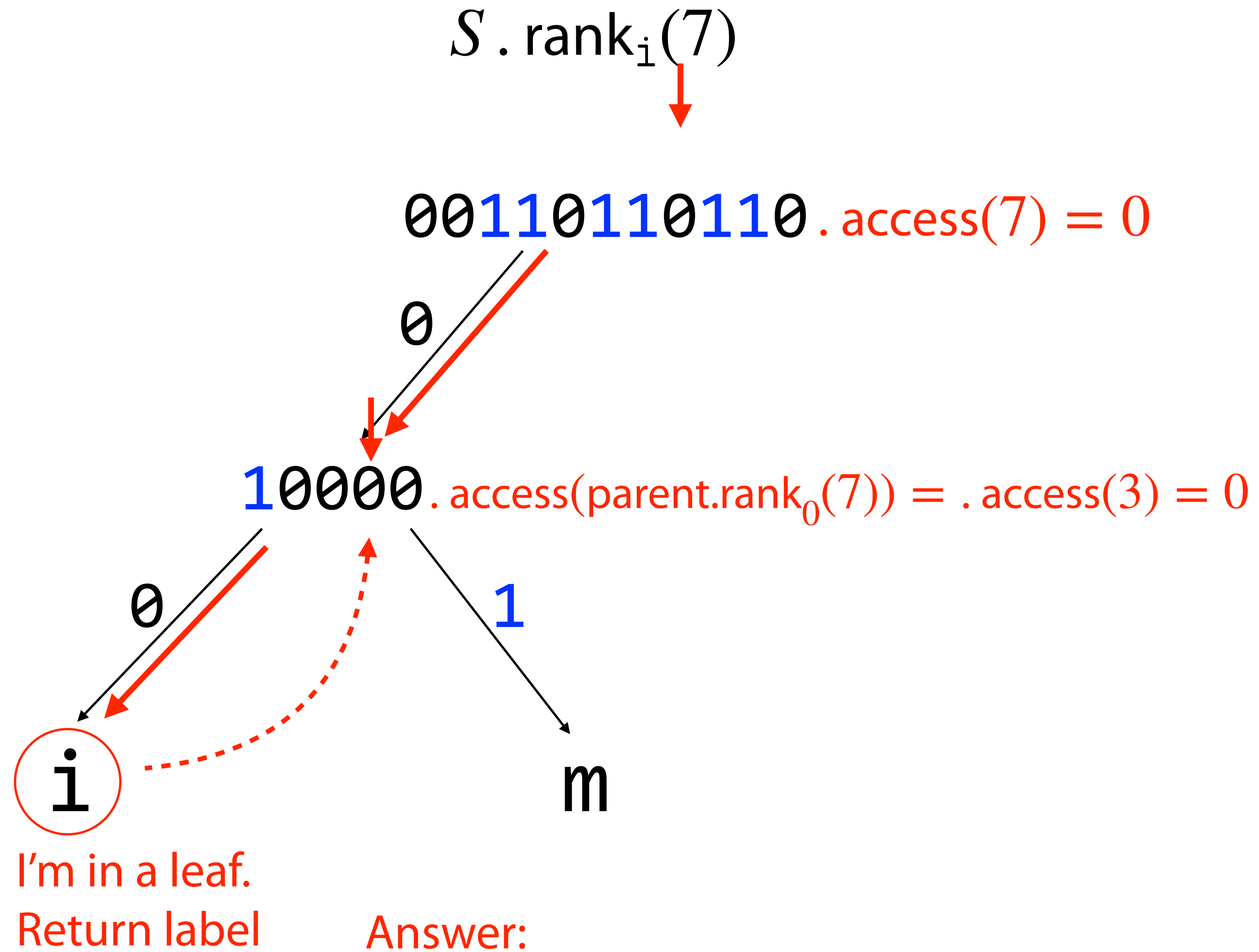
Wavelet trees



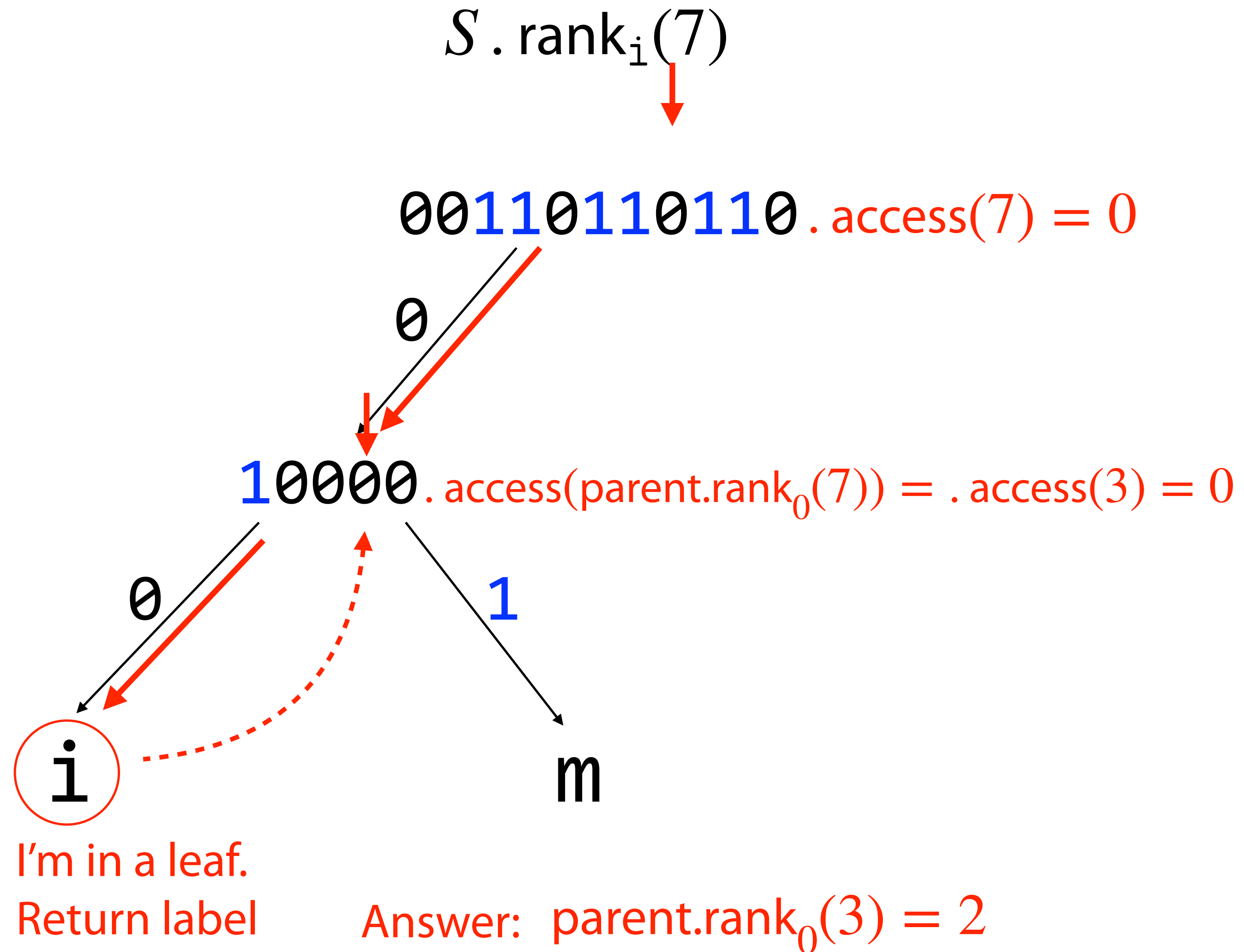
I'm in a leaf.
Return label

So....where's the answer?

Wavelet trees



Wavelet trees



Wavelet trees

$$S.\text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

Algorithm will be similar to access...

Wavelet trees

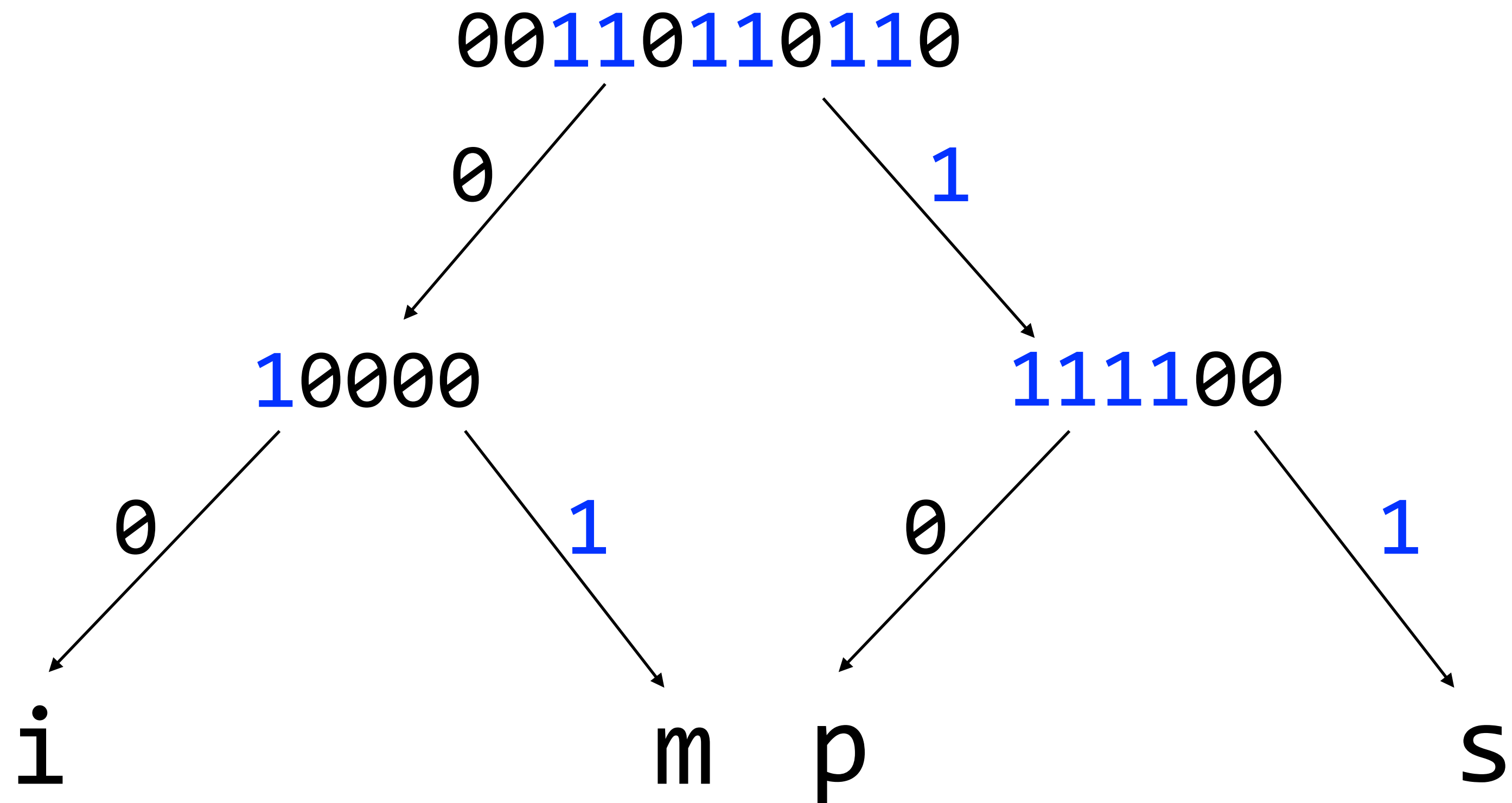
$$S.\text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

Algorithm will be similar to access...

But the path we follow corresponds to c , which isn't necessarily the character at $S[i]$

Wavelet trees

$S.\text{rank}_i(6)$

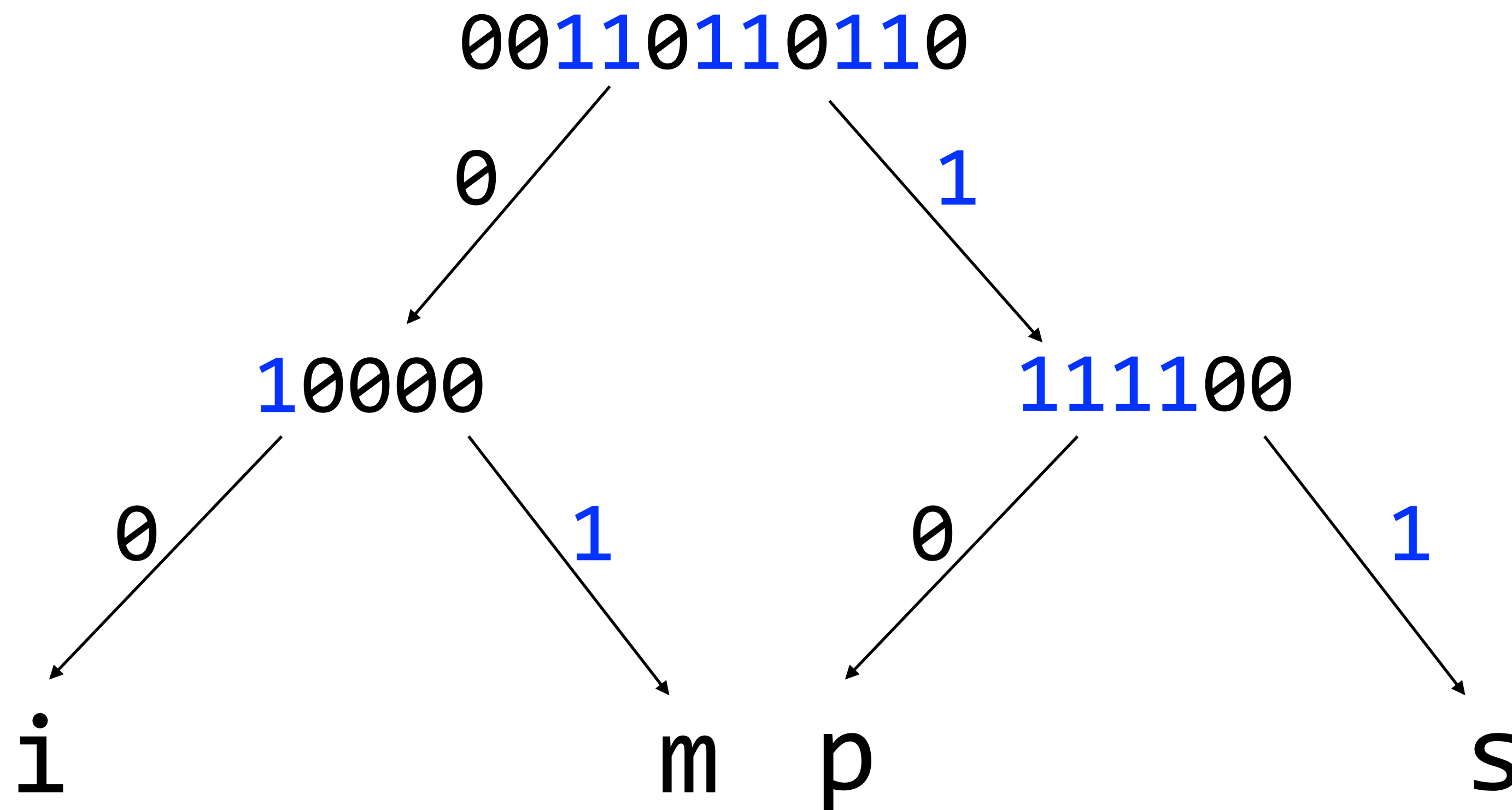


Wavelet trees

$S.\text{rank}_i(6)$



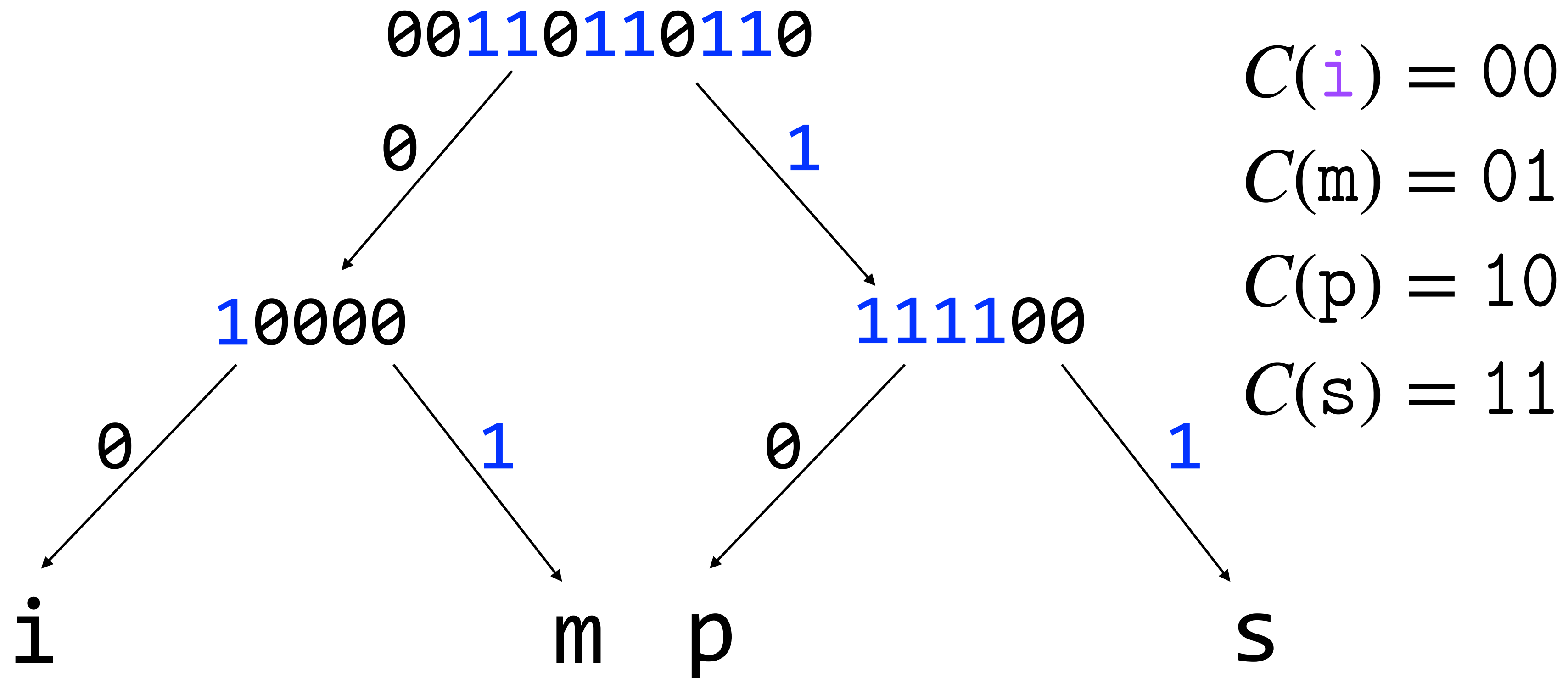
Note: $S[6] = \textcolor{red}{s} \neq \textcolor{violet}{i}$
mississippi



Wavelet trees

$S.\text{rank}_i(6)$
↓

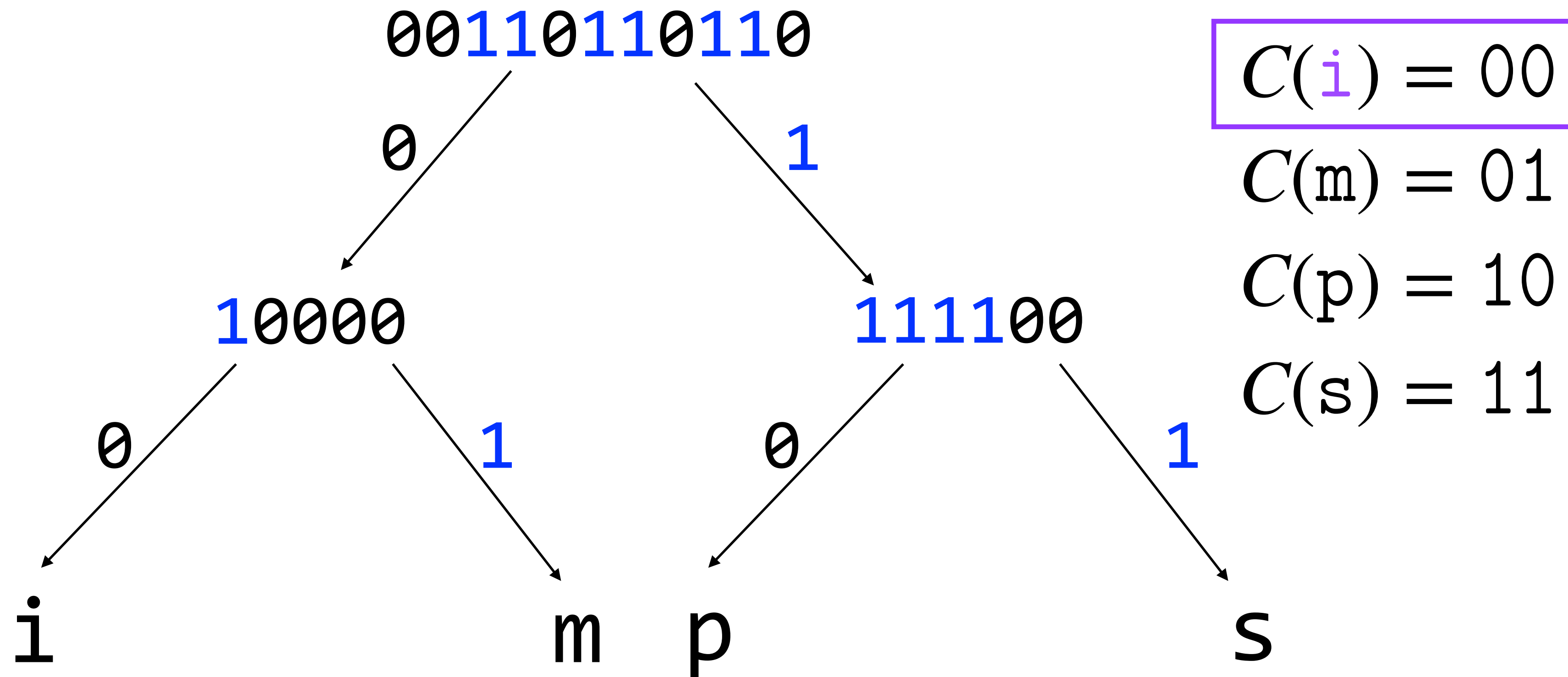
Note: $S[6] = \textcolor{red}{s} \neq \textcolor{violet}{i}$
missis $\textcolor{red}{s}$ ippi



Wavelet trees

$S.\text{rank}_i(6)$
↓

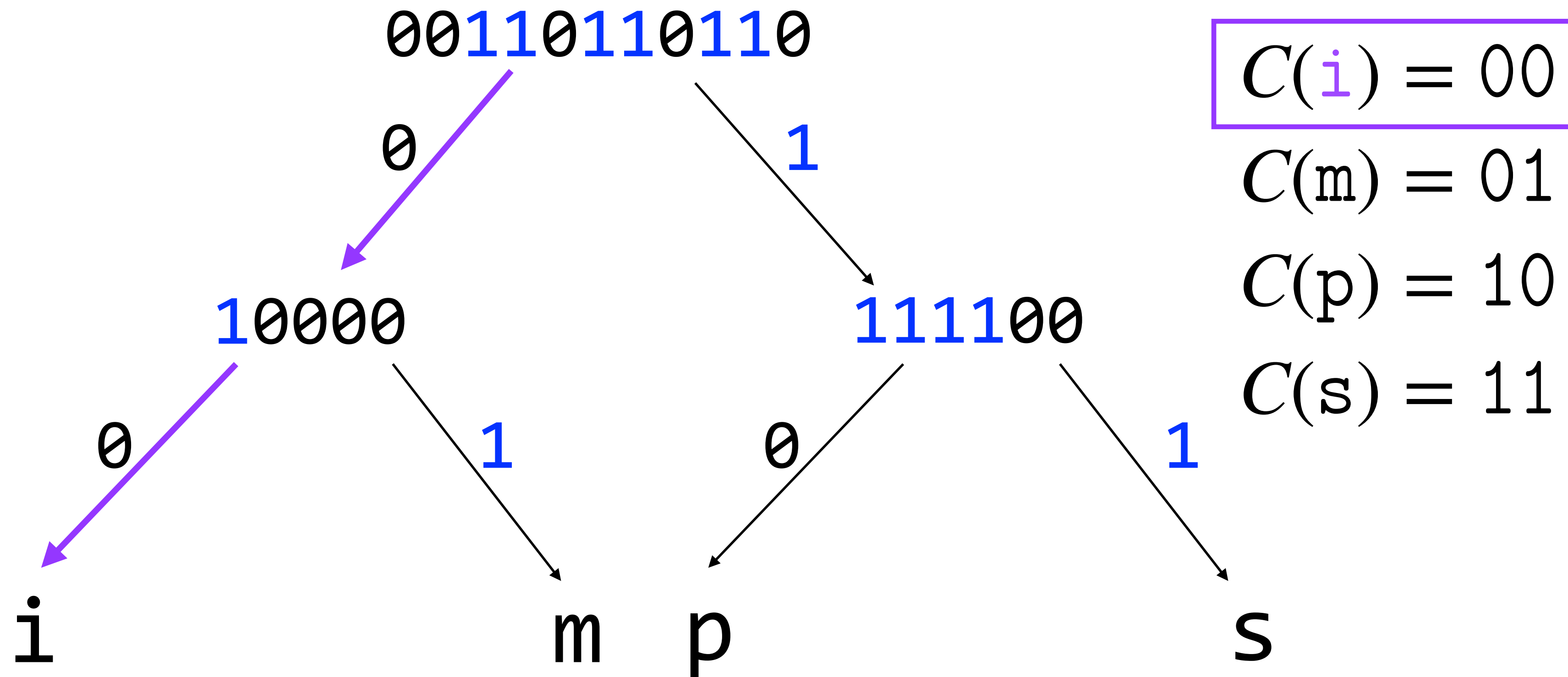
Note: $S[6] = \textcolor{red}{s} \neq \textcolor{violet}{i}$
mississsippi



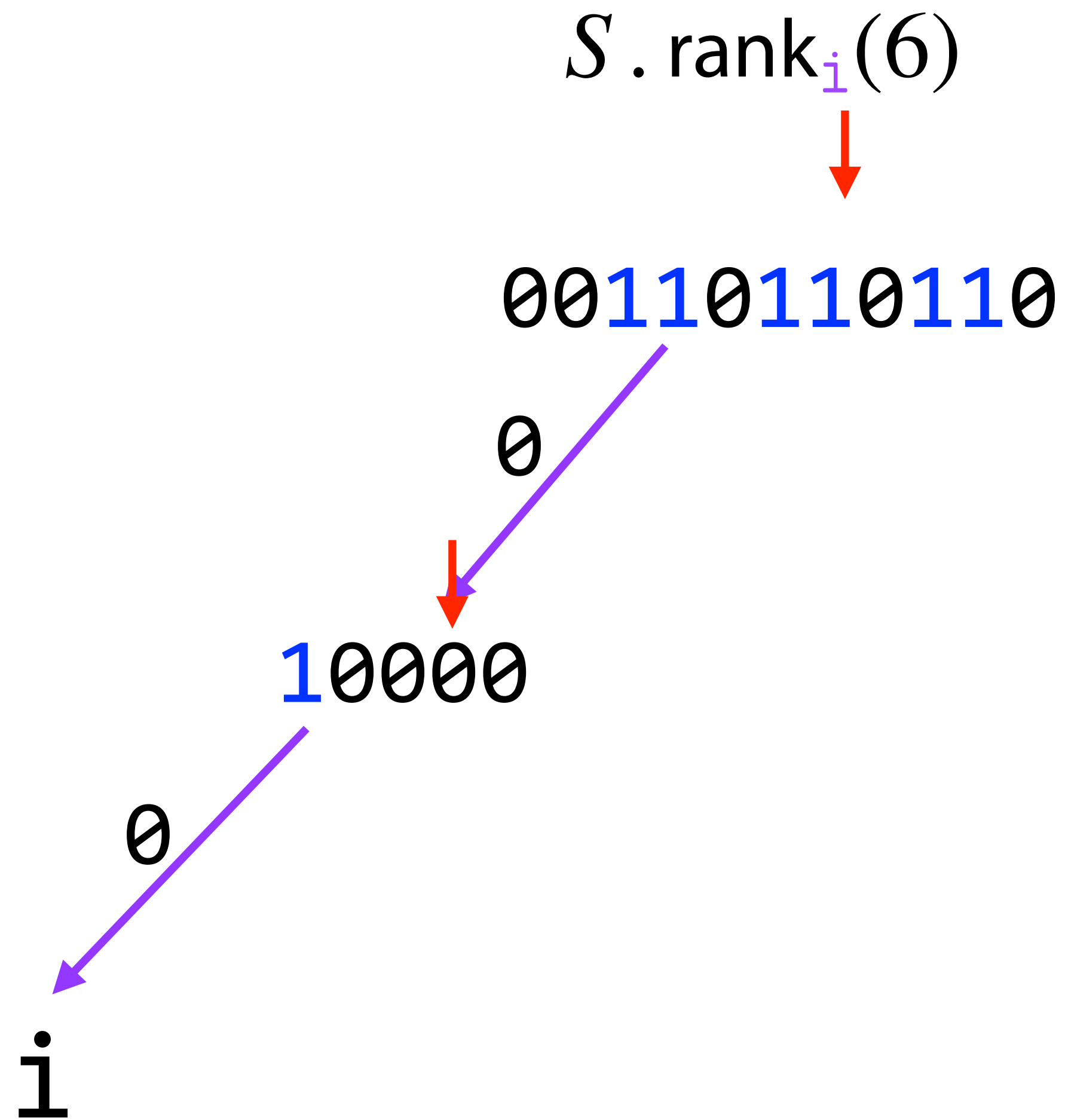
Wavelet trees

$S.\text{rank}_i(6)$

Note: $S[6] = \textcolor{red}{s} \neq \textcolor{violet}{i}$
mississippi



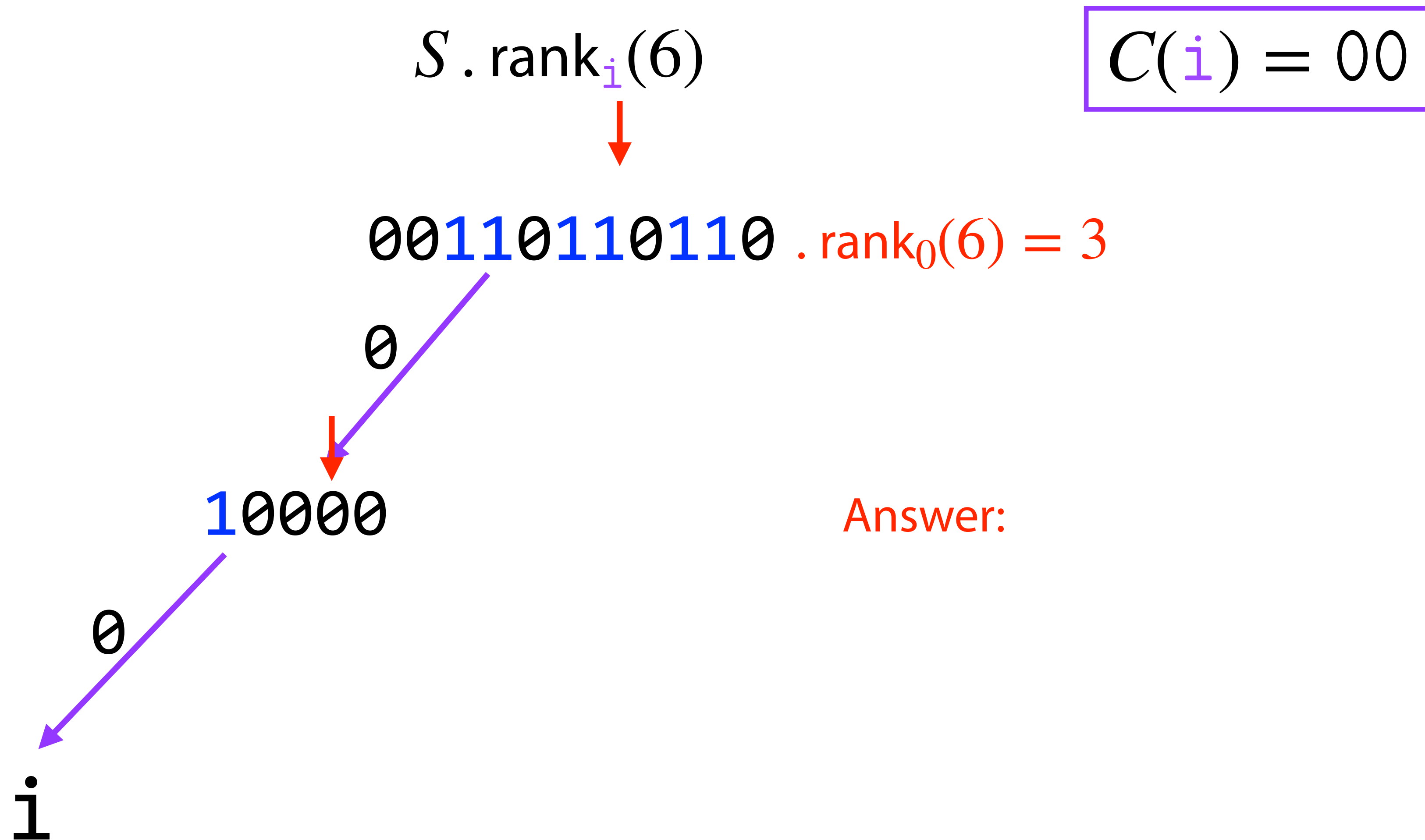
Wavelet trees



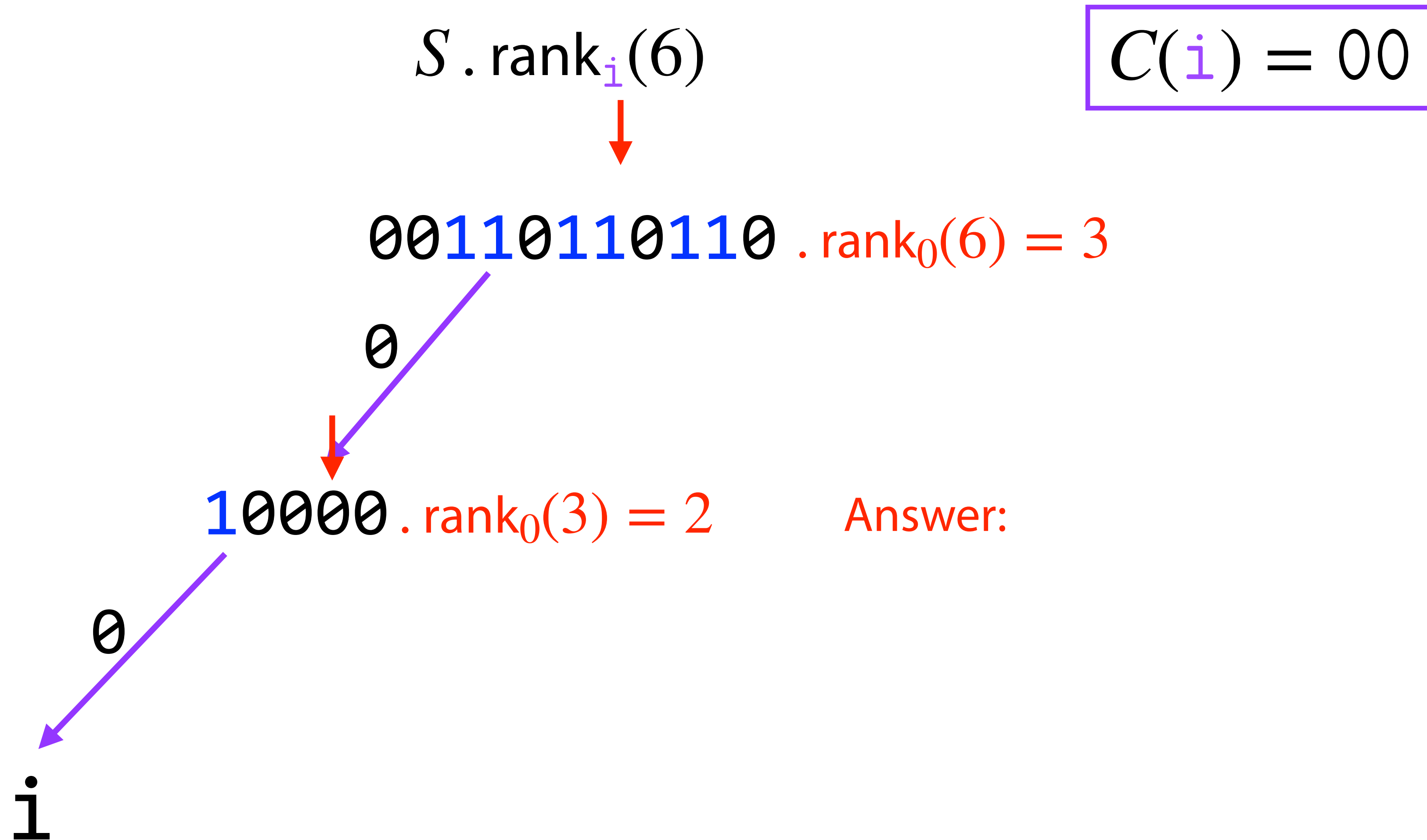
$$C(i) = 00$$

Answer:

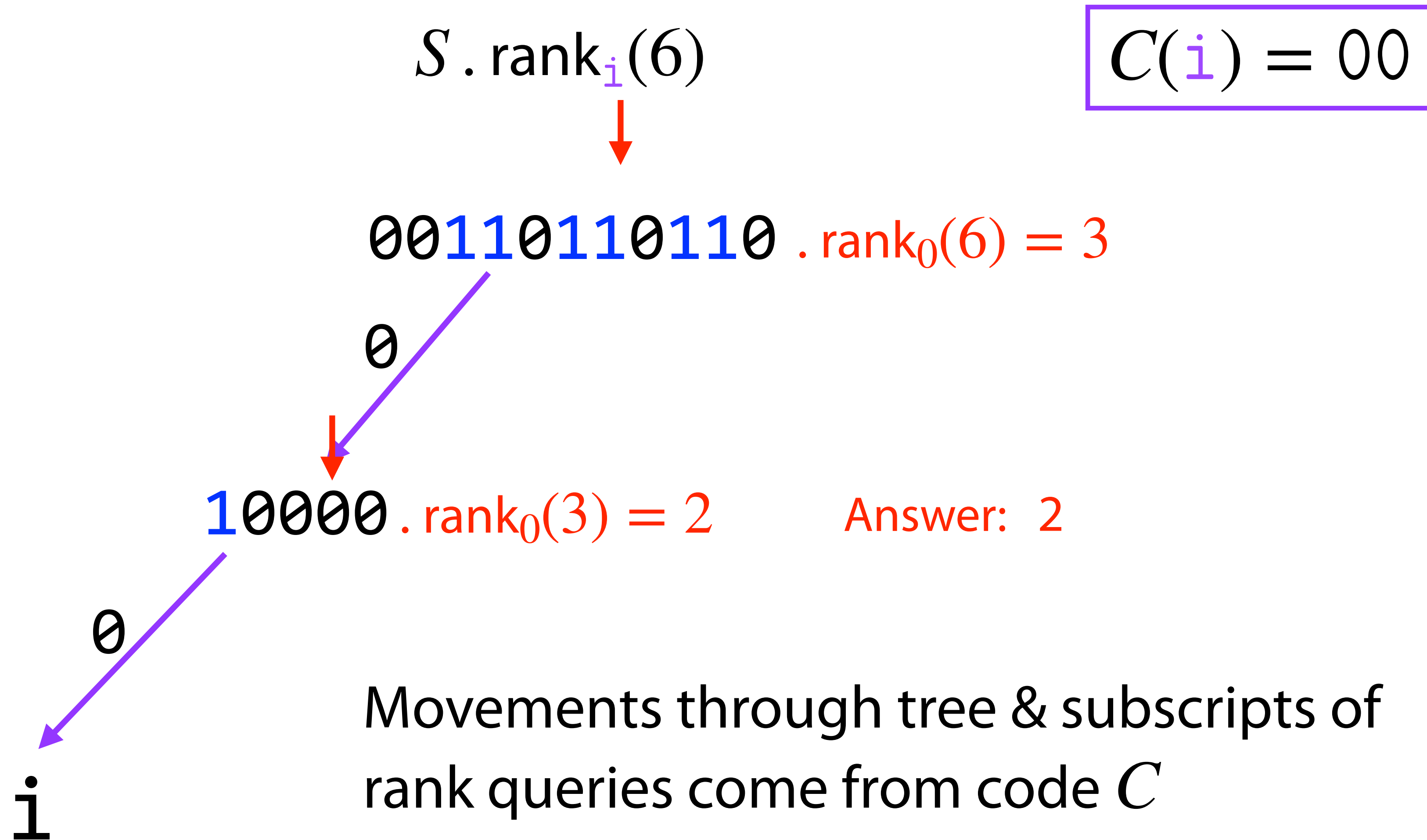
Wavelet trees



Wavelet trees



Wavelet trees



Wavelet trees

Wavelet tree $\text{rank}_x(i)$:

```
 $N \leftarrow \text{root}$   
 $k \leftarrow 0$   
while  $N$  is not leaf  
     $B \leftarrow N.\text{bitvector}$   
     $b \leftarrow c(x)[k]$   
     $i \leftarrow B.\text{rank}_b(i)$   
     $N \leftarrow N.\text{child}(b)$   
     $k \leftarrow k + 1$   
return  $i$ 
```

Wavelet trees

Wavelet tree $\text{rank}_x(i)$:

Given character x and offset i :

$N \leftarrow \text{root}$

$k \leftarrow 0$

while N is not leaf

$B \leftarrow N.\text{bitvector}$

$b \leftarrow c(x)[k]$

$i \leftarrow B.\text{rank}_b(i)$

$N \leftarrow N.\text{child}(b)$

$k \leftarrow k + 1$

return i

Wavelet trees



Wavelet trees

$$S.\text{access}(i) = S[i]$$



Wavelet trees

$$S . \text{access}(i) = S[i]$$

$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$



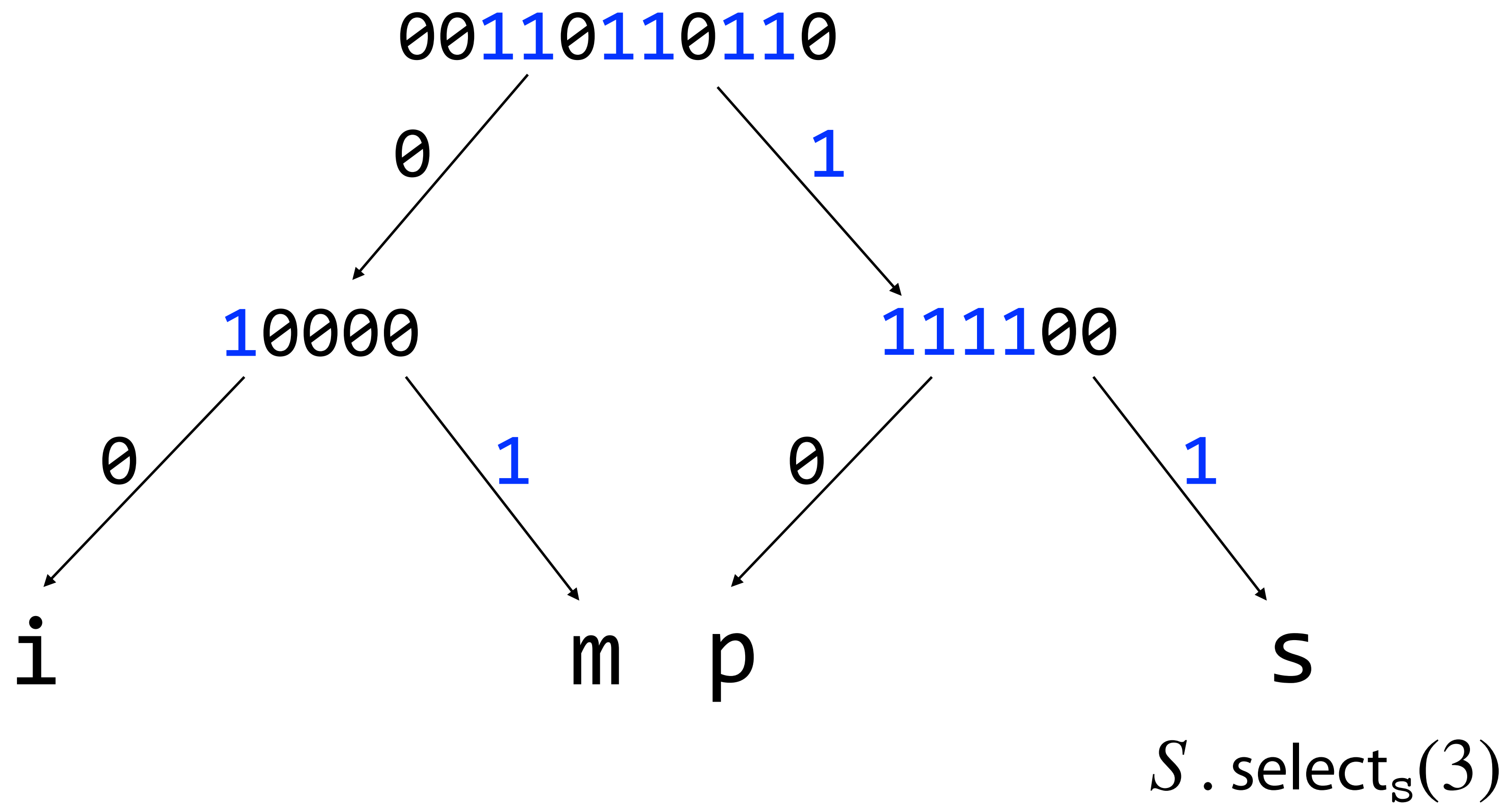
Wavelet trees

$$S . \text{access}(i) = S[i]$$

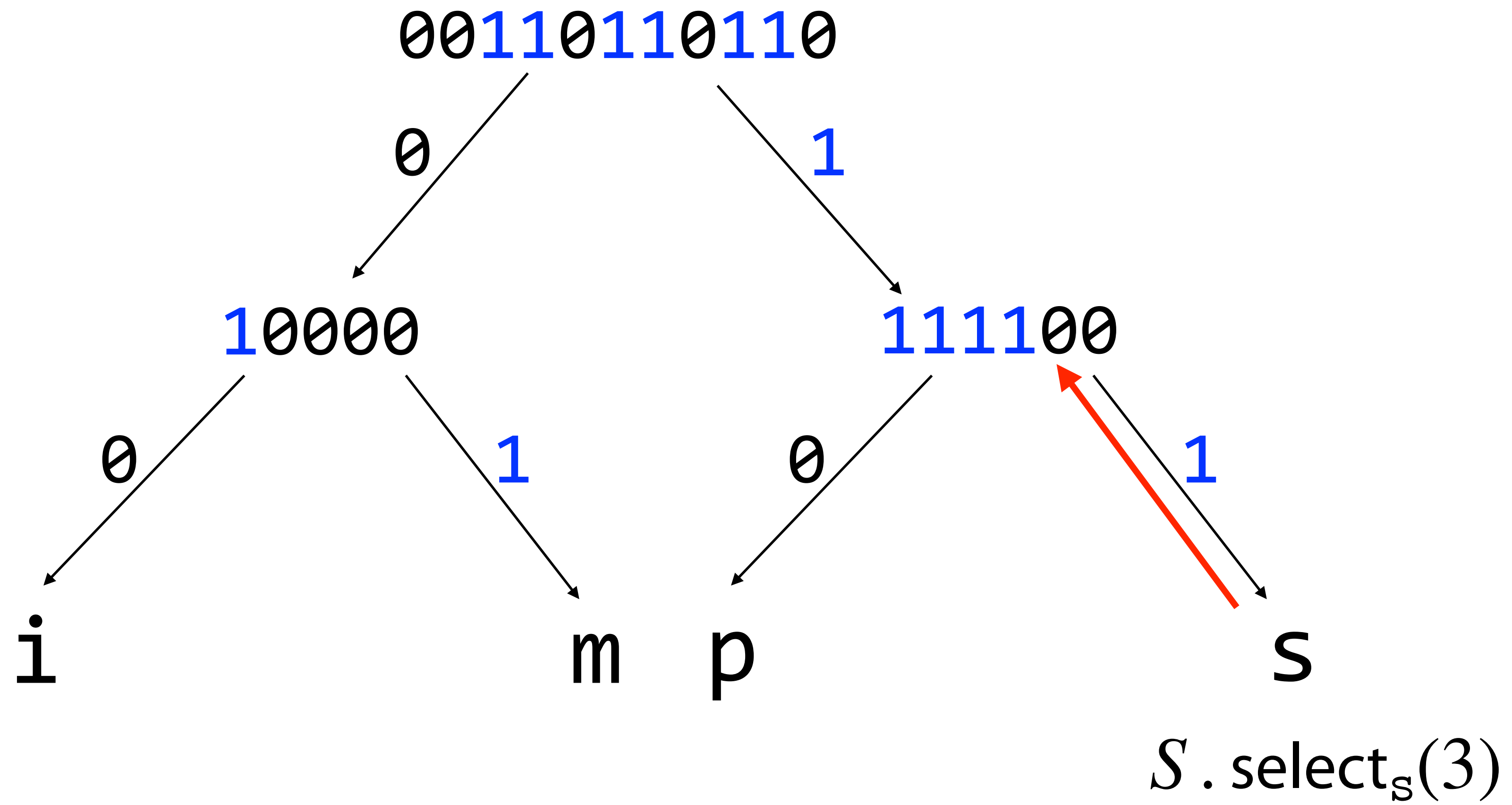
$$S . \text{rank}_c(i) = \sum_{j=0}^{i-1} \begin{cases} 1 & \text{if } S[j] = c \\ 0 & \text{otherwise} \end{cases}$$

$$S . \text{select}_c(i) = \max \{ j \mid S . \text{rank}_c(j) = i \}$$

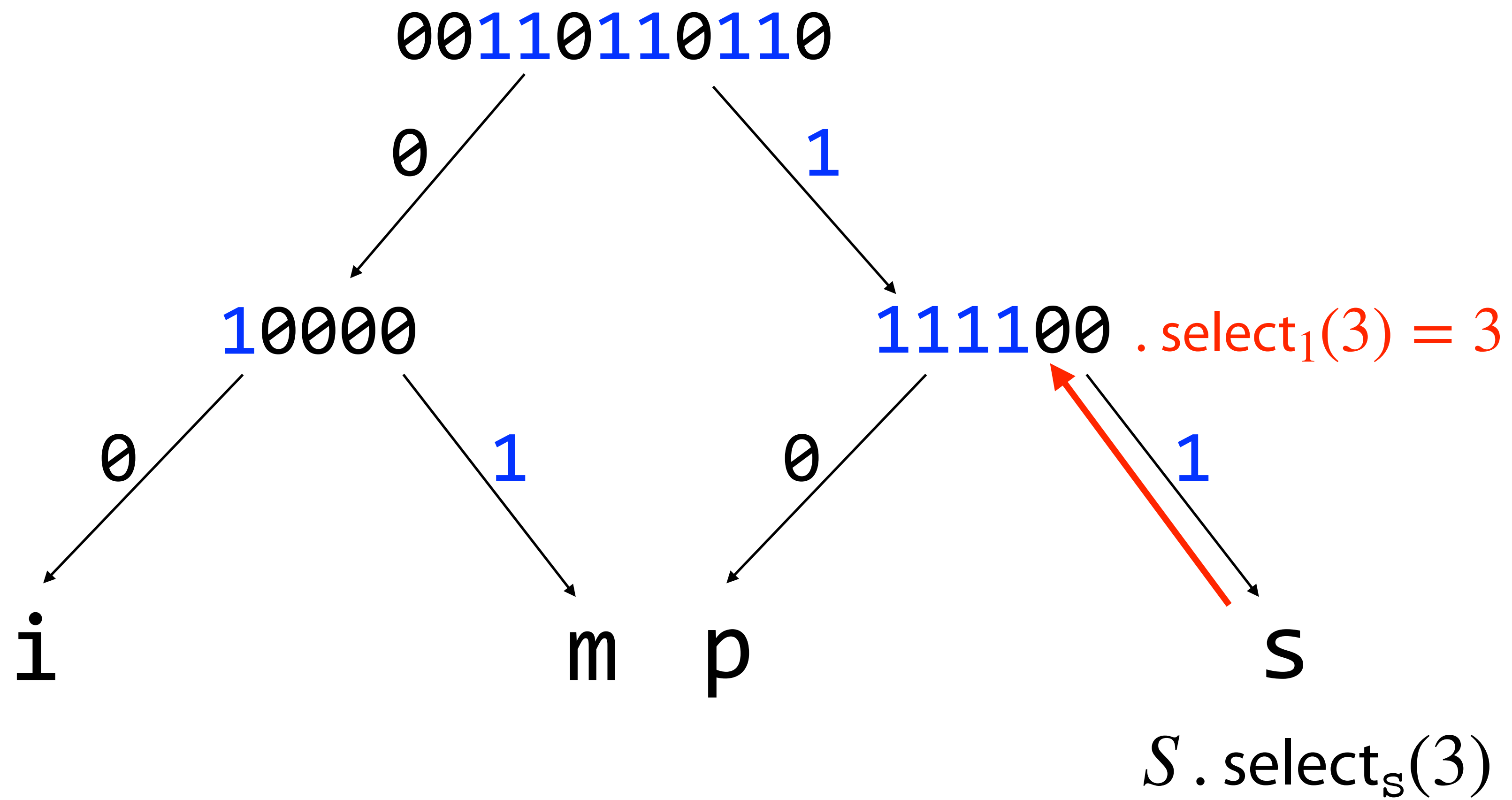
Wavelet trees



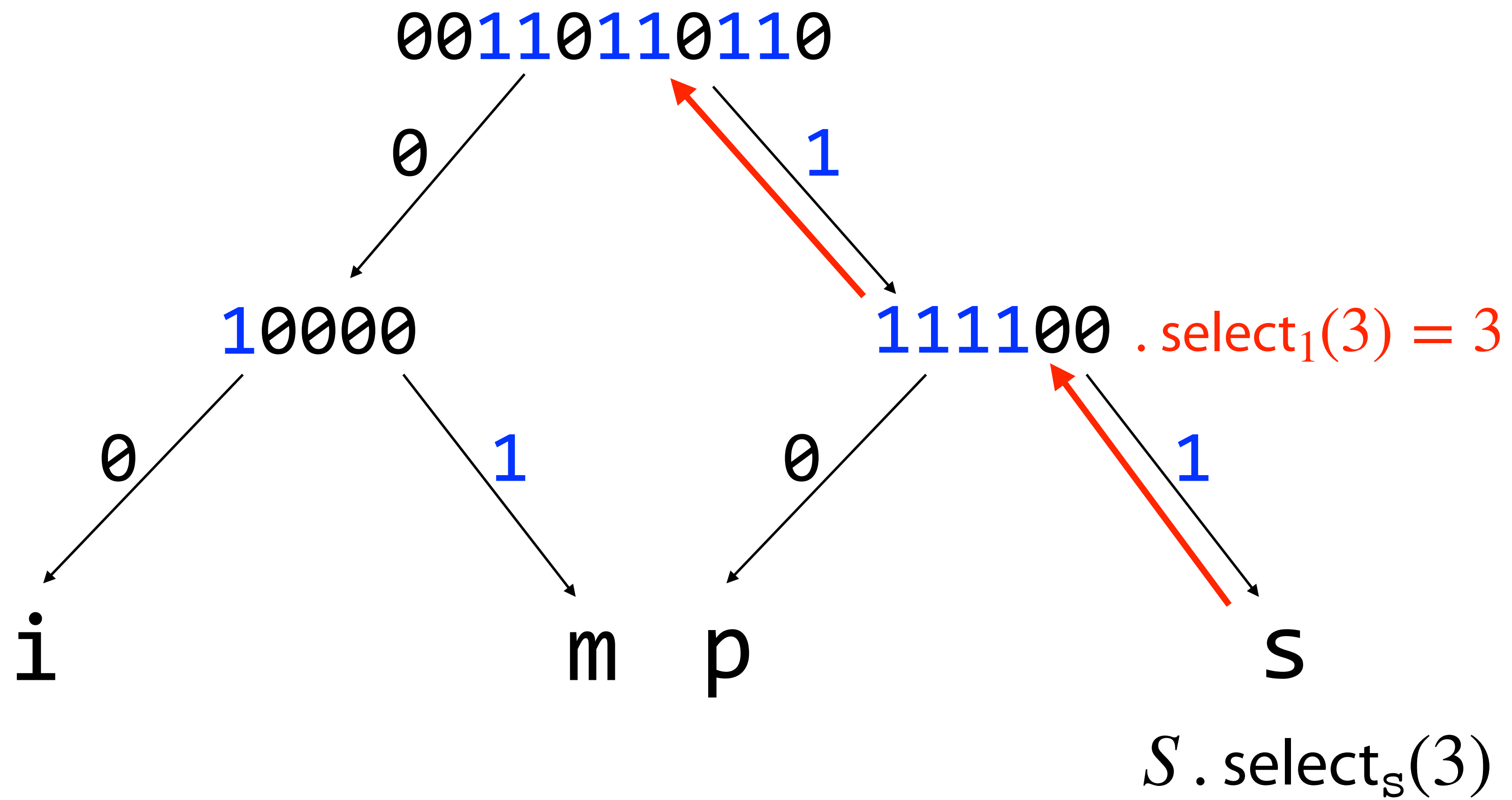
Wavelet trees



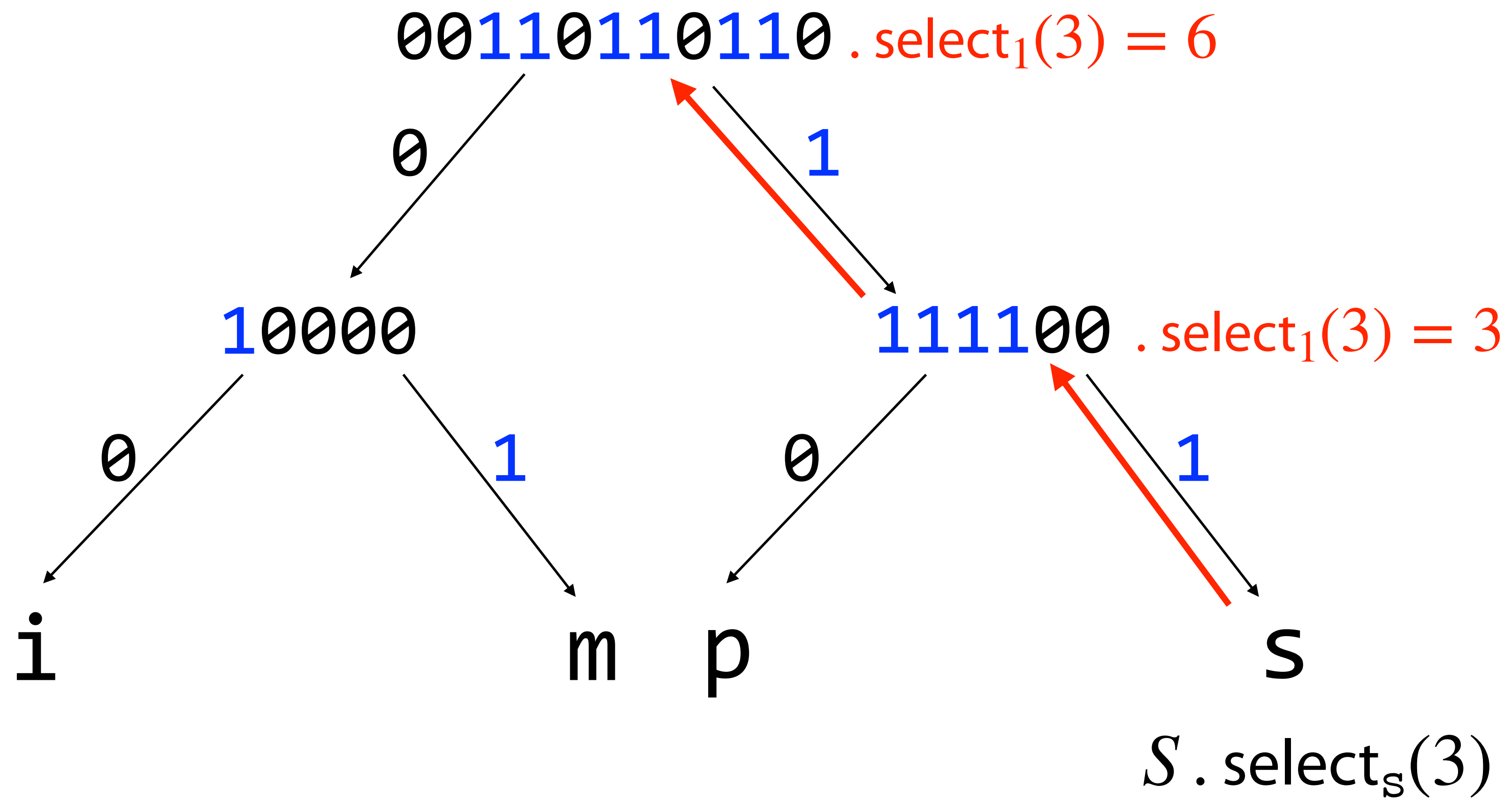
Wavelet trees



Wavelet trees

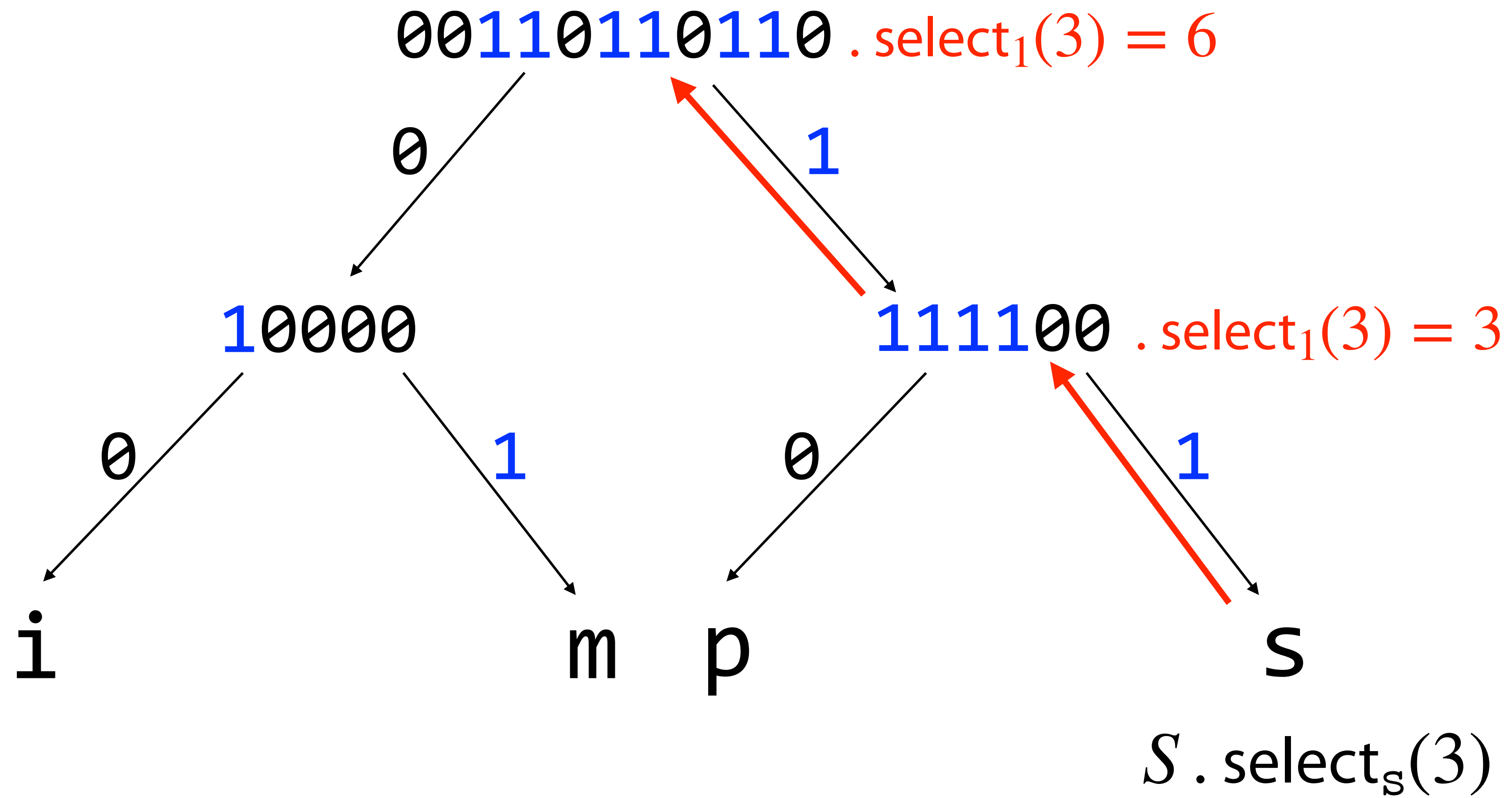


Wavelet trees



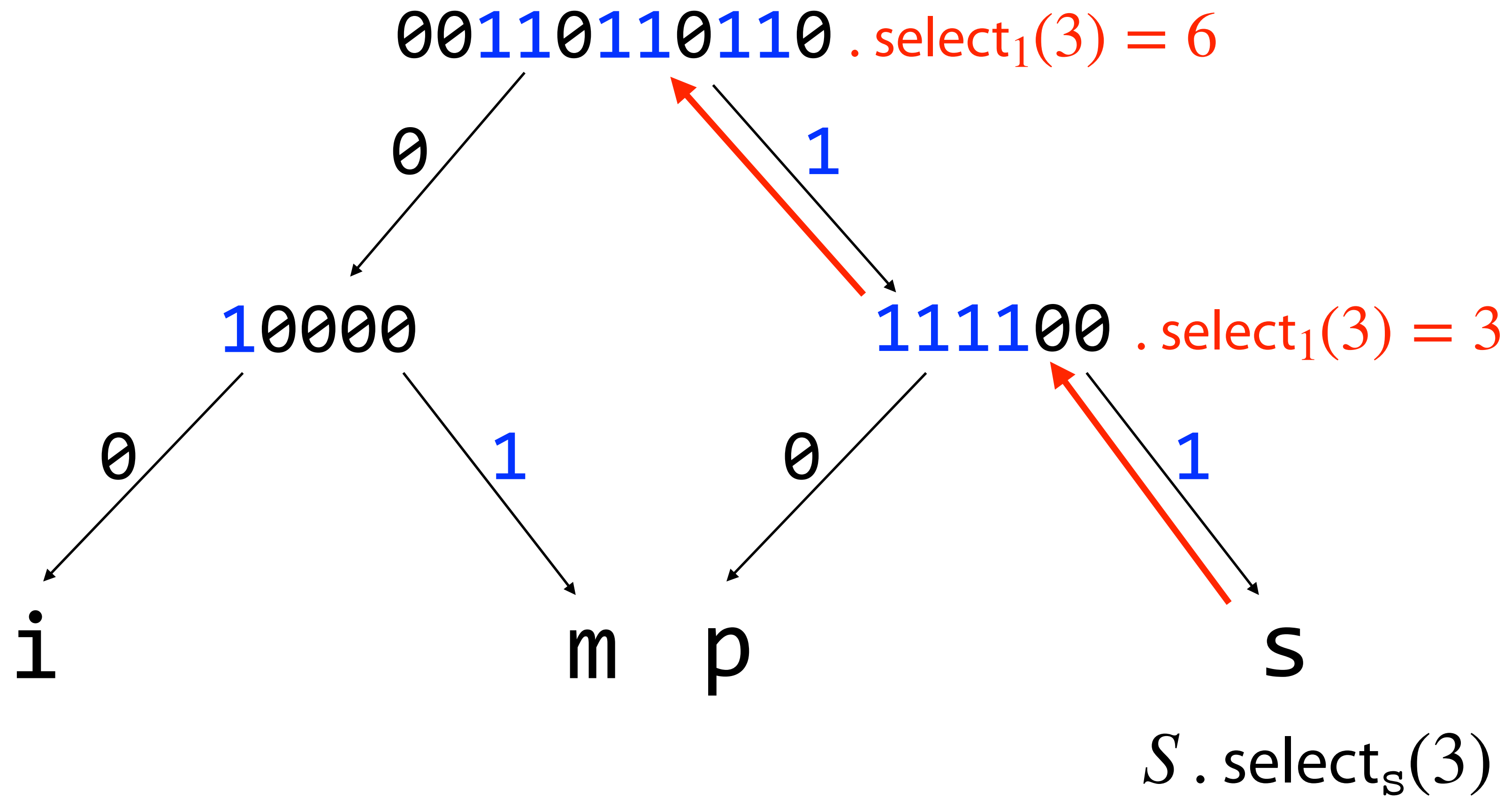
Wavelet trees

Answer:



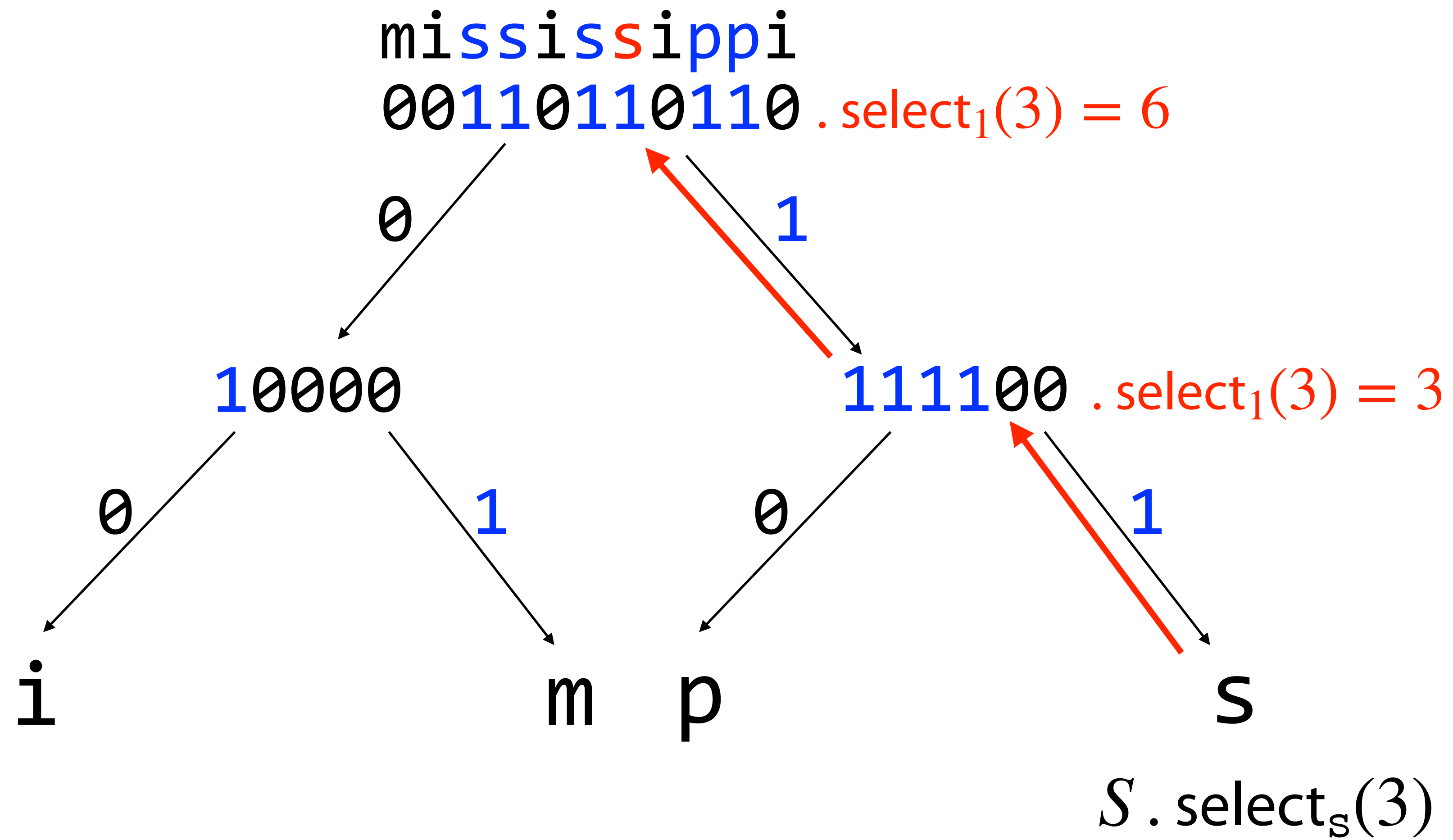
Wavelet trees

Answer: 6



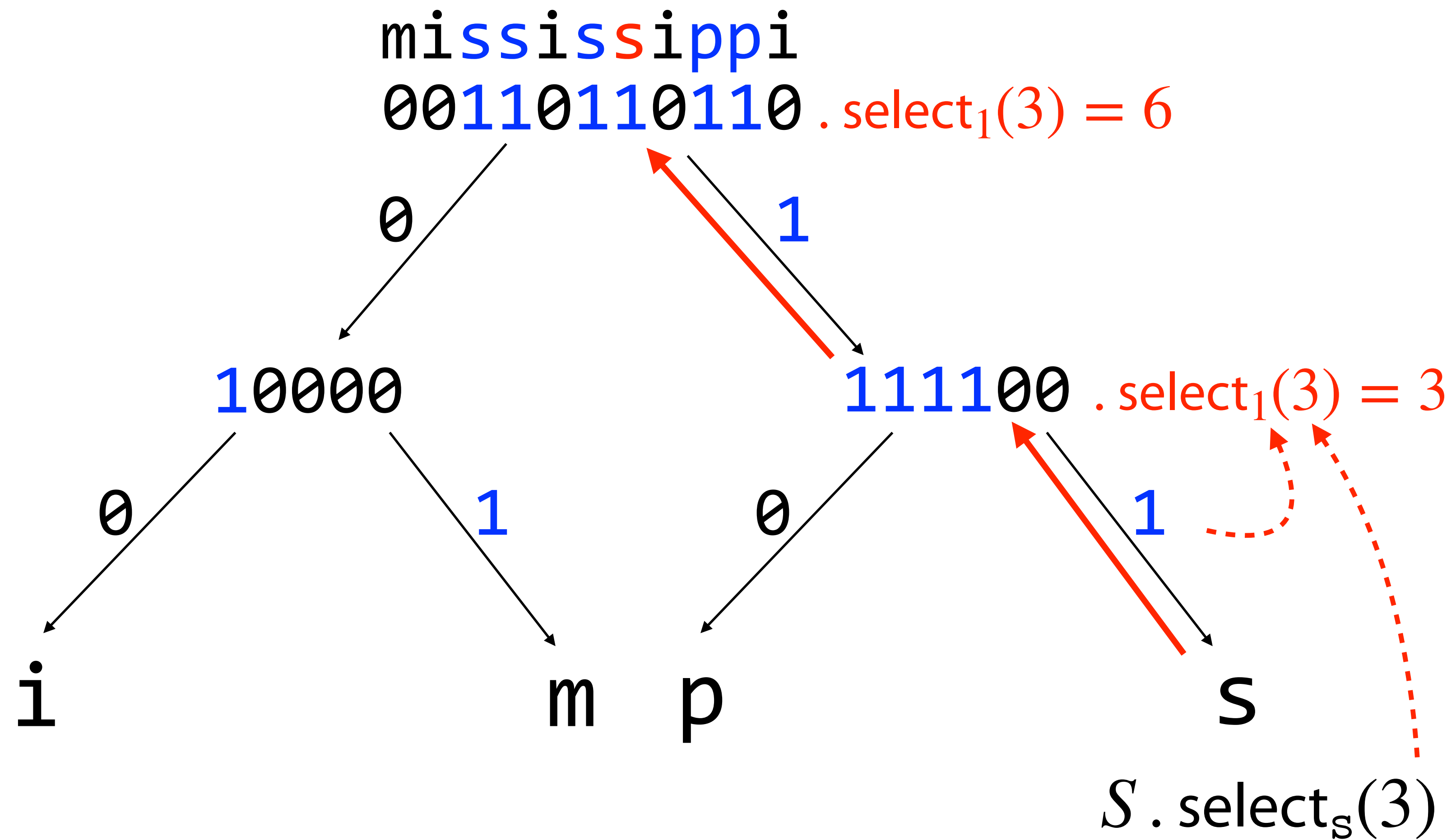
Wavelet trees

Answer: 6



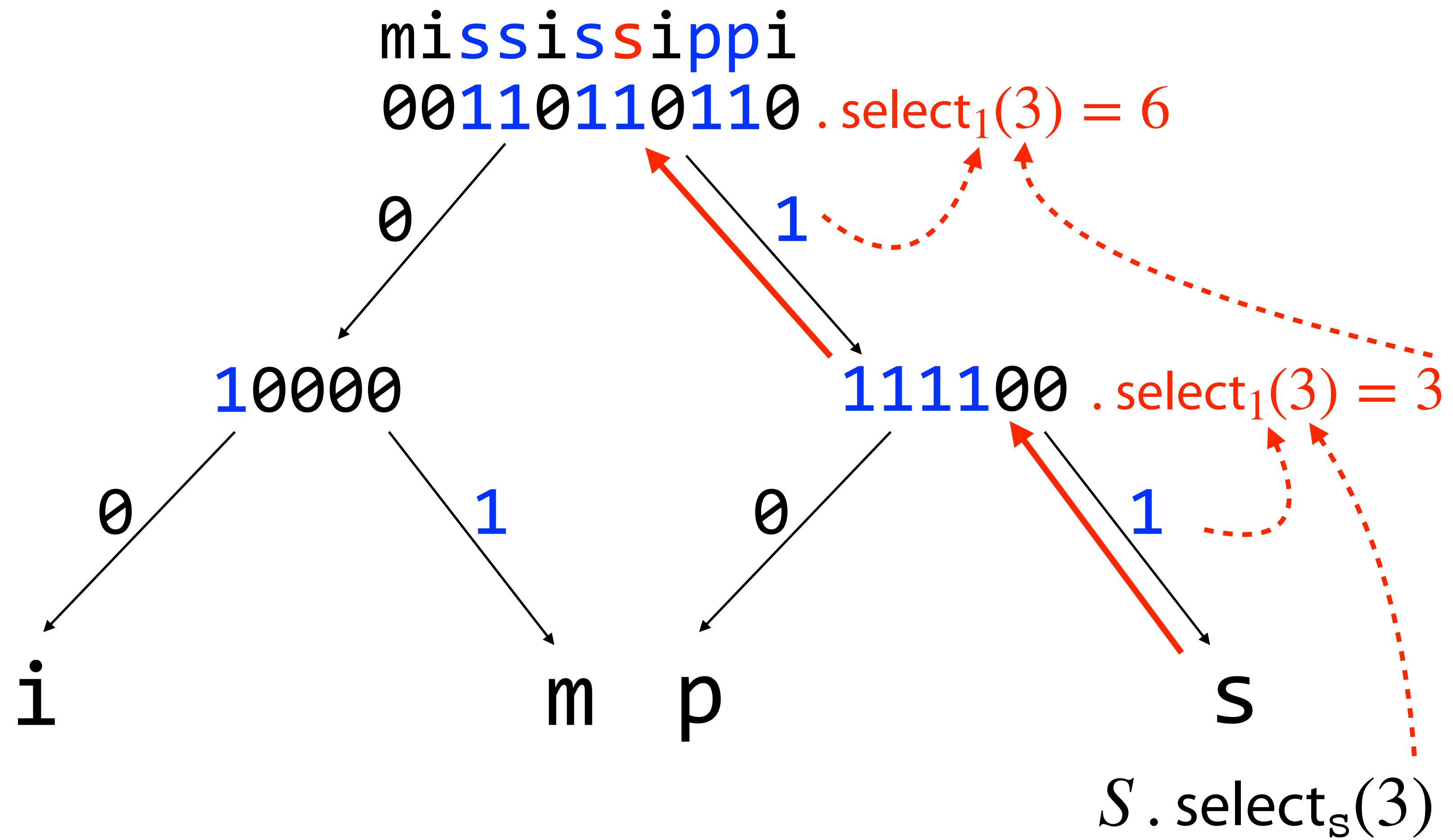
Wavelet trees

Answer: 6

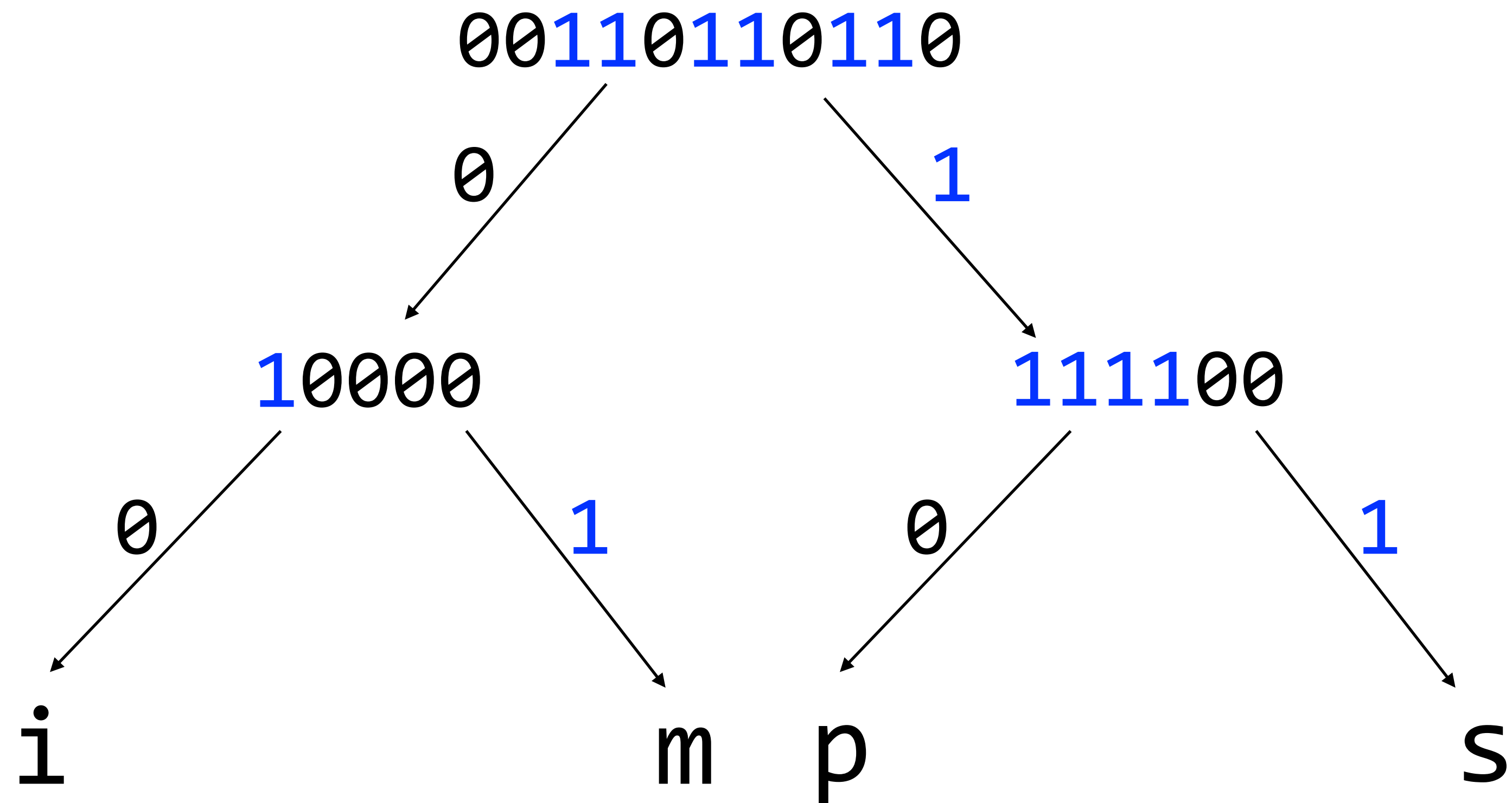


Wavelet trees

Answer: 6



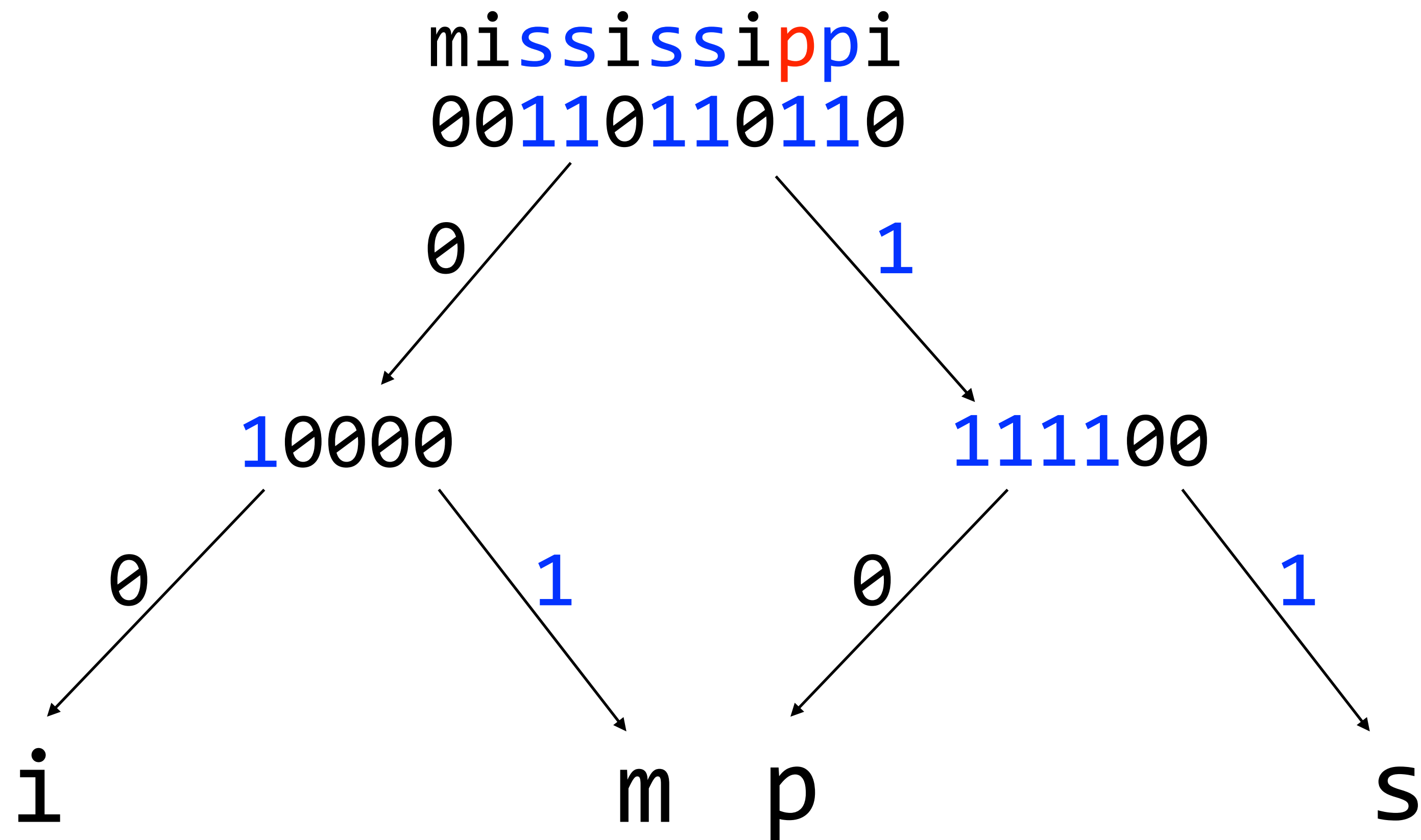
Wavelet trees



$S . \text{select}_p(0)$

Wavelet trees

Answer: 8



$S . \text{select}_p(0)$

Wavelet trees

Wavelet tree $\text{select}_x(i)$:

```
 $N \leftarrow \text{leaf}(x)$   
 $l \leftarrow |c(x)| - 1$   
while  $N$  is not root  
     $N \leftarrow N.\text{parent}()$   
     $B \leftarrow N.\text{bitvector}$   
     $b \leftarrow c(x)[k]$   
     $i \leftarrow B.\text{select}_b(i)$   
     $k \leftarrow k - 1$   
return  $i$ 
```

Wavelet trees

Wavelet tree $\text{select}_x(i)$:

Given character x and rank i :

$N \leftarrow \text{leaf}(x)$

$l \leftarrow |c(x)| - 1$

while N is not root

$N \leftarrow N.\text{parent}()$

$B \leftarrow N.\text{bitvector}$

$b \leftarrow c(x)[k]$

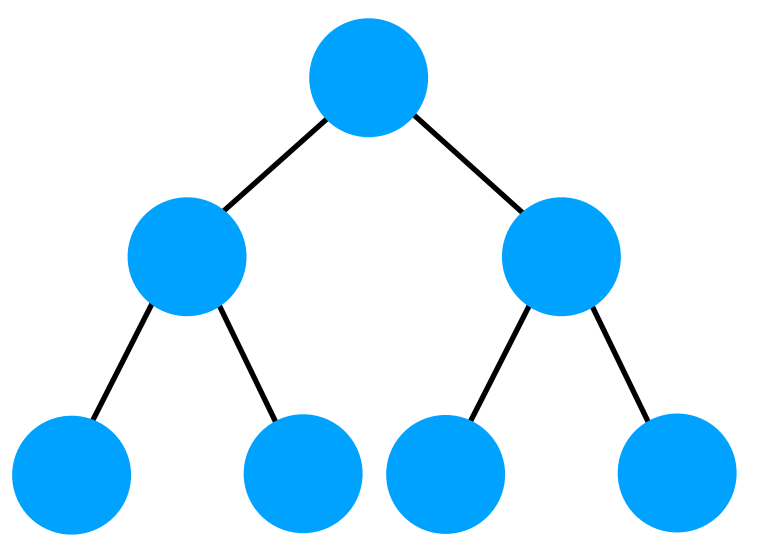
$i \leftarrow B.\text{select}_b(i)$

$k \leftarrow k - 1$

return i

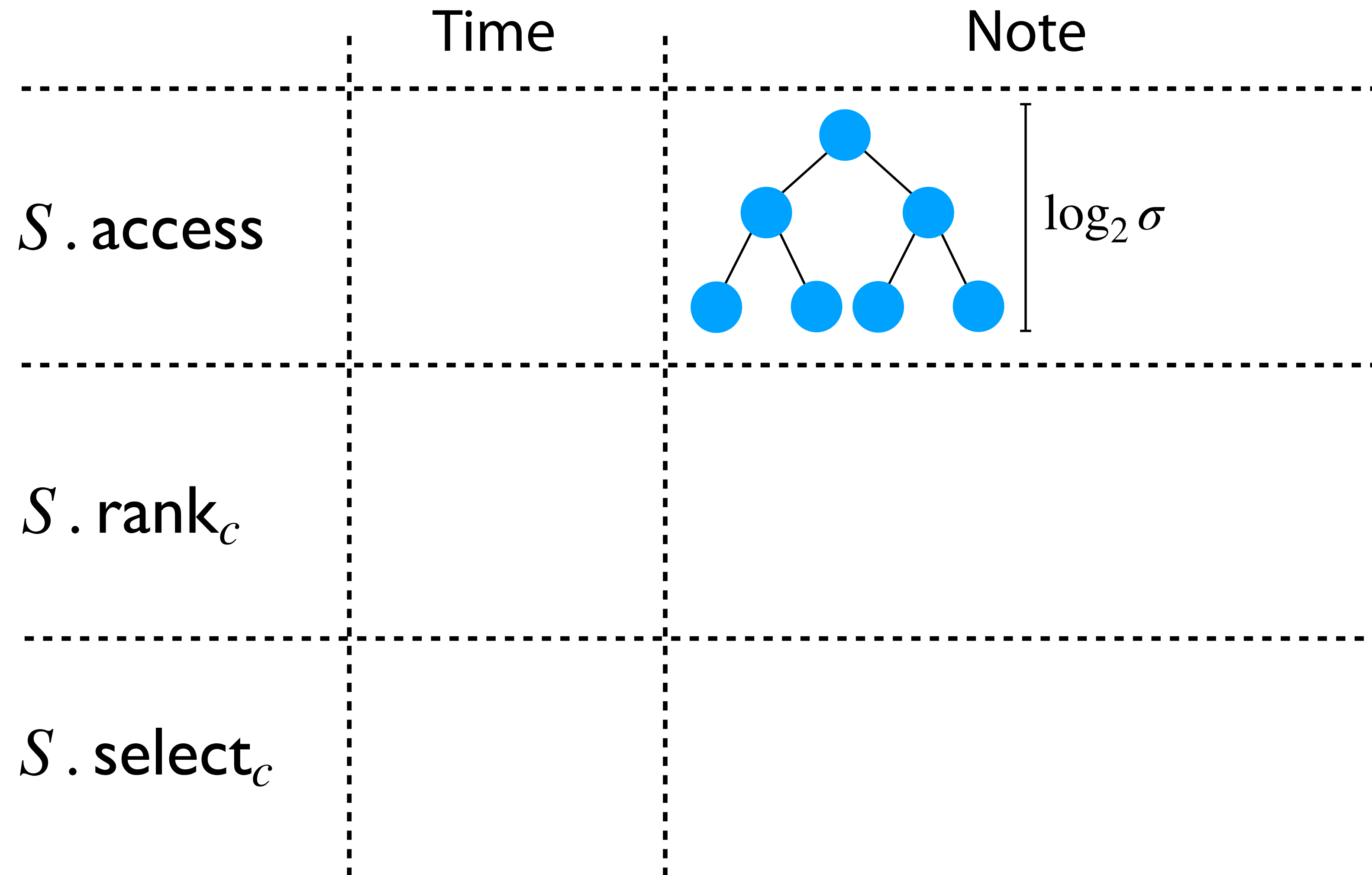
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$		
$S . \text{rank}_c$		
$S . \text{select}_c$		

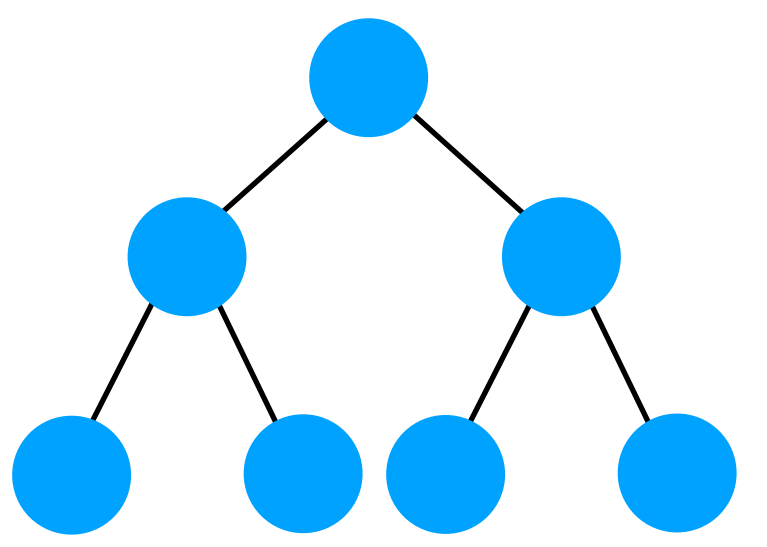
Wavelet trees

Assuming balanced tree



Wavelet trees

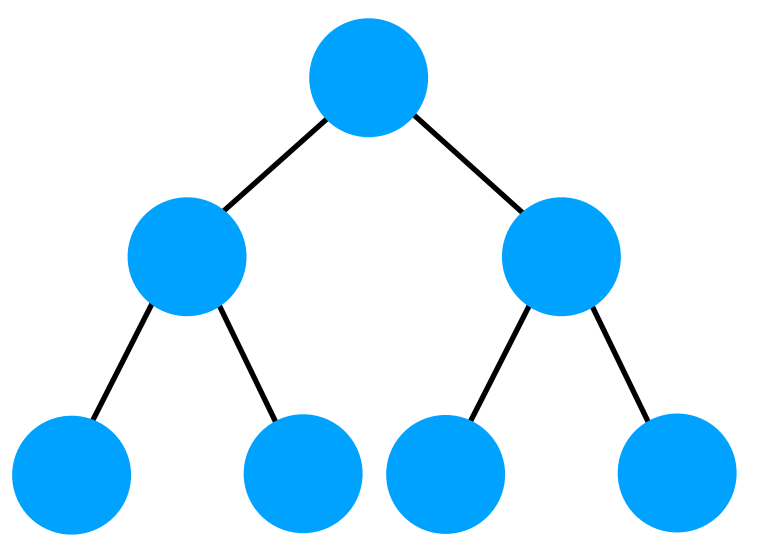
Assuming balanced tree

	Time	Note
$S . \text{access}$		 $\log_2 \sigma$
$S . \text{rank}_c$		
$S . \text{select}_c$		

Jacobson's
rank is
 $O(1)$ time

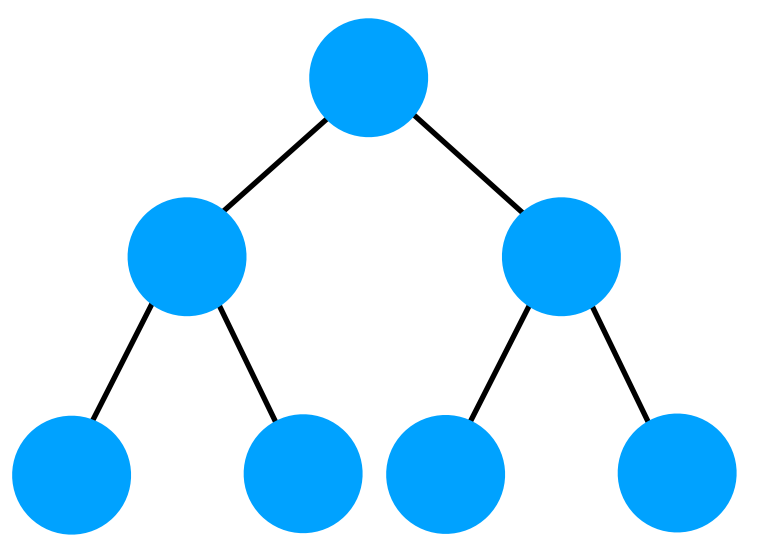
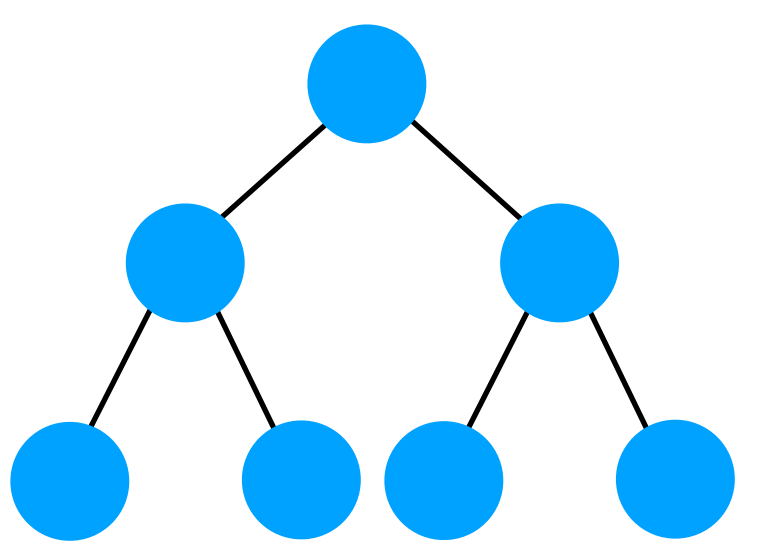
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$
$S . \text{rank}_c$		Jacobson's rank is $O(1)$ time
$S . \text{select}_c$		

Wavelet trees

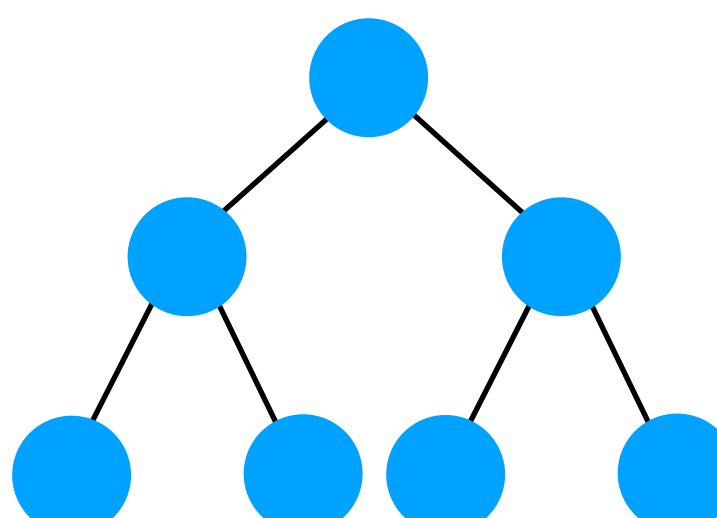
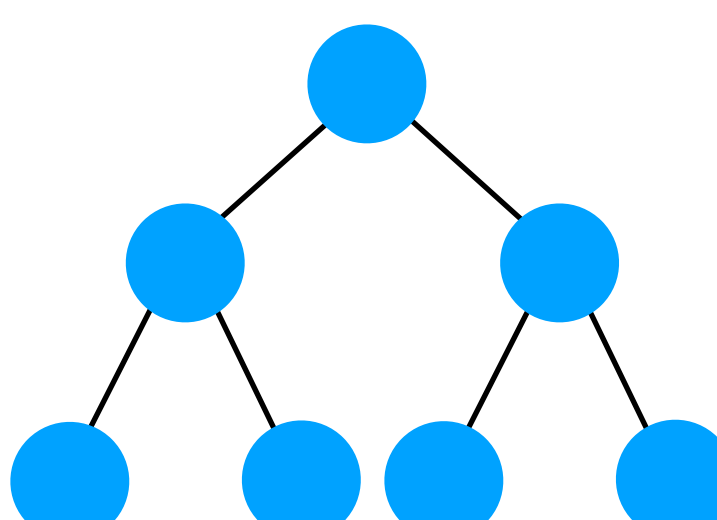
Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$
$S . \text{rank}_c$		
$S . \text{select}_c$		

Jacobson's
rank is
 $O(1)$ time

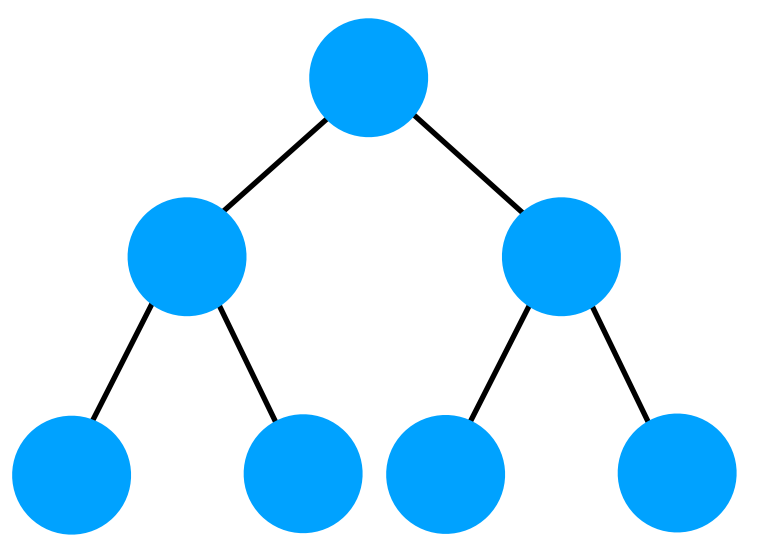
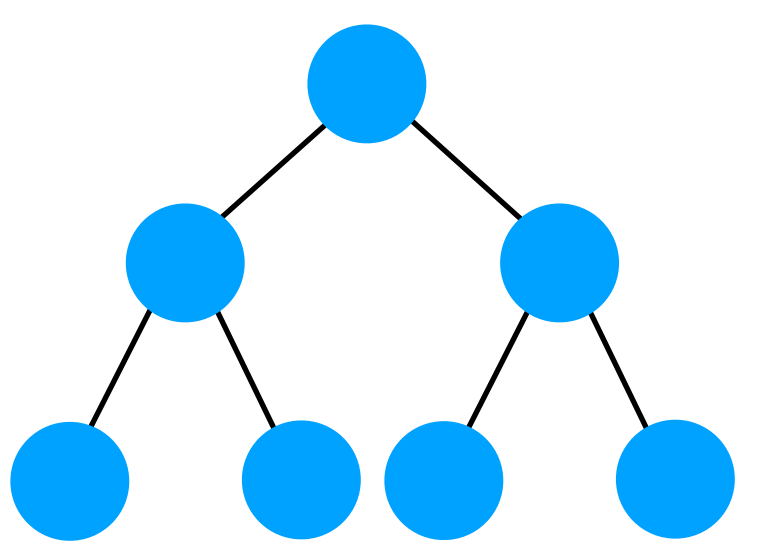
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{rank}_c$		 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{select}_c$		

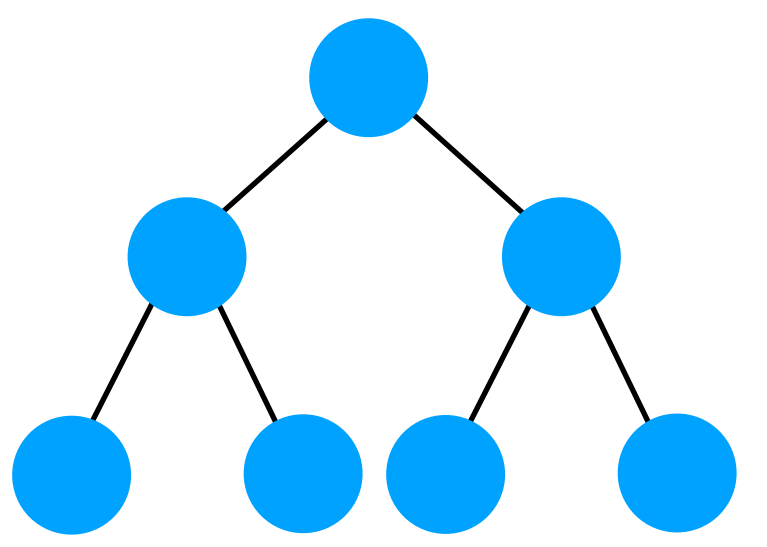
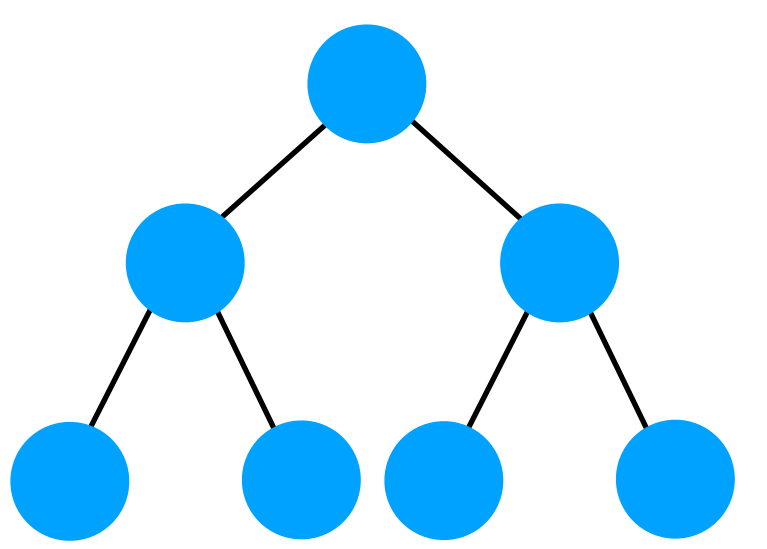
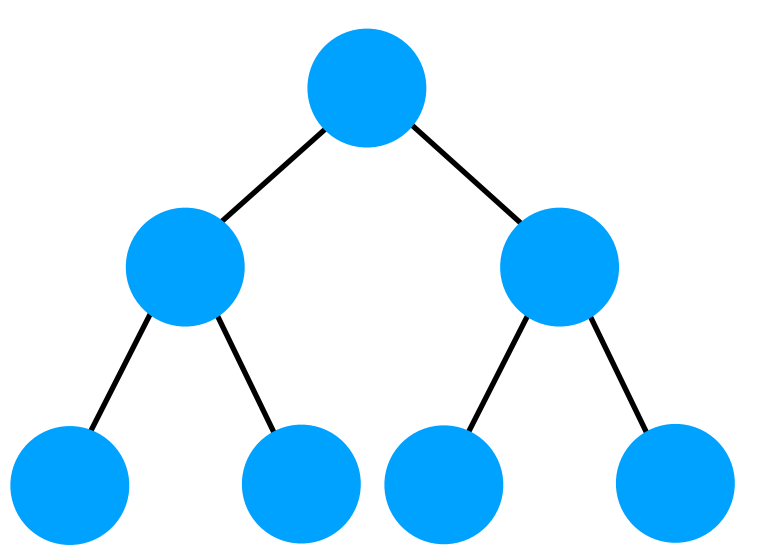
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{rank}_c$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{select}_c$		

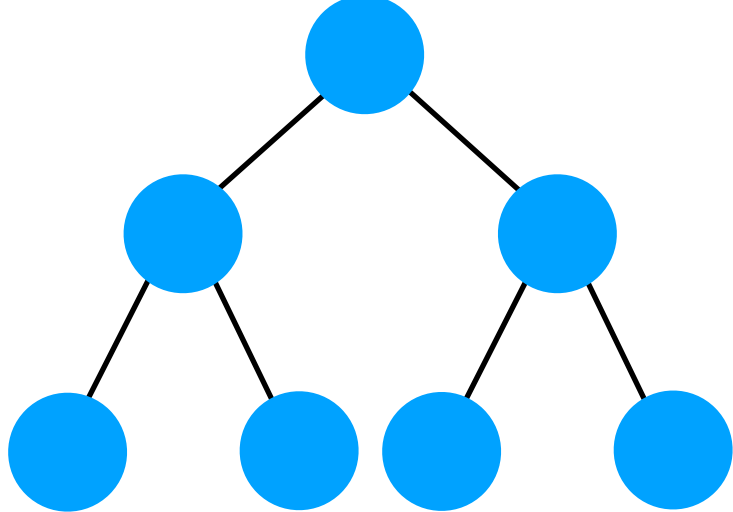
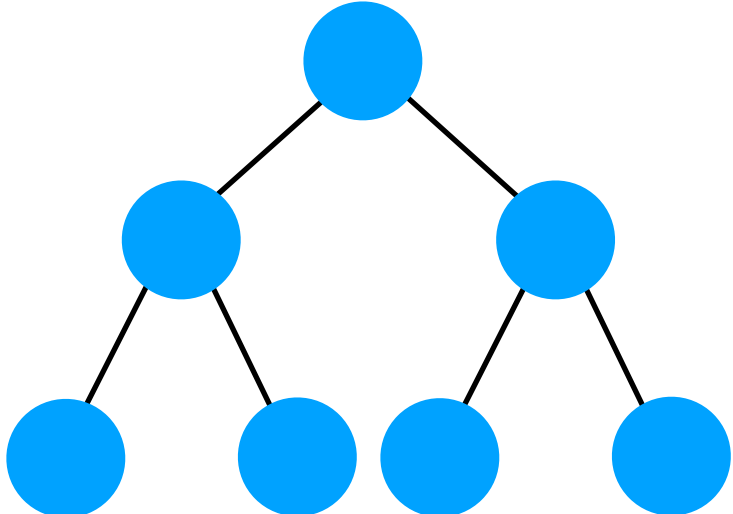
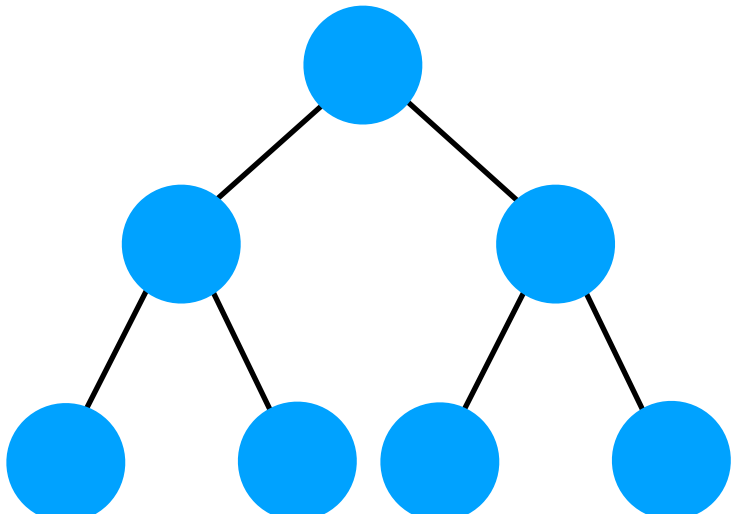
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{rank}_c$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{select}_c$		

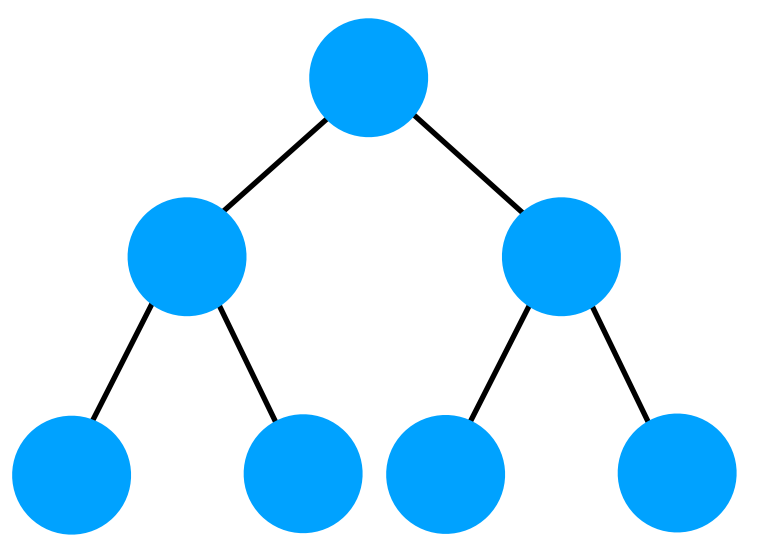
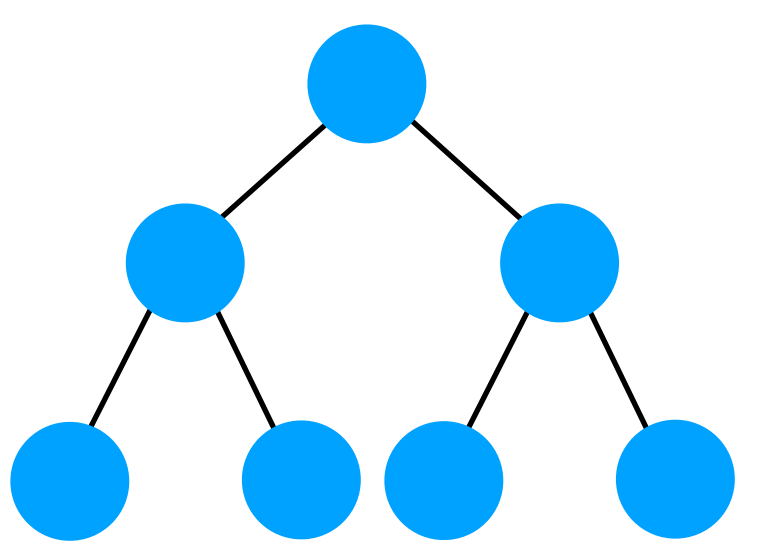
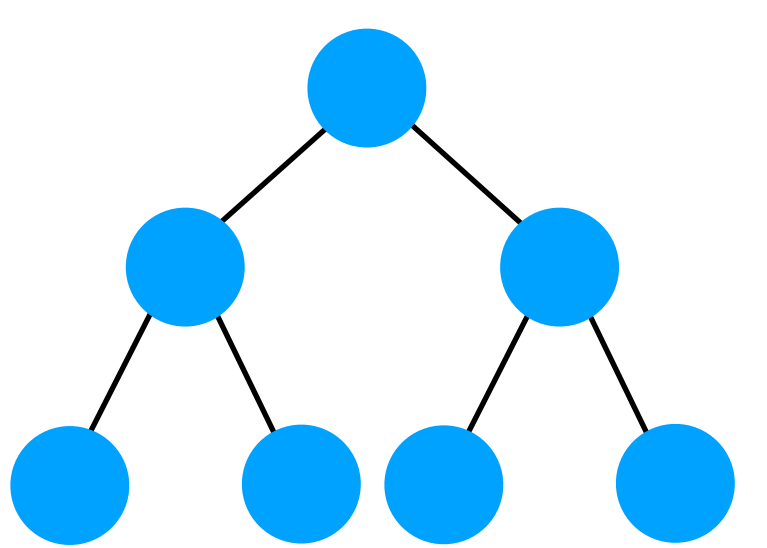
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{rank}_c$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{select}_c$		 $\log_2 \sigma$

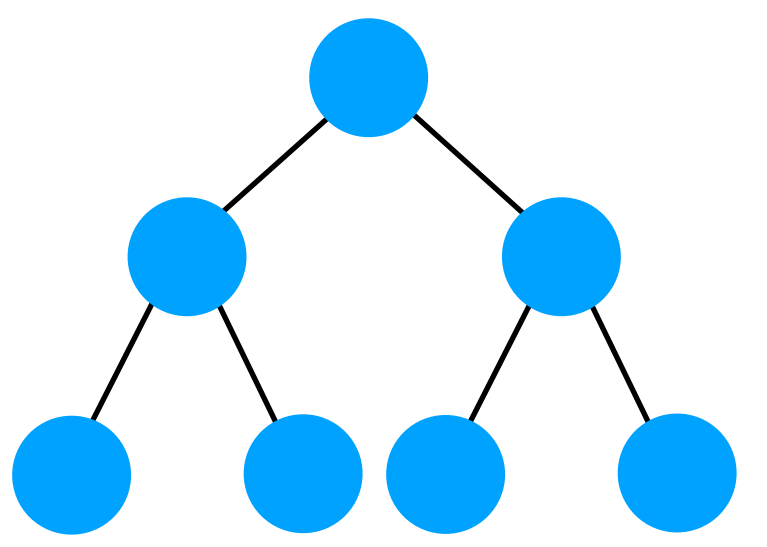
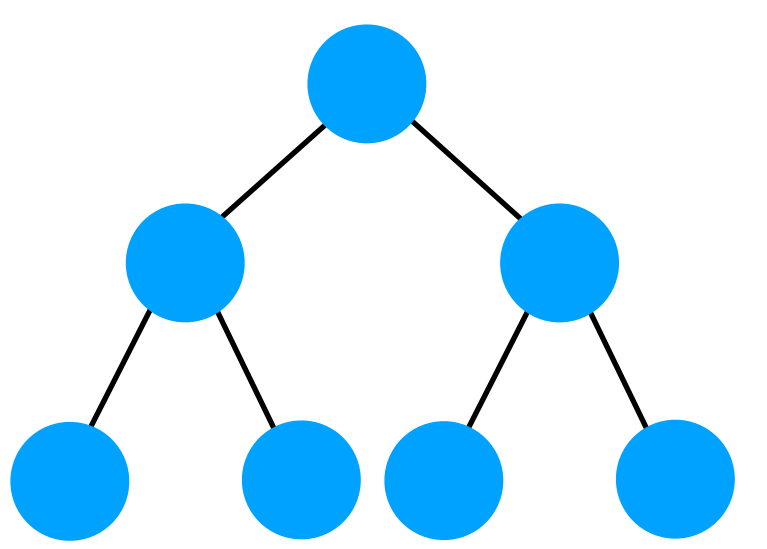
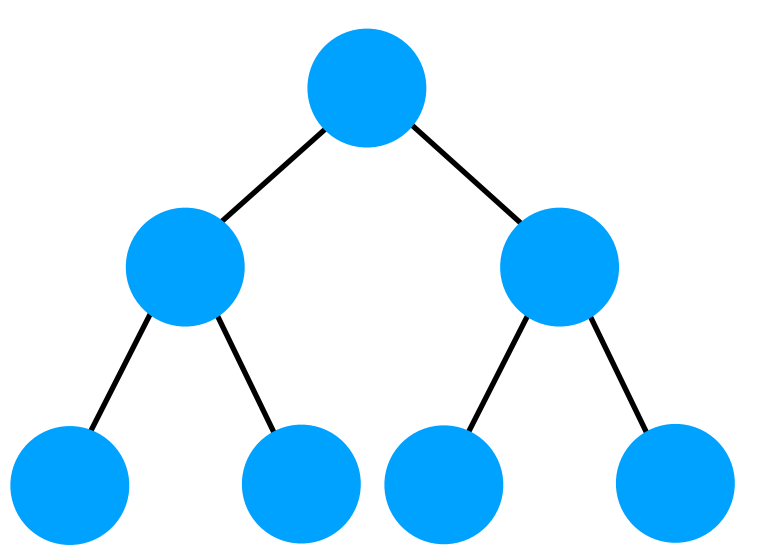
Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{rank}_c$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{select}_c$		 $\log_2 \sigma$ Clark's select is $O(1)$ time

Wavelet trees

Assuming balanced tree

	Time	Note
$S . \text{access}$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{rank}_c$	$O(\log \sigma)$	 $\log_2 \sigma$ Jacobson's rank is $O(1)$ time
$S . \text{select}_c$	$O(\log \sigma)$	 $\log_2 \sigma$ Clark's select is $O(1)$ time